Institución:	Universidad Autónoma de la Ciudad de México UACM Nada humano me es ajeno
Carrera:	Licenciatura en Ing. De Software
Ciclo:	2025-II
Grupo:	1101
Asignatura:	Arquitectura de software.
Título de actividad:	Sistema de comanda digital_AD (ISO/IEEE 42010_2011).
Alumnos:	Avilés Téllez Jean Darién. Osornio Tapia Jorge Edmundo. Rios Carrera Jesús Vicente. Vargas Angeles Uriel.
Matriculas:	19-011-1312 21-011-0194 19-011-0599 19-011-1318
Docente:	Máximo Eduardo Sánchez Gutiérrez

Índice.

1. Identificación y Visión General de la Arquitectura. 4

1.1 Identificación del sistema. 4

1.1.1. Mapa de distribución física del sistema. 5

1.2 Propósito y alcance. 6

1.3 Autores, fecha y estado del AD 7

1.4 Resumen general de la arquitectura. 8

1.5 Contexto del sistema. 10

2. Stakeholders y sus elementos de interes. 13

2.1 Stakeholders. 13

2.2 Elementos de interés de los stakeholders. 15

2.2.1 Cliente. 15

2.2.2 Mesero. 27

2.2.4 Administrador. 36

3. Puntos de Vista de la Arquitectura. 40

3.1 Punto de Vista de Contexto. 41

3.2 Punto de Vista Lógico. 47

3.3 Punto de Vista de Procesos. 51

3.4 Punto de Vista de Desarrollo. 56

3.5 Punto de Vista de Despliegue. 62

4. Vistas de Arquitectura. 68

4.1 Vista de Contexto. 69

4.2 Vista lógica. 71

4.3 Vista de Desarrollo. 76

4.4 Vista de Despliegue. 82

5. Correspondencias y Consistencia. 86

5.1 Relaciones entre vistas. 87

5.2 Reglas de Consistencia aplicadas en los modelos UML. 91

6. Decisiones y Racional de arquitectura. 94

6.1 Introducción general al proceso de decisiones. 94

6.2 Decisiones Arquitectónicas Principales. 95

6.3 Decisiones sobre Tecnologías y Plataformas. 97

6.4 Decisiones sobre Modelado y Datos. 99

6.5 Decisiones sobre la Estructura del Código y Modularidad. 101

6.6 Decisiones rechazadas o modificadas durante el proyecto. 103

6.7 Evaluación de Alternativas Consideradas 105

6.8 Impacto de las Decisiones en las Vistas Arquitectónicas. 108

6.9 Conclusiones del Racional Arquitectónico. 110

7. Escenarios de Evaluación Arquitectónica. 111

7.1 Escenario 1: Alta concurrencia de pedidos en hora pico. 112

7.2 Escenario 2: Caída temporal de la conexión Wi-Fi en la cocina. 112

7.3 Escenario 3: Inventario insuficiente en el momento de registrar una venta. 113

7.4 Escenario 4: Falla del servidor central. 113

7.5 Escenario 5: Pedido marcado como pagado sin pasar por caja. 114

7.6 Escenario 6: Flujo incorporado tardíamente – Interfaz de Caja. 114

8. Conclusiones Finales de la Arquitectura 115

1. Identificación y Visión General de la Arquitectura.

1.1 Identificación del sistema.

El Sistema de Comanda Digital (SCD, versión: 1.2.0) es una aplicación web diseñada para digitalizar y gestionar el proceso de toma y ejecución de pedidos en establecimientos de comida de tamaño pequeño a mediano (restaurantes y fonditas), escalable a establecimientos aún más grandes. El SCD se desplegará principalmente en tablets instaladas en cada mesa, a las que se asigna un identificador único de mesa (Id_Mesa) para vincular de forma inequívoca cada orden con su ubicación física. Los pedidos realizados por el cliente en la tablet se transmiten en tiempo real a un servidor central que almacena la información en una base de datos relacional y la presenta en una pantalla de cocina; adicionalmente se contempla la generación de un comprobante físico (ticket) mediante impresora conectada al sistema.

Los interesados principales en el sistema son: el cliente, quien realiza el pedido de forma autónoma desde la tablet; el mesero, que brinda asistencia cuando el cliente requiere apoyo en la mesa o prefiere que alguien tome su orden directamente; el cocinero, encargado de visualizar y preparar los pedidos en la pantalla de cocina; el administrador, responsable de la gestión de inventarios, reportes y control operativo; y el propietario del restaurante, que utiliza la información generada para la toma de decisiones estratégicas y de negocio.

Adicionalmente, se considera como stakeholders a Avilés Téllez Jean Darién, Osornio Tapia Jorge Edmundo, Rios Carrera Jesús Vicente y Vargas Angeles Uriel, en el papel de emprendedores, analistas y desarrolladores principales del sistema, quienes tienen como interés garantizar que el diseño arquitectónico sea técnicamente sólido, escalable y funcional.

Como sistemas externos relevantes se incluyen la base de datos, la impresora de tickets, y la red local del restaurante (LAN) que interconecta tablets, servidor y dispositivos auxiliares. En esta primera etapa de diseño se excluye la integración con métodos de pago en línea; se asume operación sobre red local y navegadores webs modernos en las tablets.

1.1.1. Mapa de distribución física del sistema.

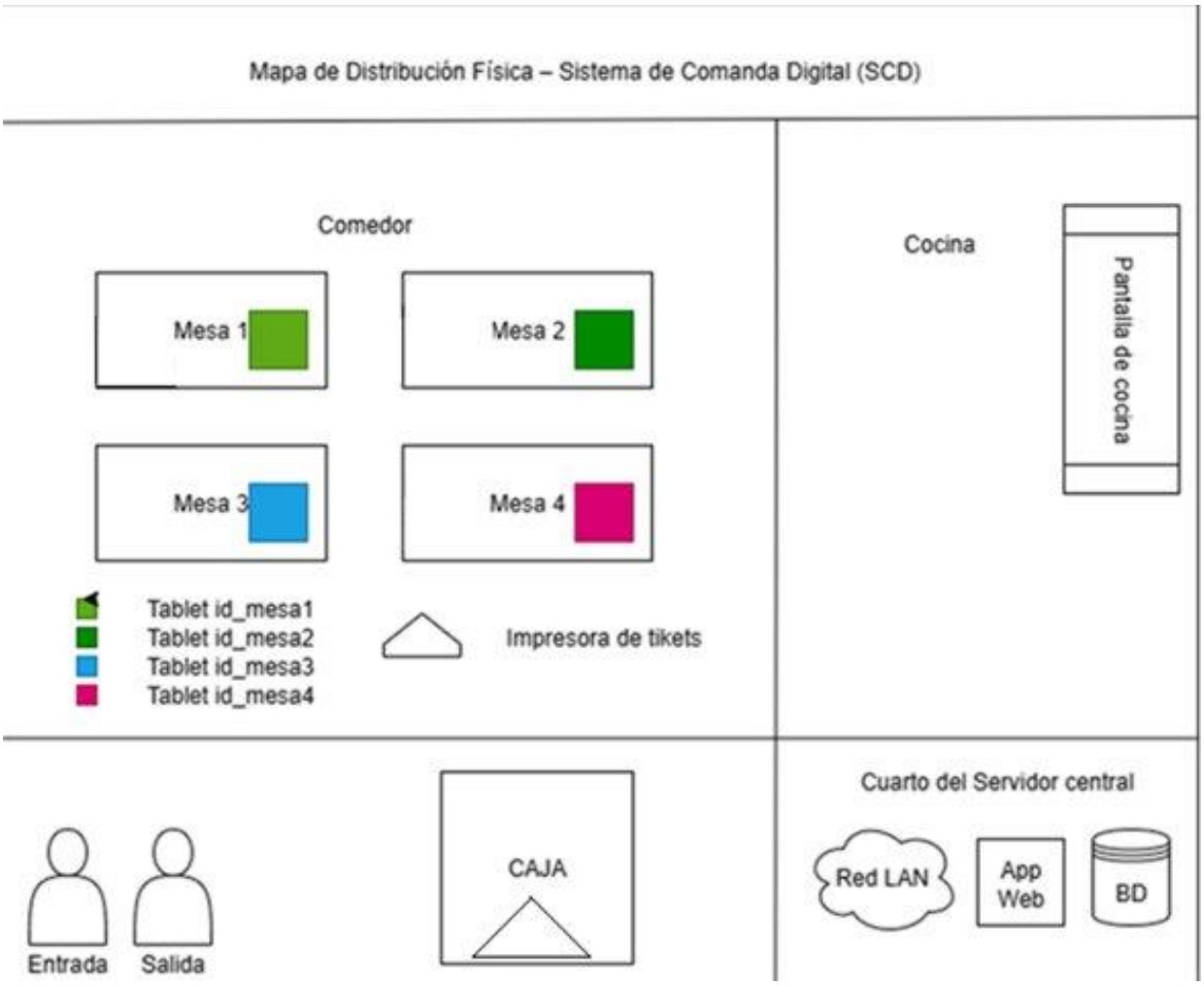


Figura 1.1: Mapa de distribución física propuesto para SCD.

La Figura 1.1 muestra la disposición física sugerida para el Sistema de Comanda Digital dentro de un restaurante de tamaño pequeño a mediano. Esta distribución fue diseñada considerando el flujo natural de trabajo, la seguridad de la información y la eficiencia en la operación diaria del negocio.

Las tablets se ubican estratégicamente en cada mesa del área de comedor, cada una con su identificador único (Id_Mesa), lo que permite a los clientes realizar pedidos personalizados sin depender directamente de un mesero. Esta ubicación reduce errores en la toma de pedidos y agiliza la comunicación con la cocina.

El servidor central se encuentra aislado en un cuarto seguro con acceso restringido, protegiendo la integridad de la aplicación web y la base de datos, además de minimizar riesgos por manipulación no autorizada. Esta decisión garantiza la estabilidad del sistema y la confidencialidad de la información.

En la cocina, la pantalla principal se sitúa en un lugar visible para el personal encargado de preparar los pedidos, permitiendo la visualización en tiempo real de las órdenes recibidas. Esto mejora la organización y disminuye las confusiones durante la preparación de alimentos.

El área de caja está ubicada cerca de la entrada y salida del restaurante, optimizando el flujo de clientes. Allí se integra la impresora de tickets, la cual permite validar cada pedido antes de proceder al cobro, asegurando que los procesos administrativos estén alineados con la operación del comedor.

Todos los dispositivos están interconectados mediante una red local (LAN), diseñada para proporcionar baja latencia y alta confiabilidad en la transmisión de datos. Esta infraestructura tecnológica asegura que la comunicación entre tablets, servidor, pantalla de cocina e impresora sea continua y sin interrupciones, incluso en momentos de alta demanda.

Esta distribución equilibra funcionalidad, ergonomía y seguridad, asegurando que el sistema pueda integrarse de manera natural en las operaciones diarias del restaurante y escalar en el futuro, permitiendo la incorporación de nuevas funcionalidades como analítica avanzada o pagos en línea.

1.2 Propósito y alcance.

El propósito de este documento es describir de manera estructurada y detallada la arquitectura propuesta para el Sistema de Comanda Digital (SCD), siguiendo las directrices del estándar ISO/IEC/IEEE 42010:2011. Su objetivo principal es proporcionar una representación clara y coherente de los elementos, relaciones y principios que conforman el sistema, permitiendo que todos los interesados comprendan su funcionamiento y evolución a lo largo del ciclo de vida del software.

Se busca servir como una guía técnica y de referencia tanto para la fase de diseño y desarrollo como para futuras actividades de mantenimiento y mejora del sistema. También pretende establecer un lenguaje común entre los stakeholders, facilitando la comunicación entre el desarrollador principal, el personal operativo del restaurante, y cualquier participante en la implementación del proyecto.

En cuanto a su alcance, este abarca únicamente el diseño arquitectónico del sistema web orientado a la digitalización del proceso de toma y gestión de pedidos dentro del restaurante. Se consideran los siguientes elementos y funcionalidades: Toma de pedidos mediante tablets instaladas en cada mesa, con identificación única por dispositivo (Id_Mesa). Personalización de pedidos por parte del cliente (agregar o quitar ingredientes, especificar cantidades, comentarios especiales). Comunicación en tiempo real de los pedidos hacia una pantalla de cocina central. Generación de tickets impresos para validación y cobro previo a la preparación. Función de asistencia para que el cliente pueda solicitar apoyo de un mesero desde la interfaz de la tablet. Gestión básica de inventario y reportes para el administrador.

Se excluyen de este alcance las siguientes funcionalidades: Procesamiento de pagos en línea o integración con sistemas bancarios. Funciones avanzadas de analítica empresarial o integración con otros sistemas externos de gestión. Implementaciones móviles nativas, considerando que el sistema se ejecutará únicamente en navegadores web modernos.

De esta manera, el documento se centra en definir la arquitectura base, sin entrar en detalles de implementación específicos, pero brindando una estructura clara que guiará el desarrollo, asegurando que las necesidades de los stakeholders sean atendidas y que la solución pueda evolucionar de forma ordenada y escalable.

1.3 Autores, fecha y estado del AD

El presente documento ha sido elaborado como parte del proyecto académico Sistema de Comanda Digital (SCD), desarrollado en la asignatura de Arquitectura de Software dentro del programa educativo de Ingeniería de Software, con el objetivo de aplicar los

lineamientos establecidos por el estándar ISO/IEC/IEEE 42010:2011 para la descripción de arquitecturas de sistemas de software.

Campo.	Descripción.
Proyecto.	Sistema de Comanda Digital (SCD).
Propósito.	Documentar la arquitectura del sistema siguiendo las directrices del estándar ISO/IEC/IEEE 42010:2011.
Autores principales.	Avilés Téllez Jean Darién. Osornio Tapia Jorge Edmundo. Rios Carrera Jesús Vicente. Vargas Angeles Uriel. – *Emprendedores, Analistas y Desarrolladores Principales del Sistema.*
Institución académica.	[Universidad Autónoma de la Ciudad de México] – *Programa Educativo de Ingeniería de Software.*
Asignatura.	Arquitectura de Software.
Fecha de elaboración.	Agosto - Noviembre 2025.
Versión del documento.	1.8.3 – Versión final.
Estado actual.	Finalización de la descripción de arquitectura. Sujeto a revisión y retroalimentación para consolidación final.

Tabla 1.3: Información General del AD.

1.4 Resumen general de la arquitectura.

El Sistema de Comanda Digital (SCD) se concibe como una aplicación web modular, diseñada bajo una arquitectura orientada a capas siguiendo el patrón Modelo-Vista-

Controlador (MVC). Esta estructura permite separar claramente la lógica de negocio, la interfaz de usuario y la gestión de datos, lo que facilita la escalabilidad, el mantenimiento y la evolución futura del sistema.

En su nivel más alto, el sistema está compuesto por tres elementos principales:

1. Interfaz de usuario en tablets.

Cada mesa del restaurante cuenta con una tablet conectada a la red local. A través de esta interfaz, el cliente puede explorar el menú, personalizar su pedido, por ejemplo, agregando o eliminando ingredientes y especificando cantidades, y solicitar asistencia de un mesero mediante un botón de ayuda.

Cada tablet tiene asignado un identificador único, Id_Mesa, que permite al sistema asociar el pedido a la mesa correspondiente y registrar su origen de forma precisa.

2. Servidor central y aplicación web.

El servidor recibe la información proveniente de las tablets y la procesa mediante la aplicación web, gestionando la comunicación entre la base de datos, la interfaz del cliente y la pantalla de cocina.

La aplicación está organizada en capas: Capa de presentación, que contiene las interfaces y páginas web visibles para el cliente, mesero, administrador y cocinero. Capa de lógica de negocio, que se encarga de procesar pedidos, validar datos, controlar inventarios y gestionar la comunicación con los diferentes roles del sistema. Capa de datos, responsable de almacenar de manera estructurada la información de pedidos, usuarios, mesas, inventarios y reportes en una base de datos relacional.

3. Pantalla de cocina y dispositivos auxiliares.

Los pedidos se visualizan en tiempo real en una pantalla instalada en el área de cocina, donde el cocinero puede identificar el platillo, la mesa y las personalizaciones solicitadas. Adicionalmente, se integra una impresora de tickets conectada al sistema, la cual genera un comprobante físico que valida la orden y permite al cliente realizar el pago antes de la preparación.

La comunicación entre estos elementos se lleva a cabo mediante la red local (LAN) del restaurante, garantizando baja latencia y confiabilidad en la transmisión de datos. Para futuras versiones del sistema, se contempla la posibilidad de habilitar conectividad en la nube para gestión remota y reportes en línea.

En su conjunto, esta arquitectura busca optimizar la experiencia tanto del cliente como del personal operativo, minimizando errores en la toma de pedidos, reduciendo tiempos de espera y mejorando el control interno del restaurante. El diseño modular asegura que el sistema pueda escalarse para incluir funcionalidades adicionales, como métodos de pago en línea o analítica avanzada, sin comprometer la estabilidad de la versión actual.

1.5 Contexto del sistema.

El Sistema de Comanda Digital (SCD) opera dentro del entorno de un restaurante, interconectando clientes, personal operativo y dispositivos tecnológicos mediante una red local. Su objetivo principal es digitalizar el proceso de toma y seguimiento de pedidos, optimizando la precisión, la rapidez y la trazabilidad de la información en todas las etapas del servicio.

Desde el punto de vista arquitectónico, el SCD se ubica en el centro del flujo operativo, funcionando como intermediario entre los clientes que realizan pedidos, el personal de apoyo que brinda asistencia y la cocina encargada de la preparación de los alimentos. Para delimitar correctamente su alcance, se distinguen dos categorías de elementos: internos y externos al sistema.

Los elementos internos son aquellos que forman parte directa del SCD, diseñados e implementados por el equipo de desarrollo. Entre ellos se encuentra la aplicación web principal alojada en un servidor central, la cual gestiona la comunicación entre todos los dispositivos. También forman parte de este grupo la interfaz de usuario desplegada en las tablets instaladas en cada mesa, que permite a los clientes explorar el menú, personalizar sus pedidos y solicitar asistencia, así como la pantalla de cocina donde se muestran en tiempo real las órdenes generadas. Complementan este grupo el módulo de administración, destinado exclusivamente al control de inventarios y generación de reportes, la base de datos relacional que almacena información clave del sistema como

pedidos, mesas, usuarios, platillos y stock, y finalmente el módulo de generación de tickets físicos para validar órdenes y habilitar el proceso de cobro.

Por otro lado, los elementos externos interactúan con el sistema, pero no son controlados directamente por él. En esta categoría se incluyen los clientes, quienes generan pedidos mediante las tablets, los meseros que brindan asistencia o ingresan pedidos de manera manual, los cocineros responsables de visualizar y preparar los pedidos, el administrador operativo que supervisa la información generada y el propietario del restaurante que utiliza los datos como apoyo para la toma de decisiones estratégicas. También se consideran externos la red local del restaurante, utilizada como medio para la transmisión de datos entre tablets, servidor y dispositivos auxiliares, así como el sistema de cobro físico que opera de forma independiente y se integra únicamente mediante la recepción del ticket impreso.

El flujo básico de interacción inicia cuando un cliente utiliza la tablet para explorar el menú y registrar su pedido. Este se procesa a través de la aplicación web, se almacena en la base de datos y se refleja simultáneamente en la pantalla de cocina. Una vez confirmado, el sistema genera un ticket físico que el cliente presenta en caja para realizar el pago. Finalmente, cuando el cocinero concluye la preparación, el pedido se marca como completado y queda registrado en el sistema para fines de control, seguimiento e inventario.

A continuación, en la Figura 1.5, se muestra el diagrama de contexto que representa el sistema explicado anteriormente, donde se muestran sus componentes principales, los actores externos que interactúan con él y la red tecnológica que permite la integración de dispositivos como tablets, pantallas de cocina e impresoras de tickets.

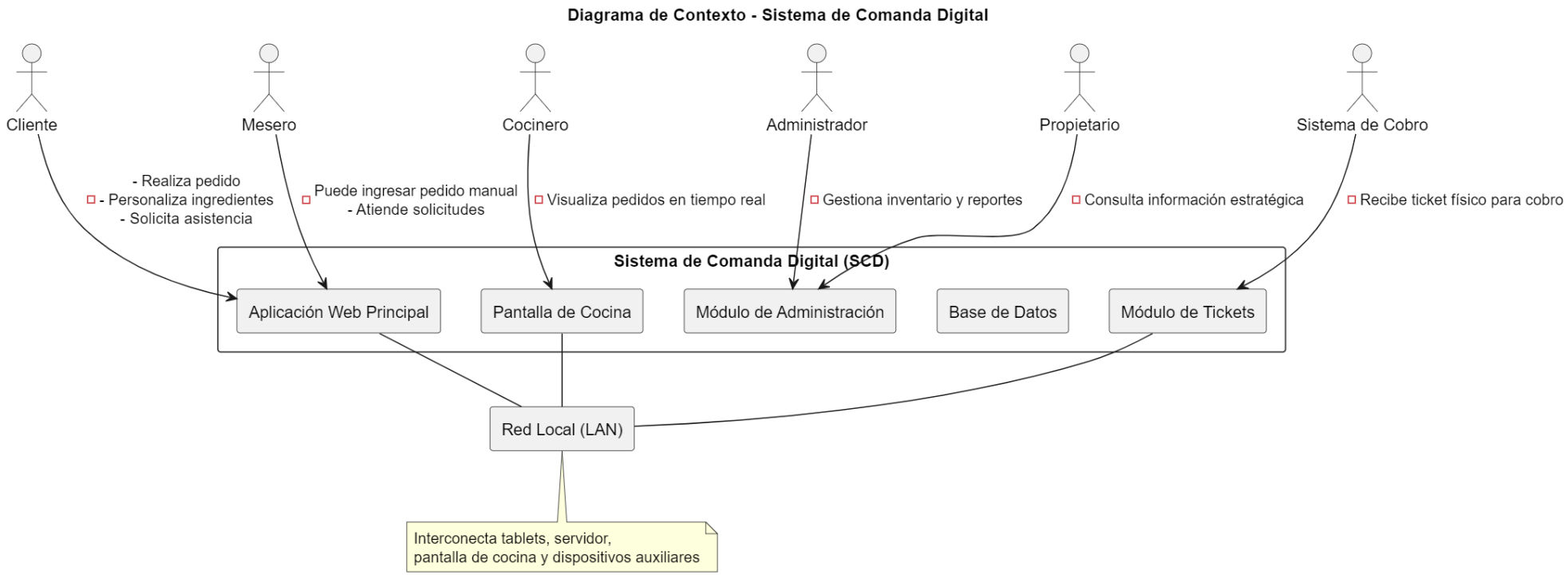


Figura 1.5: Diagrama de contexto del Sistema de Comanda Digital (SCD).

2. Stakeholders y sus elementos de interés.

2.1 Stakeholders.

Dentro del diseño y funcionamiento del Sistema de Comanda Digital (SCD) se identifican dos grandes grupos de stakeholders, Stakeholders Operativos y de Negocio. Estos son aquellos que interactúan directamente con el sistema en el día a día del restaurante y cuyo desempeño depende del correcto funcionamiento del SCD como se identificaron los siguientes.

1. Cliente.

Es el actor principal del sistema, ya que es quien realiza el pedido. Su interés se centra en que la experiencia de uso sea intuitiva, rápida y personalizada. A través de la tablet ubicada en cada mesa, el cliente puede consultar el menú, personalizar platillos, añadir comentarios especiales y confirmar su orden. Busca comodidad y autonomía, pero también tener la opción de pedir asistencia cuando lo necesite.

2. Mesero.

Actúa como intermediario entre el cliente y el sistema en situaciones donde el comensal prefiere atención personalizada. Su rol no desaparece, sino que se transforma en un soporte híbrido: puede tomar la orden manualmente desde su dispositivo, supervisar el estado de pedidos en curso y responder solicitudes de ayuda generadas desde las tablets.

3. Cocinero.

Es responsable de preparar los pedidos basándose en la información enviada desde el sistema. Su interacción con el SCD ocurre mediante una pantalla instalada en la cocina, donde recibe los pedidos en tiempo real, con detalle de mesa, platillos y personalizaciones. Además, puede actualizar el estado del pedido (ejemplo: *"En preparación"*, *"Listo para entregar"*) y consultar la disponibilidad de ingredientes, lo que garantiza un flujo más organizado.

4. Administrador.

Encargado de la gestión operativa del sistema, accede mediante un módulo de administración con mayores privilegios. Sus tareas incluyen el control de inventario, la supervisión de operaciones, la generación de reportes y el manejo de usuarios y permisos. Se convierte en el vínculo entre las operaciones del restaurante y la información estratégica que permite al negocio mantenerse eficiente.

5. Propietario del restaurante.

Su interacción es más indirecta y estratégica. El propietario accede principalmente a los reportes y análisis de datos generados por el módulo de administración, los cuales incluyen ventas, tendencias de consumo, platillos más solicitados y rendimiento general del restaurante. Con esta información puede tomar decisiones estratégicas relacionadas con marketing, menú, precios y expansión del negocio.

6. Equipo de desarrollo y emprendedores.

Son los responsables de diseñar, construir y mantener el sistema en óptimas condiciones técnicas.

Su objetivo principal es que el sistema cumpla con los requisitos de calidad, escalabilidad y robustez técnica. Están interesados en garantizar una arquitectura confiable que permita futuras mejoras, soporte de múltiples dispositivos y una experiencia de usuario consistente.

2.1.2 Comunicación de los Stakeholders con el Sistema.

Cada stakeholder interactúa con el SCD de manera distinta, en función de su rol y responsabilidades:

Cliente.

Comunicación: Tablet en la mesa con aplicación web intuitiva.

Funciones: Explorar menú, personalizar pedidos, añadir notas especiales, enviar orden a la cocina, solicitar ayuda a un mesero y recibir un ticket digital de validación.

Mesero.

Comunicación: Dispositivo con interfaz gráfica para gestión de pedidos.

Funciones: Tomar órdenes directamente, gestionar solicitudes de ayuda y supervisar el avance de pedidos en curso.

Cocinero.

Comunicación: Pantalla en la cocina conectada al sistema.

Funciones: Recibir pedidos detallados en tiempo real, cambiar el estado de preparación, verificar disponibilidad de ingredientes y coordinar la entrega a los meseros.

Administrador.

Comunicación: Módulo de administración con permisos avanzados.

Funciones: Controlar inventarios, monitorear operaciones, generar reportes e informes, y gestionar roles de usuarios.

Propietario.

Comunicación: Acceso indirecto mediante reportes consolidados.

Funciones: Analizar datos de ventas y desempeño, detectar tendencias y tomar decisiones estratégicas orientadas al crecimiento del negocio.

Todas ellas fueron la comunicación y función de cada stakeholder, como interactúan y como es el rol con el SCD, asimismo haciendo notar la responsabilidad de cada uno.

2.2 Elementos de interés de los stakeholders.

2.2.1 Cliente.

2.2.1.1 Errores en la toma de pedidos.

Explicación de la preocupación.

Para atender esta preocupación, el sistema propone el uso de tablets en cada mesa, a través de las cuales los clientes podrán realizar directamente su pedido. De esta manera, se elimina la posibilidad de errores de comunicación entre cliente y mesero, ya que la orden queda registrada de forma digital y exacta en el sistema. Además, la tablet permitirá

personalizar los platillos (por ejemplo, retirar o añadir ingredientes), garantizando que la cocina reciba la instrucción precisa y reduciendo significativamente la probabilidad de equivocaciones.

Se escogió la tablet porque elimina al intermediario (el mesero) del proceso inicial de toma de orden, permitiendo que el cliente ingrese su pedido directamente. Esto reduce casi por completo los errores humanos de transcripción, escritura ilegible o confusión.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura del sistema de las siguientes maneras:

1. Infraestructura de Red Local (LAN): La arquitectura tuvo que diseñarse en torno a una red local (LAN) robusta y confiable. Esta red es el pilar que interconecta todas las tablets de las mesas con el servidor central, la pantalla de cocina y otros dispositivos como la impresora de tickets. Se diseñó para ofrecer baja latencia y alta confiabilidad, asegurando una comunicación continua incluso en momentos de alta demanda.

2. Arquitectura Cliente-Servidor: La elección de tablets consolidó la necesidad de una arquitectura cliente-servidor. Las tablets actúan como los "clientes" del sistema, ejecutando una aplicación web a través de un navegador moderno. El "servidor" central se encarga de recibir los pedidos, procesar la lógica de negocio y gestionar la comunicación con la base de datos y la pantalla de cocina.

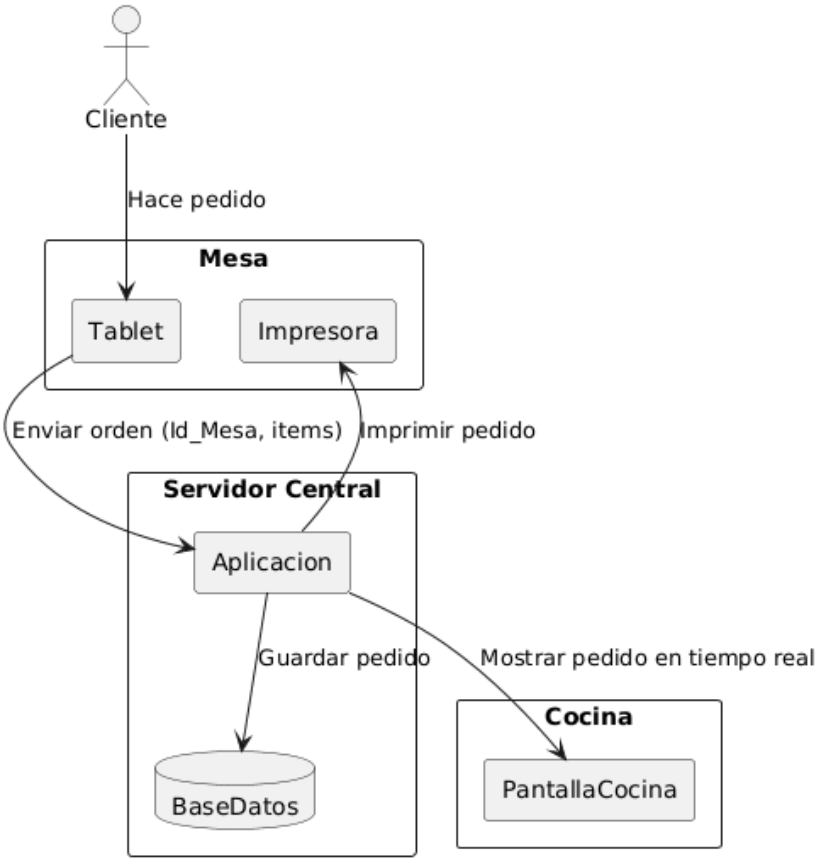
3. Identificación Única por Mesa (Id_Mesa): Para que el sistema funcione correctamente, fue crucial que cada tablet se asocie de forma inequívoca con una ubicación física. Esto llevó a la creación del concepto Id_Mesa, un identificador único asignado a cada tablet que permite al sistema saber exactamente de qué mesa proviene cada pedido. Este identificador es un dato esencial que se almacena con cada orden en la base de datos relacional.

4. Diseño Modular y en Capas (MVC): El sistema se diseñó bajo una arquitectura modular orientada a capas, siguiendo el patrón Modelo-Vista-Controlador (MVC). La Capa de Presentación (Vista) es la interfaz que se despliega en las tablets, diseñada para ser intuitiva para el cliente.

La Capa de Lógica de Negocio (Controlador) procesa las órdenes enviadas desde las tablets, valida los datos y gestiona la comunicación.

La Capa de Datos (Modelo) almacena de manera estructurada toda la información en una base de datos central. Esta separación facilita el mantenimiento y la escalabilidad del sistema.

5. Comunicación en Tiempo Real: La arquitectura tuvo que soportar la transmisión y visualización de datos en tiempo real. Cuando un cliente confirma un pedido en la tablet, la información se transmite de inmediato al servidor, se almacena en la base de datos y se refleja instantáneamente en la pantalla de la cocina. Esto es fundamental para agilizar la operación y cumplir con la promesa de rapidez del sistema



2.2.1.2 Falta de control y claridad al ordenar.

En este caso, el sistema garantiza que cada pedido se registre con un formato único y homogéneo, lo que elimina ambigüedades propias de la comunicación verbal. Este

registro estructurado incluye datos esenciales como el identificador de la mesa (Id_Mesa), los platillos seleccionados, las cantidades y las especificaciones adicionales. Además, antes de enviar un pedido a la cocina, el sistema ejecuta una serie de validaciones automáticas para confirmar que la información sea correcta y completa, evitando órdenes vacías o inconsistentes. Esto asegura que tanto el personal de cocina como el de servicio trabajen con la misma información clara y organizada.

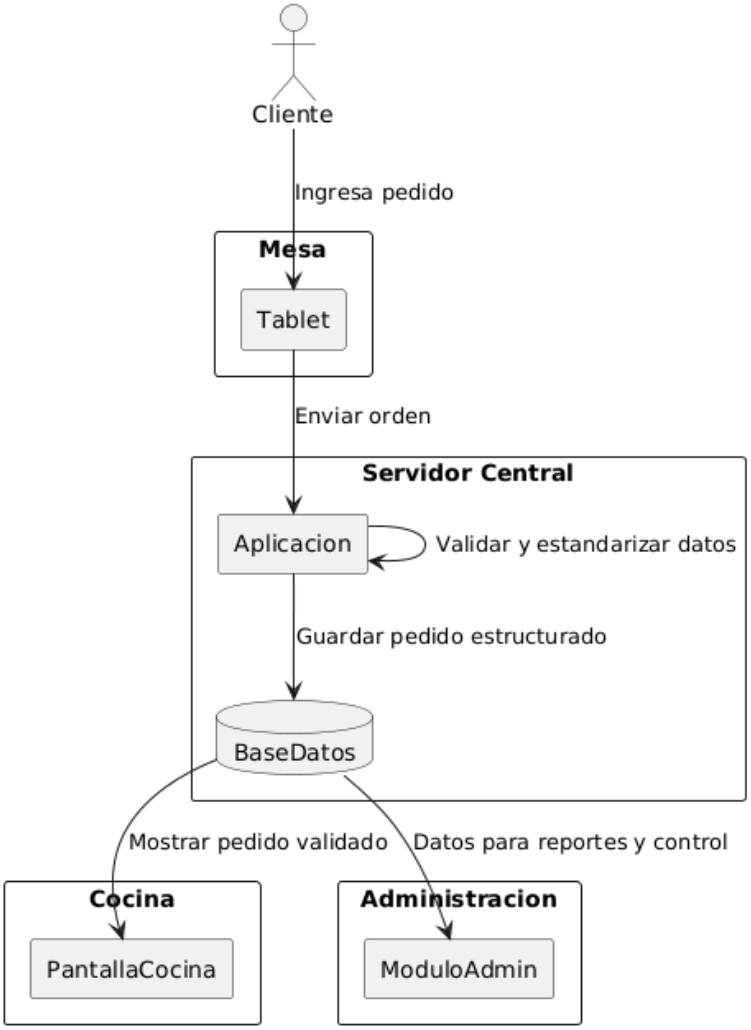
Se escogió la estandarización y validación del sistema porque la estandarización de la información permite generar reportes, métricas de consumo y trazabilidad de cada orden sin riesgo de datos incompletos o interpretaciones erróneas. La validación automática asegura que solo los pedidos viables lleguen a la cocina, lo que reduce los tiempos de aclaración y contribuye a una mayor eficiencia.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Base de Datos Centralizada: La consistencia de la información se logra gracias a que el sistema almacena todos los datos de los pedidos en una base de datos central de tipo relacional, que funge como la única fuente de verdad. Esto garantiza que todas las áreas involucradas (cocina, administración y caja) accedan a los mismos datos en tiempo real.

2. Consistencia entre Vistas: El diseño arquitectónico asegura que todas las vistas (interfaz del cliente, vista de cocina y módulo de administración) reflejen de manera coherente y actualizada la misma información.

3. Flujo Lógico de Comanda: El pedido se procesa a través de la aplicación web, se almacena en la base de datos y se refleja simultáneamente en la pantalla de cocina, garantizando la trazabilidad de la información.



2.2.1.3 Dificultad para personalizar los platillos.

El sistema está diseñado para incrementar la satisfacción del cliente al ofrecer la capacidad de personalizar los platillos. A través de la interfaz de la tablet, el cliente puede seleccionar qué ingredientes desea mantener, añadir o eliminar (útil para alergias o preferencias). También se permite ajustar el número de porciones con controles simples. Para casos especiales, se ofrece un campo de texto libre para añadir comentarios o instrucciones específicas (ej. “sin picante” o “salsa aparte”).

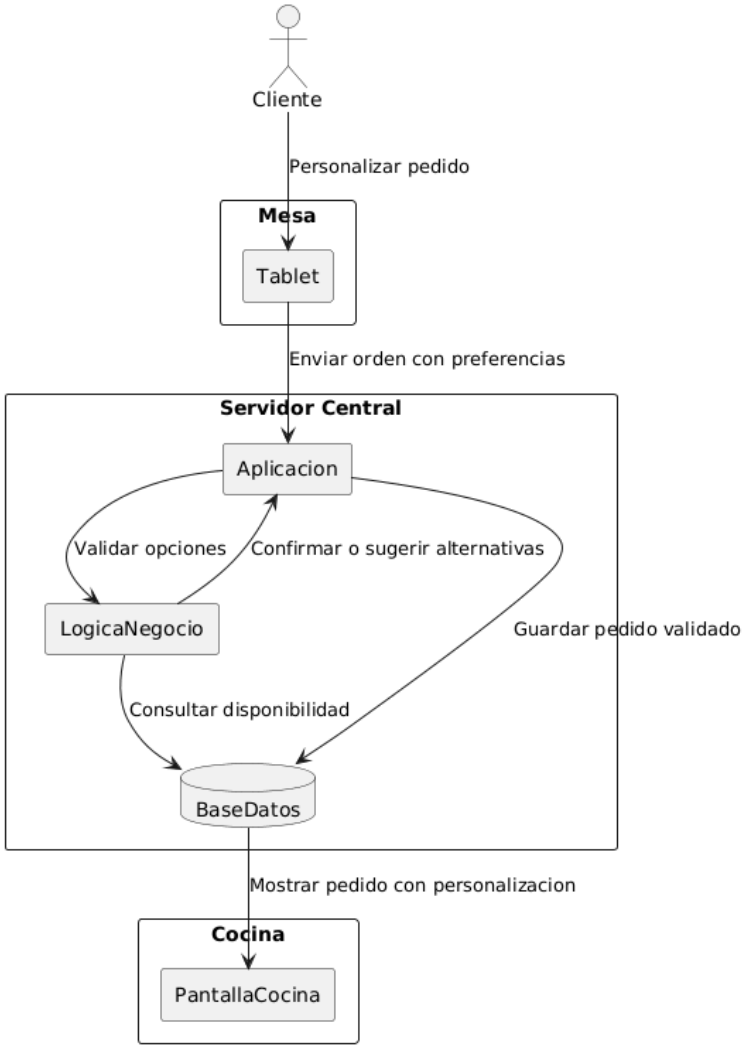
Se escogió la interfaz interactiva para personalización porque brinda flexibilidad al pedido, reduce errores en la cocina y asegura que el cliente se sienta escuchado y atendido en sus necesidades particulares. La interfaz es visual e interactiva para reducir errores.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Personalización de Pedidos (Alcance): La personalización de pedidos (agregar o quitar ingredientes, especificar cantidades, comentarios especiales) es una funcionalidad clave considerada en el alcance del diseño arquitectónico.

2. Lógica de Validación de Opciones: El sistema incluye mecanismos de control que verifican la disponibilidad de ingredientes y la coherencia de las personalizaciones antes de confirmar la orden. Si un ingrediente no está disponible, el sistema lo notifica y sugiere alternativas.

3. Capa de Lógica de Negocio (MVC): La gestión de estas reglas de personalización y validación recae en la Capa de Lógica de Negocio de la arquitectura, que se encarga de validar datos y procesar las órdenes.



2.2.1.4 Tiempos de espera largos.

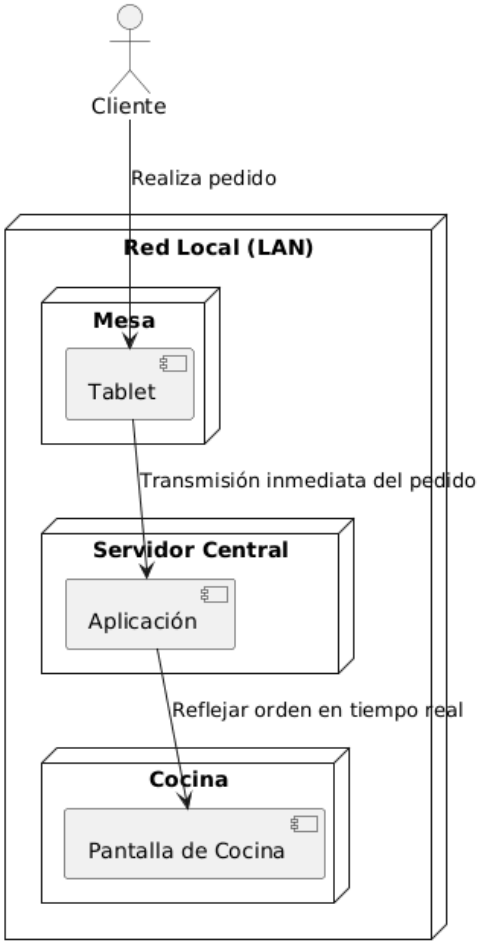
La rapidez en el procesamiento de la información es clave para asegurar un servicio ágil. Cuando el cliente realiza el pedido en la tablet, este se transmite de forma instantánea al servidor central y se refleja de manera inmediata en la pantalla de la cocina. Esto elimina el tiempo de espera asociado al traslado manual de la orden por parte del mesero, permitiendo que la cocina inicie la preparación sin demoras.

Se escogió la transmisión inmediata de pedidos porque optimiza la rotación de mesas, minimiza errores en la toma de pedidos y reduce los tiempos de espera, mejorando la experiencia del cliente y la eficiencia del personal operativo.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Red Local (LAN) de Baja Latencia: Para garantizar la comunicación continua y sin interrupciones, el sistema se implementa sobre una red local (LAN) diseñada para proporcionar baja latencia y alta confiabilidad en la transmisión de datos. Esta infraestructura tecnológica soporta la carga de múltiples dispositivos conectados simultáneamente.

2. Comunicación en Tiempo Real: La arquitectura fue diseñada para soportar la transmisión y visualización de datos en tiempo real. El servidor central procesa la información y la presenta en una pantalla de cocina inmediatamente.



2.2.1.5 Pedir algo que no está disponible.

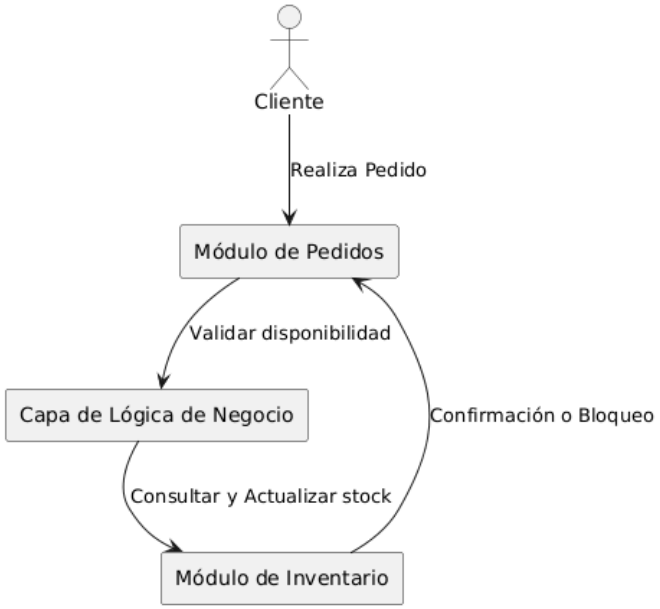
Esta preocupación se resuelve mediante la Actualización del Inventario en Tiempo Real. Cada vez que se confirma un pedido, el sistema descuenta automáticamente los ingredientes del inventario. Si un ingrediente se agota, el sistema puede bloquear automáticamente los pedidos que lo requieran y notificar al usuario sobre la falta de disponibilidad. Esto evita que el cliente pida un platillo que ya no puede prepararse.

Se escogió la actualización automática del inventario porque permite que la información sobre la disponibilidad de platillos y materias primas esté siempre actualizada, evitando situaciones incómodas y optimizando la operación interna del restaurante.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Módulo de Inventario Integrado: El Módulo de Inventario forma parte de la estructura lógica del sistema y controla los ingredientes y productos disponibles. El módulo de pedidos interactúa con el inventario y lo actualiza automáticamente al confirmarse un pedido.

2. Validación de Disponibilidad: La Capa de Lógica de Negocio valida el pedido antes de procesarlo, revisando la disponibilidad de platillos en inventario.



2.2.1.6 Necesidad de asistencia.

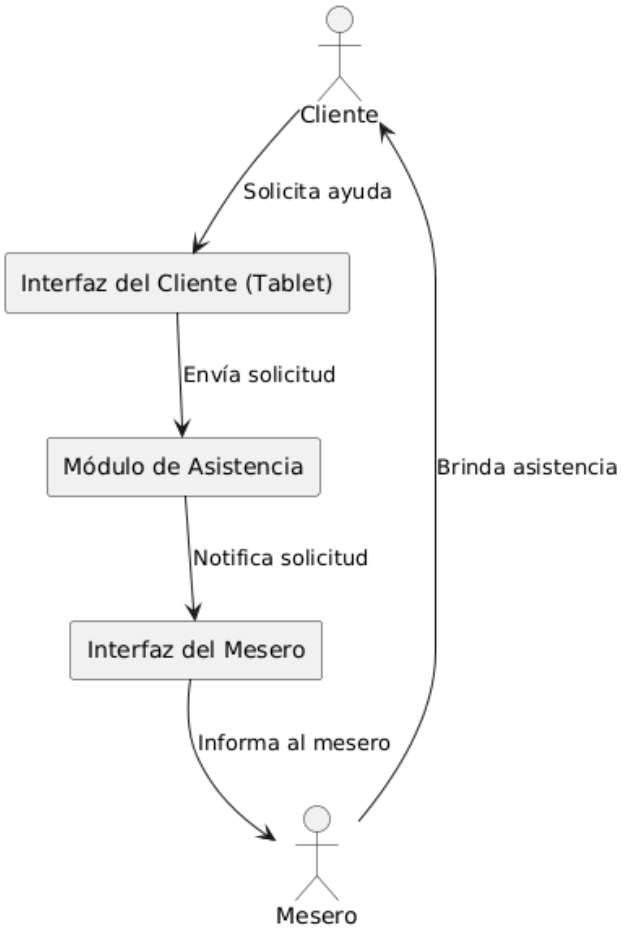
Aunque el sistema promueve la autonomía del cliente para realizar el pedido, se reconoce que puede requerir apoyo. El diseño contempla una Función de asistencia para que el cliente pueda solicitar el apoyo de un mesero desde la interfaz de la tablet. El mesero actúa como soporte híbrido, listo para asistir al cliente cuando lo necesite.

Se escogió mantener el rol del mesero como soporte porque garantiza una experiencia de usuario completa y flexible, permitiendo la toma de órdenes manualmente si el cliente lo prefiere, o gestionando las solicitudes de ayuda generadas digitalmente.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Interesados Definidos: El mesero es un interesado principal (stakeholder) en el sistema. Su rol incluye brindar asistencia cuando el cliente requiere apoyo o prefiere que alguien tome su orden directamente.

2. Interfaz del Mesero: El mesero requiere un dispositivo con interfaz gráfica para gestión de pedidos, lo que le permite supervisar el estado de pedidos en curso y responder a las solicitudes de ayuda



2.2.1.7 Validación de la orden y el cobro.

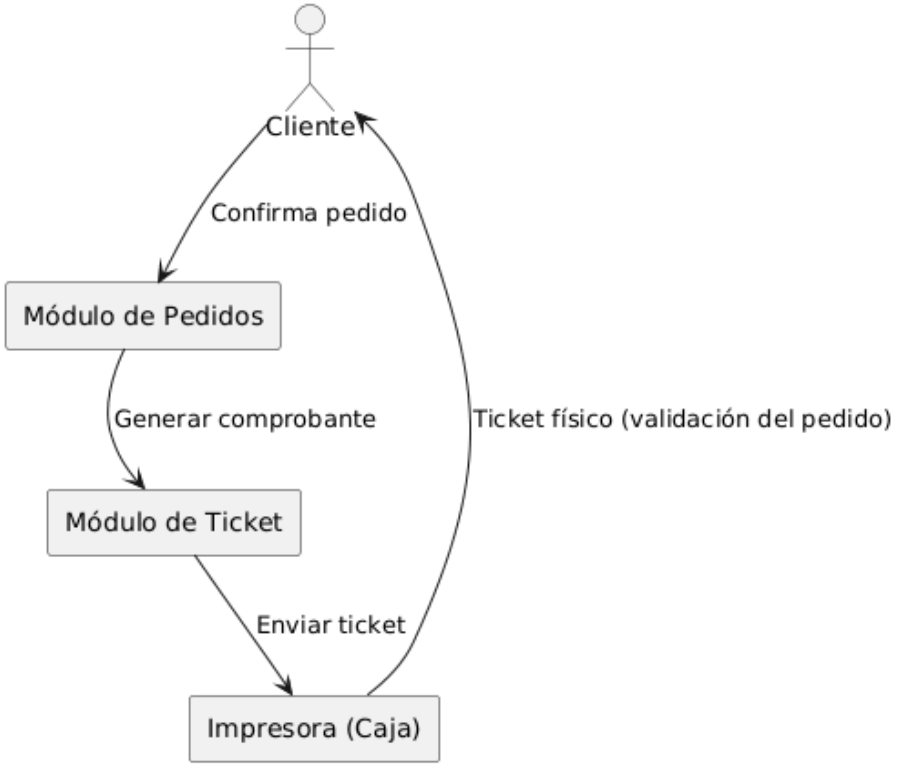
Se requiere un mecanismo para validar la orden y asegurar que el cobro sea preciso. El sistema genera un comprobante físico (ticket) una vez que el cliente confirma su pedido. Este ticket sirve para validar la orden antes de que inicie la preparación en la cocina. La impresora de tickets se ubica en el área de caja, optimizando el flujo de clientes y permitiendo validar cada pedido antes de proceder al cobro.

Se escogió la generación del ticket impreso porque refuerza la coherencia entre los procesos administrativos (cobro) y las operaciones del restaurante (preparación del platillo). Además, proporciona una constancia clara para el cliente con un desglose detallado de su solicitud.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Módulo de Ticket: Existe un Módulo de Ticket dentro del sistema que emite comprobantes detallados por cada pedido.

2. Dispositivos Auxiliares: La impresora de tickets se considera un sistema externo relevante que se integra al sistema mediante la red local (LAN).
3. Proceso de Cobro (Exclusión): Se excluye de este alcance la integración con métodos de pago en línea o sistemas bancarios; la operación se asume mediante un sistema de cobro físico que opera de forma independiente y se integra únicamente mediante la recepción del ticket impreso.



2.2.2 Mesero.

2.2.2.1 Transformación del rol y necesidad de asistencia.

El sistema fue diseñado para promover la autonomía del cliente, pero reconoce la necesidad de asistencia. El Mesero debe mantener un rol como soporte híbrido, estando listo para brindar asistencia cuando el cliente lo solicite o prefiera que alguien tome su orden directamente. Esto requiere un mecanismo para que el cliente pueda solicitar el apoyo de un mesero desde la interfaz de la tablet.

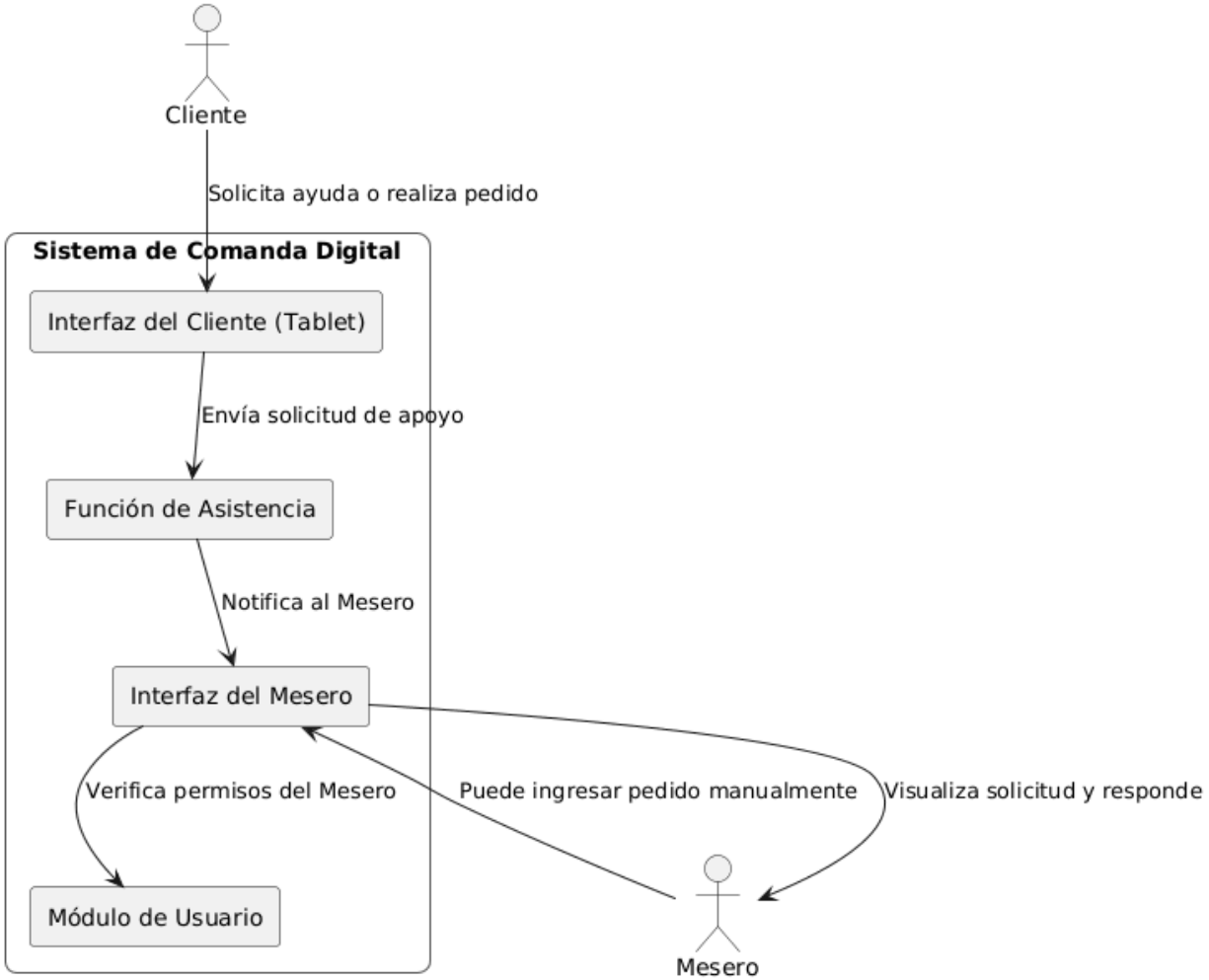
Se escogió el rol de soporte híbrido porque garantiza una experiencia de usuario completa y flexible, permitiendo la toma de órdenes manualmente si el cliente lo prefiere. El sistema implementa una Función de asistencia en la interfaz de la tablet para que el cliente pueda solicitar el apoyo de un mesero.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. El Mesero es un interesado principal (stakeholder) en el sistema. Su rol incluye la capacidad de ingresar pedidos de manera manual y gestionar solicitudes de ayuda.

2. Interfaz Específica: El Mesero requiere de un dispositivo con interfaz gráfica para gestión de pedidos. Esta interfaz le permite supervisar el estado de pedidos en curso y responder a las solicitudes de ayuda generadas digitalmente.

3. Módulo de Usuario: El sistema contempla un Módulo de Usuario que gestiona los roles y permisos, asegurando que el Mesero solo tenga acceso a las funciones que le corresponden (supervisar pedidos, apoyar en la atención al cliente y gestionar solicitudes adicionales, no funciones administrativas)



2.2.2.2 Necesidad de seguimiento y supervisión de pedidos.

El Mesero es responsable de la coordinación de la entrega del platillo y, por lo tanto, necesita supervisar el estado de pedidos en curso para mantener al cliente informado y garantizar un servicio ágil. Requiere saber con precisión cuándo un platillo ha finalizado su preparación y está listo para ser servido.

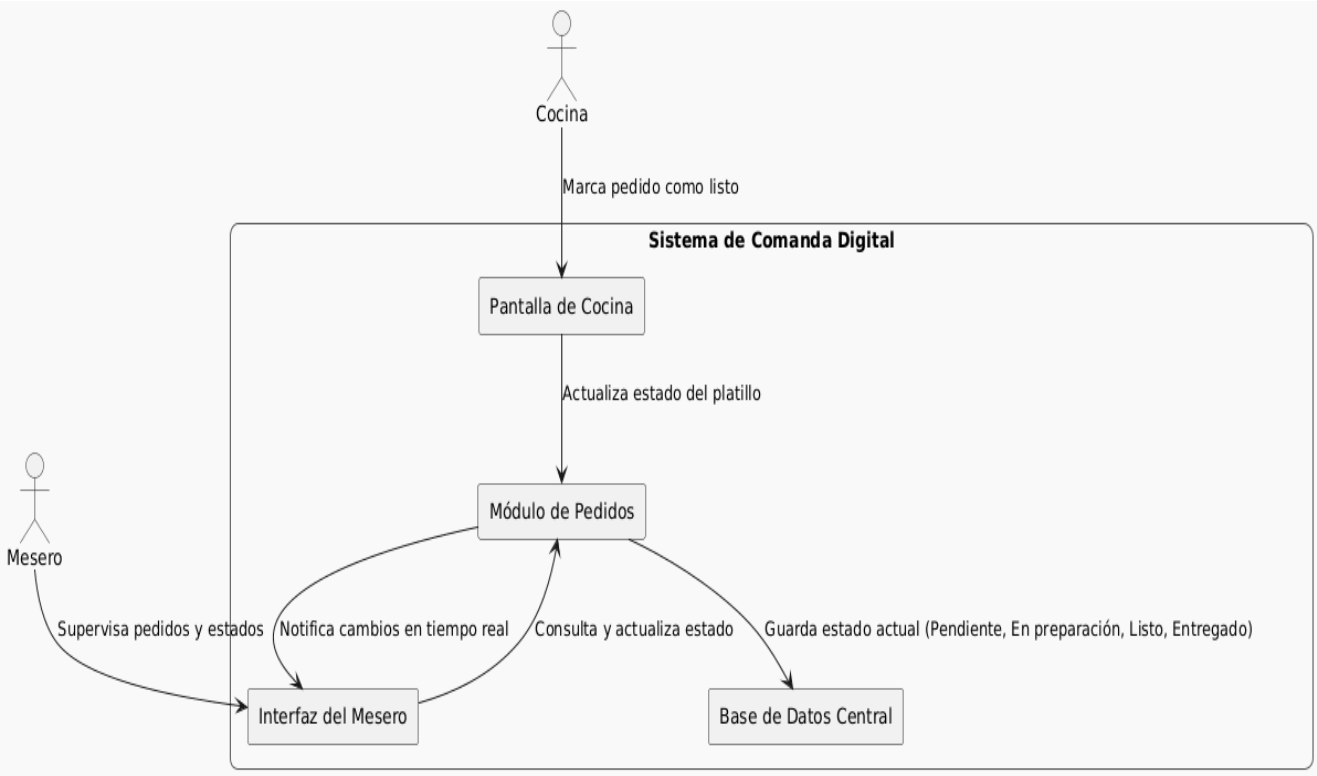
Se escogió el seguimiento de estados en tiempo real porque optimiza la eficiencia del servicio al asegurar que el Mesero pueda coordinar la entrega en el momento justo, lo que reduce la rotación de mesas y mejora la experiencia del cliente. La trazabilidad es crucial.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Módulo de Pedidos: El sistema de Pedidos facilita la creación de pedidos que avanzan por distintos estados a medida que se procesan. Los estados clave son: Pendiente, En preparación, Listo para entrega, y Entregado.

2. Consistencia de la Información: El sistema almacena la información de los pedidos en una base de datos central de tipo relacional. El diseño arquitectónico asegura que todas las vistas, incluida la del Mesero, reflejen de manera coherente y actualizada la misma información.

3. Comunicación en Tiempo Real: La Red Local (LAN) de baja latencia asegura que la comunicación entre los dispositivos sea continua y sin interrupciones, permitiendo la actualización de estados en tiempo real.



2.2.3 Cocinero.

El Cocinero necesita recibir la información del pedido de manera clara, precisa y sin ambigüedades para iniciar la preparación. El sistema elimina la dependencia de la comunicación verbal y la transcripción manual, garantizando que cada pedido se registre con un formato único y homogéneo. Esto incluye datos esenciales como el identificador de la mesa (Id_Mesa), los platillos y las personalizaciones detalladas solicitadas por el cliente (como añadir o eliminar ingredientes)

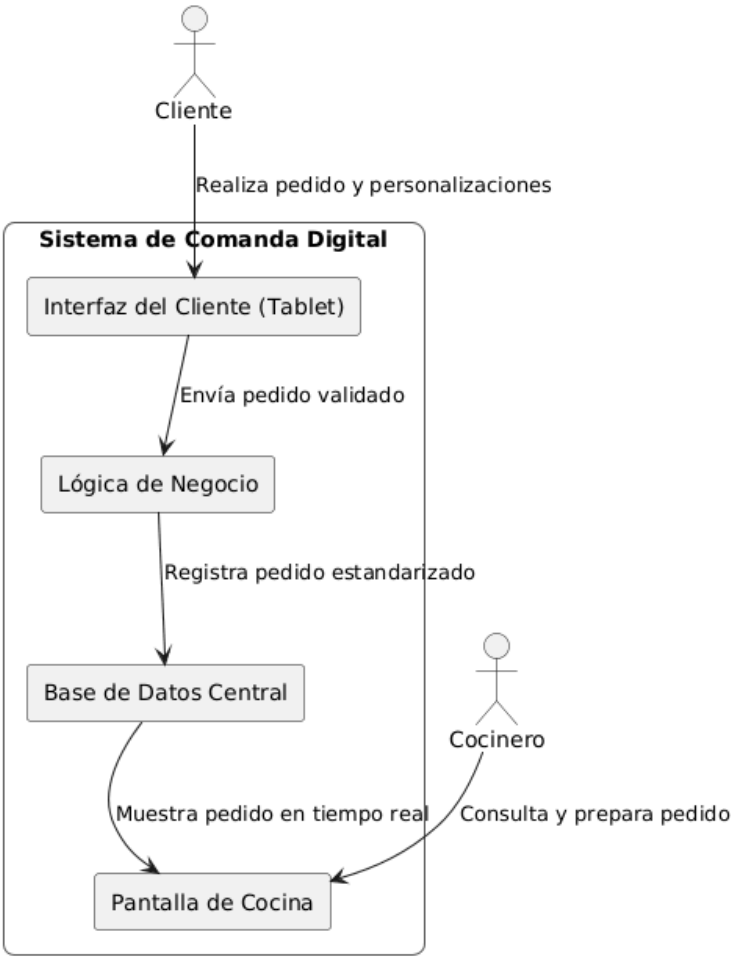
Se escogió la digitalización y estandarización porque reduce la posibilidad de errores humanos de transcripción y de confusiones por escritura ilegible, asegurando que la cocina reciba únicamente pedidos viables y claros. La estandarización permite que la cocina y el personal de servicio trabajen con la misma información organizada.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Base de Datos Centralizada: El sistema almacena la información de los pedidos en una base de datos relacional central, que funge como la única fuente de verdad. Esto garantiza que el Cocinero, el Mesero y el Administrador accedan a los mismos datos coherentes.

2. Pantalla de Cocina: Se integra una pantalla principal situada en un lugar visible en la cocina para la visualización en tiempo real de las órdenes recibidas, mejorando la organización.

3. Lógica de Negocio: La Capa de Lógica de Negocio valida datos y procesa las órdenes, asegurando que solo lleguen a la cocina las órdenes coherentes.



2.2.3.1 Rapidez en la Recepción de la Orden.

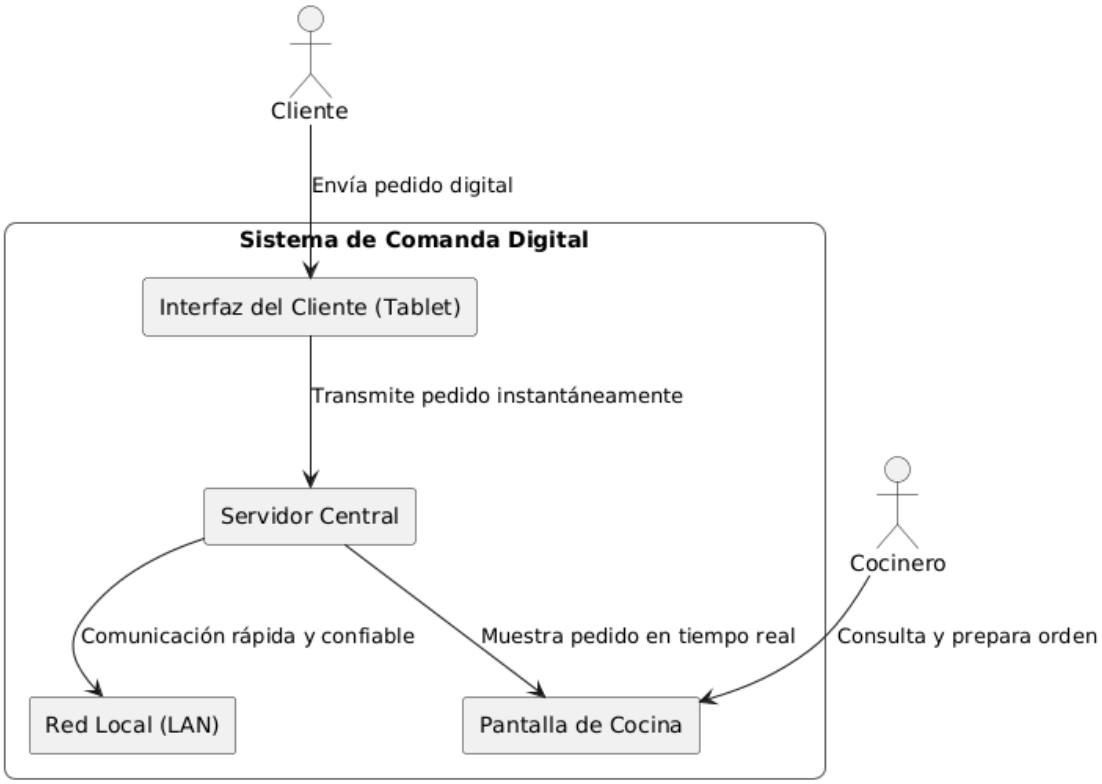
La rapidez en la recepción de la comanda es fundamental para reducir los tiempos de espera y agilizar el servicio. El pedido se transmite de forma instantánea al servidor central y se refleja inmediatamente en la pantalla de la cocina. Esto elimina el tiempo asociado al traslado manual de la orden por parte del mesero.

Se escogió la transmisión inmediata de pedidos porque permite que el personal de cocina comience la preparación del platillo sin demoras, optimizando la rotación de mesas y mejorando la eficiencia operativa.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Comunicación en Tiempo Real: La arquitectura fue diseñada para soportar la transmisión y visualización de datos en tiempo real. El servidor central procesa la información y la presenta en la pantalla de cocina inmediatamente.

2. Infraestructura de Red Local (LAN): La comunicación continua y sin interrupciones se garantiza mediante una red local (LAN) diseñada para proporcionar baja latencia y alta confiabilidad. Esta red interconecta las *tablets*, el *servidor* y la *pantalla de cocina*.



2.2.3.2 Gestión y Trazabilidad del Flujo de Trabajo.

El Cocinero requiere un mecanismo para organizar su carga de trabajo y notificar cuándo un platillo ha sido terminado. El sistema permite al Cocinero actualizar el estado del pedido (ej. "En preparación", "Listo para entregar") desde su interfaz.

Se escogió la actualización de estados digitalmente porque mantiene sincronizada la información con el Mesero y la Administración, permitiendo al Mesero coordinar la

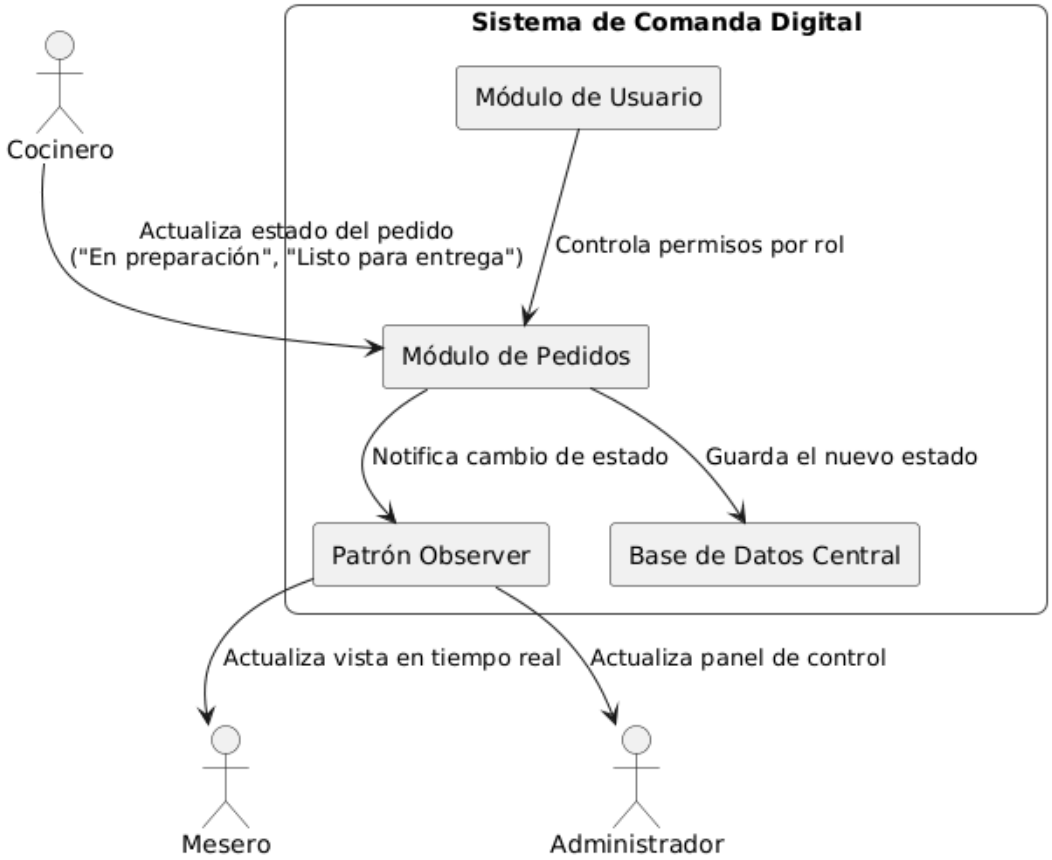
entrega en el momento justo (M.2). Esta función es clave para la trazabilidad y el control del ciclo de vida del pedido.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Módulo de Pedidos con Estados: El Módulo de Pedidos gestiona los pedidos, facilitando la creación de pedidos que avanzan por distintos estados (e.g., *En preparación*, *Listo para entrega*).

2. Patrón Observer: Se emplea el patrón Observer para notificar automáticamente al Cocinero y al Administrador sobre cambios de estado en el pedido, manteniendo el flujo de trabajo ágil.

3. Módulo de Usuario: El rol de Cocinero se limita a las funciones de visualizar pedidos pendientes y actualizar su estado, sin permisos para modificarlos o acceder a información administrativa, gracias a la Gestión de Roles



2.2.3.3 Disponibilidad de Ingredientes.

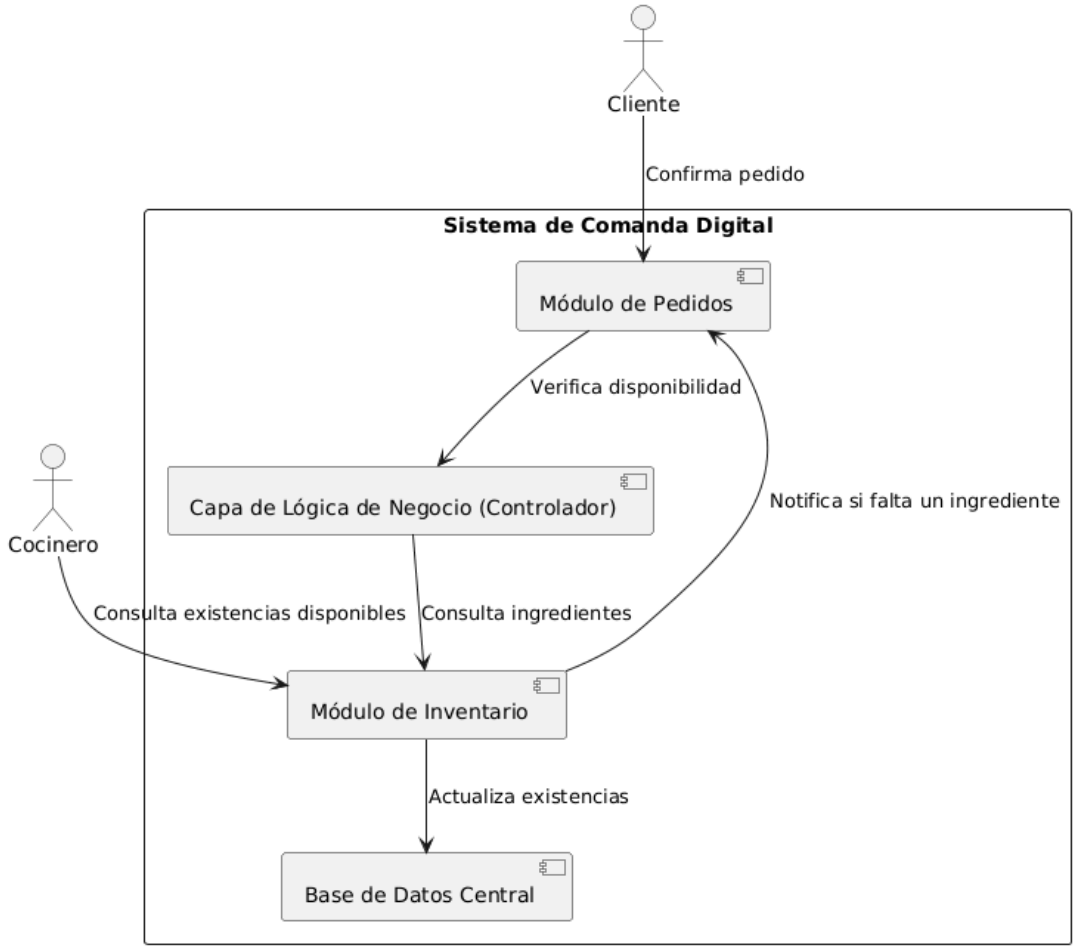
El Cocinero debe asegurarse de que los ingredientes necesarios para preparar el platillo estén disponibles. El sistema resuelve esto mediante la Actualización del Inventario en Tiempo Real. Si un ingrediente se agota, el sistema puede bloquear automáticamente los pedidos que lo requieran y notificar al usuario sobre la falta de disponibilidad, lo que previene que la cocina comience un platillo inviable.

Se escogió la actualización automática del inventario porque evita situaciones incómodas y la pérdida de tiempo al intentar preparar un platillo que ya no se puede completar. La información de disponibilidad se mantiene actualizada tanto para el personal como para los clientes.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Módulo de Inventario Integrado: El Módulo de Inventario es un componente central que controla los ingredientes. El módulo de pedidos interactúa con el inventario y lo actualiza automáticamente al confirmarse un pedido. El Cocinero también tiene acceso a este módulo para verificar existencias.

2. Validación de Disponibilidad: La Capa de Lógica de Negocio (Controlador) es responsable de verificar la disponibilidad de platillos en el inventario antes de confirmar la orden.



2.2.4 Administrador.

2.2.4.1Gestión centralizada de inventarios y control operativo.

El Administrador es el responsable de la gestión operativa del sistema y requiere un control centralizado. Sus tareas incluyen el control de inventario, la supervisión de operaciones y la gestión de roles de usuarios. Necesita la capacidad de modificar el contenido del inventario y tener acceso completo para gestionar pedidos, inventario e informes.

Se escogió el control operativo centralizado porque el Administrador es el responsable de la gestión operativa del sistema y requiere un control centralizado. Sus tareas incluyen el control de inventario, la supervisión de operaciones y la gestión de roles de usuarios. Necesita la capacidad de modificar el contenido del inventario y tener acceso completo para gestionar pedidos, inventario e informes.

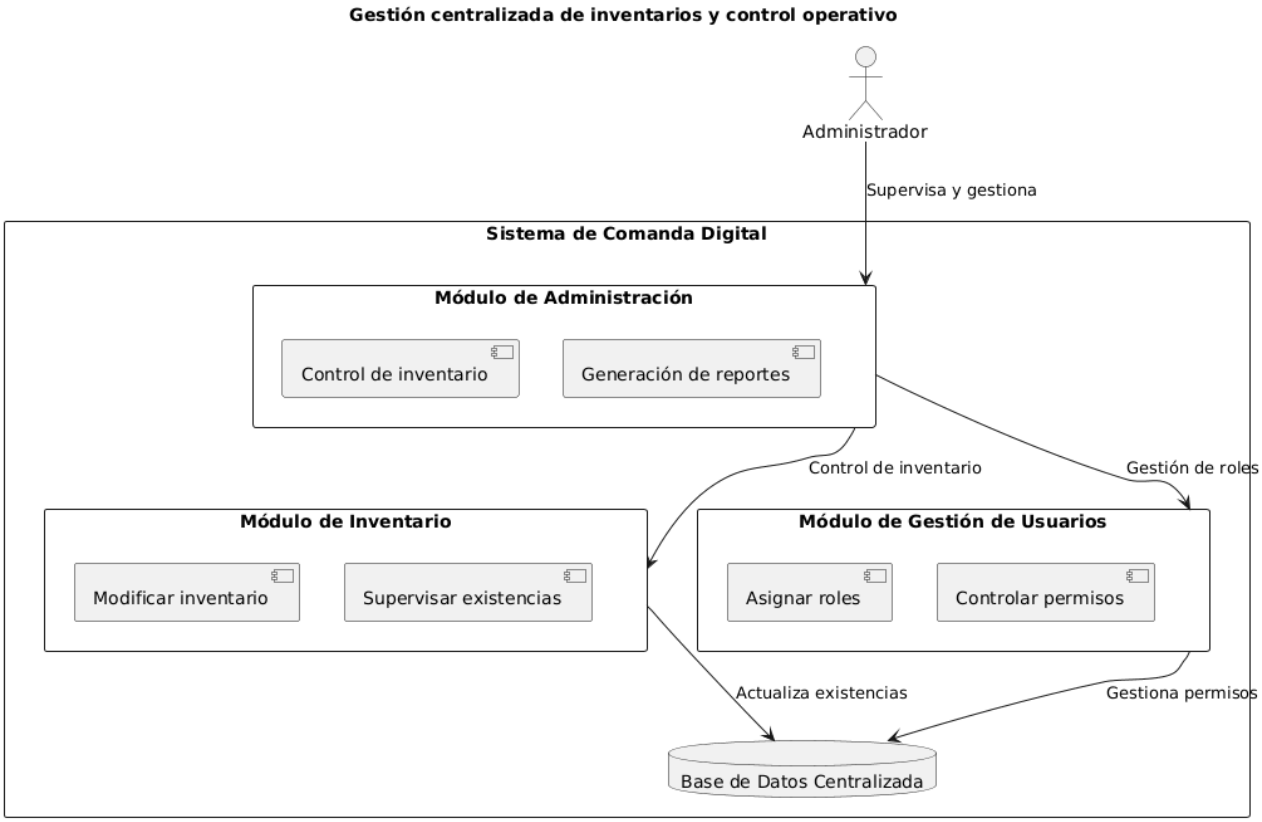
Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Módulo de Administración: Se diseñó un módulo de administración con mayores privilegios destinado exclusivamente al control de inventarios y generación de reportes.

2. Módulo de Inventario: Solo los administradores pueden modificar el contenido del inventario, lo que asegura un control centralizado.

3. Módulo de Gestión de Usuarios: Se contempla un Módulo de Usuario que gestiona los roles y permisos, asegurando que el Administrador tenga acceso completo a la gestión.

4. Base de Datos Centralizada: Se almacena toda la información en una base de datos central de tipo relacional que actúa como la única fuente de verdad.



2.2.4.2 Necesidad de Reportes Estratégicos y Métricas de Negocio.

El Administrador necesita generar reportes e informes para evaluar el desempeño del restaurante. El propietario del restaurante, cuyo rol es más estratégico, utiliza esta información generada para la toma de decisiones estratégicas y de negocio. Estos reportes incluyen ventas, tendencias de consumo, platillos más solicitados y rendimiento general.

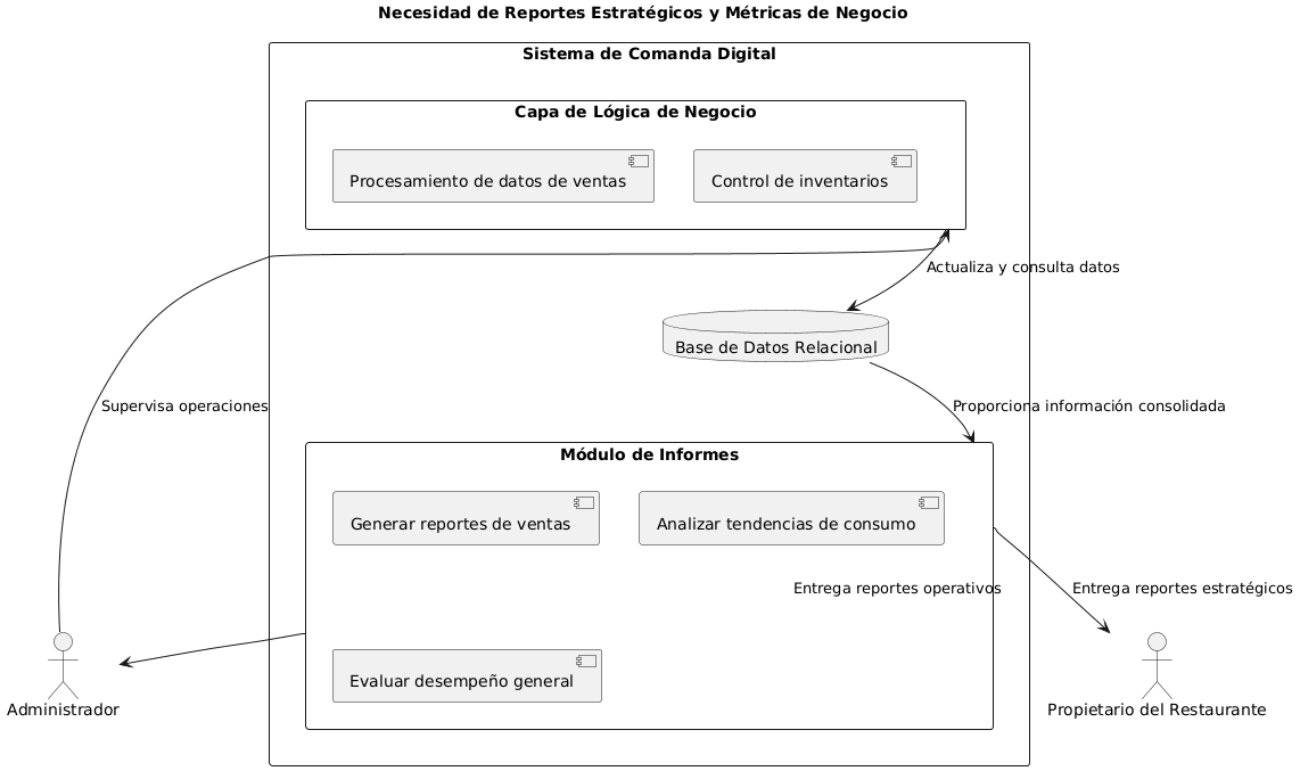
Se escogió la generación estructurada de informes porque la disponibilidad de información estructurada y consistente facilita la elaboración de reportes estratégicos que apoyan la toma de decisiones en áreas como abastecimiento, gestión de personal y diseño de promociones. Permite obtener métricas confiables sobre ventas, consumo de ingredientes y desempeño general.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Módulo de Informes: Existe un Módulo de Informes que reúne datos relacionados con pedidos, uso del inventario y ventas. Los datos generados se integran de forma consistente en el módulo de informes.

2. Consistencia de la Información: El sistema utiliza una base de datos relacional que almacena pedidos, usuarios, mesas, inventarios y reportes. El diseño arquitectónico asegura la consistencia entre vistas, garantizando que los datos sean confiables y actualizados.

3. Capa de Lógica de Negocio: La Capa de Lógica de Negocio es la encargada de controlar inventarios y procesar la información necesaria para la generación de reportes.



2.2.4.3 Seguridad del sistema y restricción de acceso.

La seguridad es fundamental para manejar datos sensibles. El Administrador, al tener permisos avanzados, debe garantizar que otros roles (como clientes o cocineros) se limiten a sus funciones específicas, evitando usos indebidos, la alteración de pedidos o el acceso no autorizado a información financiera o sensible.

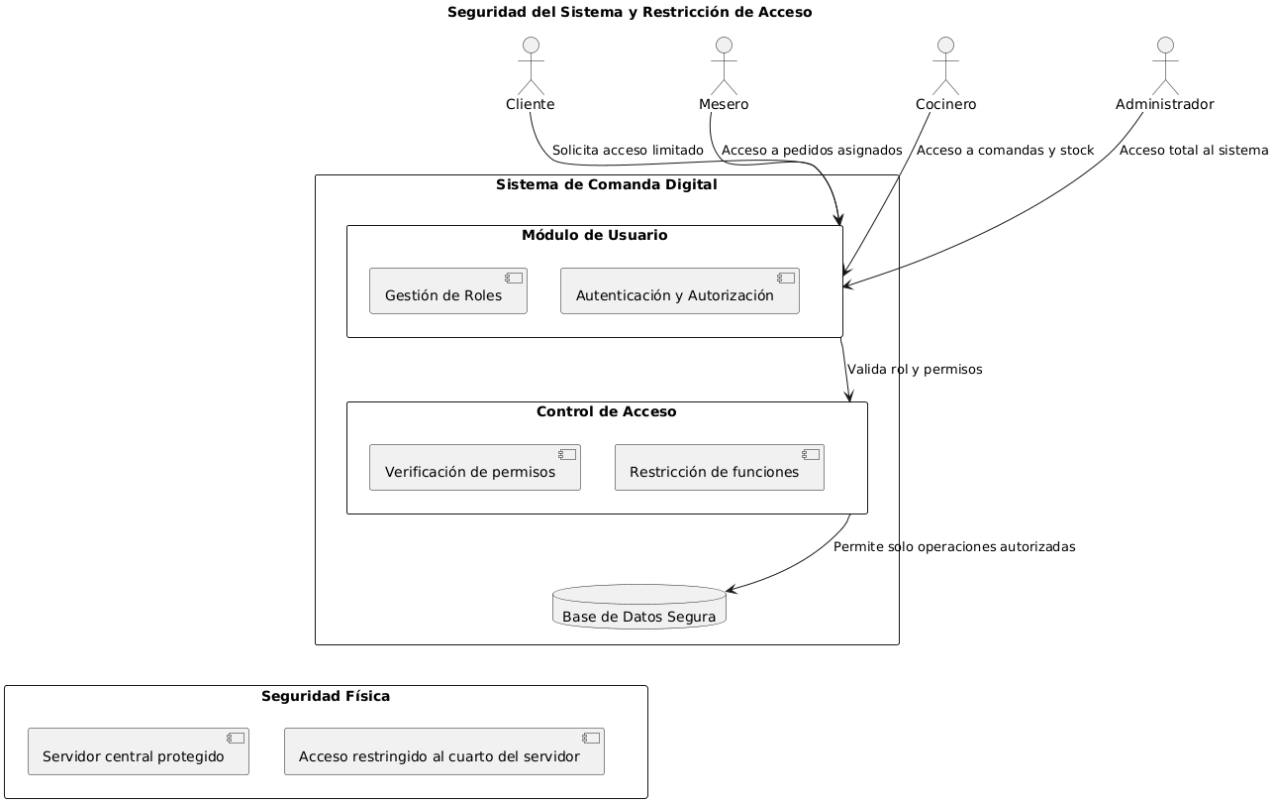
Se escogió la gestión jerárquica de roles porque permite delimitar claramente las funciones y permisos de cada usuario según su rol. Esto evita riesgos como la manipulación indebida de datos o la alteración de pedidos.

Esta decisión tuvo un impacto directo y fundamental en el diseño de la arquitectura de las siguientes maneras:

1. Gestión de Roles: El sistema contempla una estructura jerárquica de roles (cliente, mesero, cocinero y administrador). El Administrador es el único que dispone de permisos avanzados para la gestión integral del negocio.

2. Control de Acceso: Se implementan controles de autenticación y autorización. El Módulo de Usuario se encarga de gestionar los roles y permisos, asegurando que cada perfil solo acceda a las funciones que le corresponden.

3. Seguridad Física: El servidor central y la base de datos se encuentran aislados en un cuarto seguro con acceso restringido para proteger la integridad del sistema.



3. Puntos de Vista de la Arquitectura.

La arquitectura del Sistema de Comanda Digital (SCD) se describe desde diferentes puntos de vista, con el propósito de ofrecer una comprensión integral del sistema desde múltiples perspectivas. De acuerdo con el estándar ISO/IEC/IEEE 42010:2011, un punto de vista define un conjunto de convenciones, modelos y criterios que guían la forma en que se representa y analiza la arquitectura, en función de las preocupaciones específicas de uno o varios interesados.

En otras palabras, cada punto de vista constituye una “lente” que permite observar la arquitectura del sistema bajo un ángulo particular, destacando los elementos, relaciones y propiedades relevantes según el tipo de usuario o necesidad que se esté considerando. Por ejemplo, mientras el punto de vista de contexto muestra la relación del sistema con su entorno y actores externos, el punto de vista lógico se enfoca en su estructura interna, clases y módulos.

Para el SCD, se seleccionaron los puntos de vista que mejor reflejan las necesidades identificadas en las secciones anteriores, así como la naturaleza modular y distribuida del sistema. Estos puntos de vista se derivan de los lineamientos del estándar IEEE 1016 y se adaptan al marco de referencia de IEEE 42010, garantizando una documentación coherente, trazable y alineada con las buenas prácticas de ingeniería de software.

Los puntos de vista definidos en este documento permiten abordar las principales preocupaciones de los stakeholders, tales como la usabilidad, mantenibilidad, rendimiento, comunicación entre componentes, integridad de datos y escalabilidad. En conjunto, proporcionan una base sólida para la toma de decisiones de diseño y sirven como referencia para futuras fases de desarrollo, mantenimiento o ampliación del sistema.

A continuación, se describen los puntos de vista seleccionados para el Sistema de Comanda Digital (SCD), cada uno acompañado de su justificación, objetivos y los modelos arquitectónicos que lo representan.

3.1 Punto de Vista de Contexto.

El Punto de Vista de Contexto describe la relación del Sistema de Comanda Digital (SCD) con su entorno operacional y con los actores y sistemas externos que interactúan directamente con él. Su objetivo es representar los límites del sistema, los servicios principales que ofrece desde la perspectiva externa y las interfaces de integración con elementos fuera del SCD. Este punto de vista atiende preocupaciones de stakeholders orientadas a la comprensión del alcance del sistema, la identificación de actores y la seguridad de las interconexiones.

Justificación: dado que el SCD se integra físicamente en un restaurante (tablets en mesas, pantalla de cocina, impresora y servidor), el Punto de Vista de Contexto es esencial para asegurar que los interesados comprendan claramente qué está dentro del sistema (componentes gestionados por el SCD) y qué elementos permanecen fuera o se consideran sistemas externos (por ejemplo, el sistema de cobro físico o proveedores externos de pago en futuras versiones). Además, facilita la identificación temprana de dependencias tecnológicas que afectan la disponibilidad y el rendimiento.

Objetivos del punto de vista:

1. Precisar los límites del SCD en el entorno del restaurante.
2. Identificar actores humanos y sistemas externos relevantes (clientes, meseros, cocineros, administrador, propietario, impresora de tickets, red local y servidor).
3. Documentar las interacciones y flujos de información principales que son críticos para la operación en tiempo real.
4. Servir como base para requerimientos no funcionales relacionados con seguridad, disponibilidad y latencia.

Stakeholders a los que sirve este punto de vista.

Clientes, Meseros, Cocineros, Administrador operativo, Propietario del restaurante, Personal de caja, Equipo de desarrollo y encargado de infraestructura.

Criterios de consistencia.

El diagrama de contexto deberá concordar con la sección 1 (Identificación y Visión General) y con las vistas detalladas que vendrán a continuación (vista lógica y de despliegue). Cualquier actor o sistema mostrado en el diagrama de contexto debe existir también en las listas de stakeholders y en las trazas de requisitos; si se descubre una nueva dependencia, se actualizará la sección de alcance y las reglas de correspondencia.

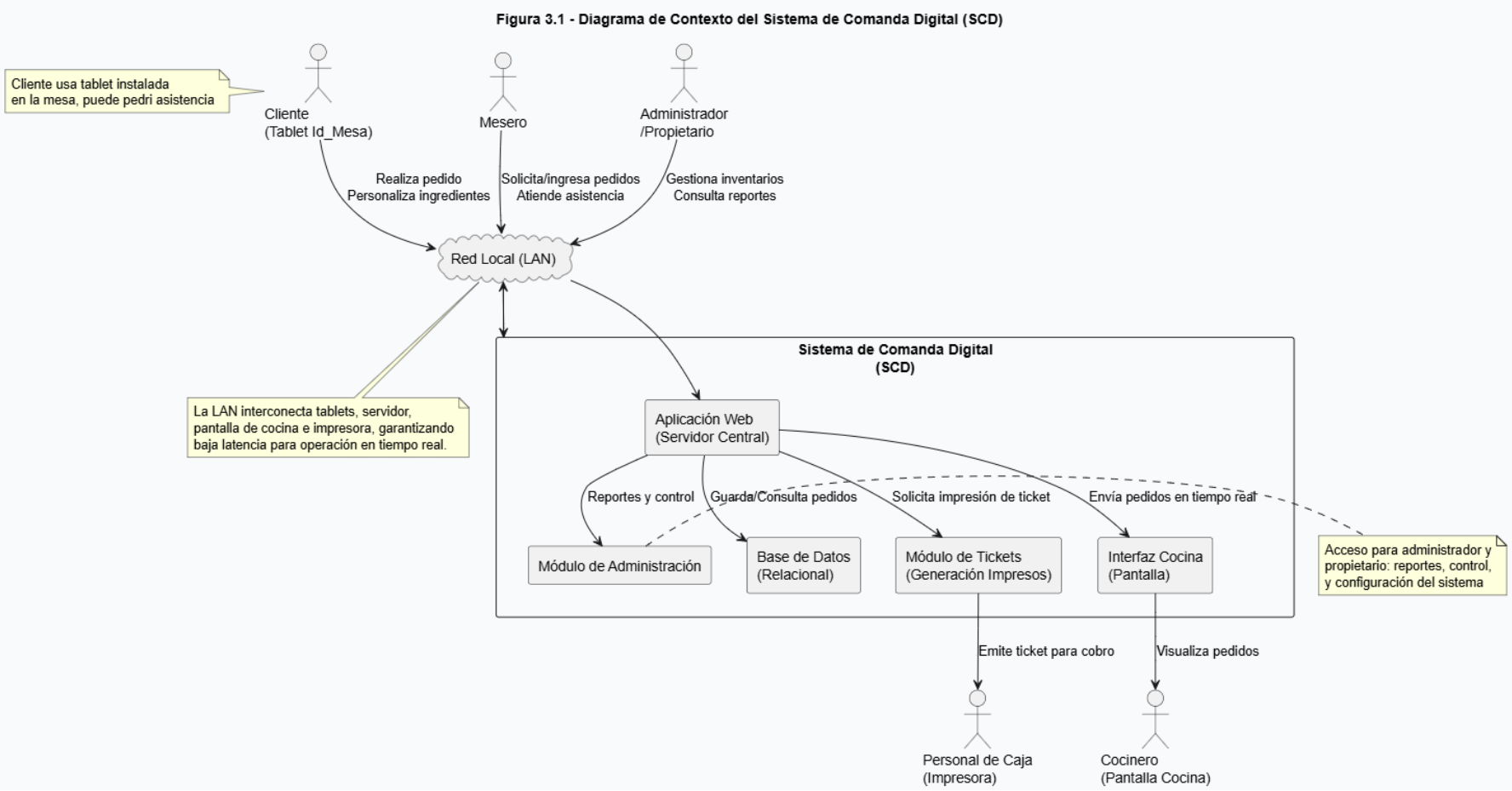


Figura 3.1: Diagrama de contexto del Sistema de Comanda Digital (SCD).

Descripción breve del contenido del diagrama.

El diagrama de la Figura 3.1 muestra los actores y sistemas externos que interactúan directamente con el SCD, así como las principales relaciones de comunicación entre ellos. Este modelo representa la vista lógica del entorno inmediato del sistema, previo a su distribución física en el restaurante. El SCD se representa como un único sistema central que expone interfaces hacia las tablets (clientes), el módulo de administración (administrador/propietario), la pantalla de cocina (cocinero), la impresora de tickets (caja) y el cuarto del servidor (infraestructura). La red local (LAN) se muestra como el medio de transporte que permite la comunicación en tiempo real. Los flujos principales de información son: creación y envío de pedidos desde tablet → servidor, visualización en cocina ← servidor, generación de ticket ← servidor → impresora, y acceso administrativo a reportes ←→ módulo de administración.

Mapa de distribución Arquitectónica.

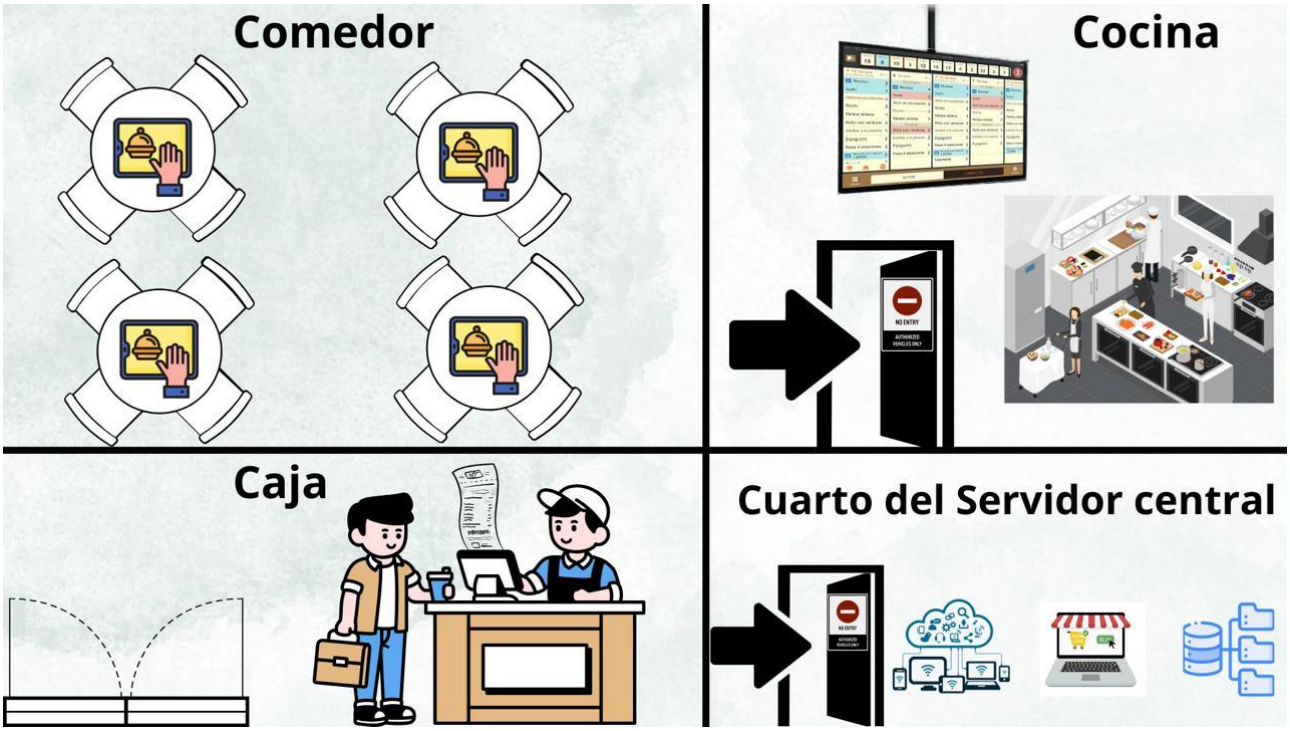


Figura 3.1.1: Mapa de distribución Arquitectónica.

El mapa mostrado en la Figura 3.1.1 representa la disposición física de los elementos del SCD en un restaurante de tamaño mediano. Se observan las tablets distribuidas en cada mesa, la impresora de tickets junto a la caja, la pantalla de cocina en el área de preparación y el cuarto del servidor central con el equipo de red, aplicación web y base de datos. Esta configuración busca asegurar conectividad estable y eficiencia operativa dentro del entorno real.

A continuación, se presenta la justificación de los principales elementos que conforman este entorno físico y tecnológico.

Justificación de los elementos de interés.

Cada componente representado en el punto de vista de contexto cumple una función específica dentro de la operación del Sistema de Comanda Digital (SCD) y ha sido ubicado en el entorno del restaurante con base en criterios de eficiencia, seguridad y continuidad del servicio. A continuación, se describen las principales decisiones que sustentan esta configuración:

Tablets instaladas en las mesas:

Se asigna una tablet a cada mesa del restaurante con un identificador único (Id_Mesa), lo que permite asociar los pedidos a una ubicación física concreta. Esta decisión busca evitar errores en la entrega de platillos y mejorar la trazabilidad de las órdenes. Además, al permitir que el cliente gestione su pedido directamente, se reduce la carga operativa del mesero y se optimiza el tiempo de atención.

Servidor local (cuarto técnico):

El servidor se ubica en un cuarto técnico o área segura del establecimiento para garantizar el control físico del hardware y la protección de la información almacenada. La elección de un servidor local, en lugar de un servicio en la nube, se justifica por la necesidad de asegurar disponibilidad continua incluso ante fallos en la conexión a internet, así como por la simplicidad de mantenimiento en entornos pequeños o medianos.

Red local (LAN):

El uso de una red local conecta de forma directa las tablets, el servidor, la impresora y la pantalla de cocina. Esta decisión responde a la necesidad de baja latencia y alta confiabilidad en la transmisión de datos, condiciones esenciales para el procesamiento en tiempo real de pedidos. Además, permite mantener la independencia operativa del sistema sin depender de servicios externos.

Pantalla de cocina:

La pantalla ubicada en el área de cocina permite a los cocineros visualizar en tiempo real los pedidos recibidos, las modificaciones solicitadas por los clientes y los tiempos de preparación. Su conexión directa con el servidor elimina la necesidad de comunicación verbal o escrita, reduciendo errores y mejorando la eficiencia del proceso.

Impresora de tickets (caja):

Se integra una impresora de tickets conectada al sistema para generar comprobantes físicos que validan los pedidos y facilitan el proceso de cobro. Esto asegura que cada pedido tenga respaldo tangible antes de iniciar su preparación, y además habilita el registro de ventas sin depender exclusivamente de medios digitales.

Módulo de administración:

El administrador o propietario accede al sistema a través de un módulo dedicado, desde el cual puede gestionar inventarios, consultar reportes de ventas y supervisar el flujo de pedidos. Esta decisión responde a la preocupación de los interesados por contar con herramientas de control y monitoreo que apoyen la toma de decisiones operativas y estratégicas.

Meseros y personal de apoyo:

Aunque el cliente puede realizar pedidos de manera autónoma, se mantiene la posibilidad de asistencia por parte del mesero mediante un botón de ayuda integrado en la interfaz. Esto permite conservar la atención personalizada y resolver incidencias de manera inmediata, equilibrando la automatización con la interacción humana.

Por lo tanto, estas decisiones reflejan un equilibrio entre automatización, control y confiabilidad, garantizando que el Sistema de Comanda Digital pueda adaptarse a distintos tipos de establecimientos sin perder continuidad operativa. La justificación detallada de cada decisión arquitectónica se desarrollará en la sección 6 del presente documento, dedicada a las *Decisiones Arquitectónicas.*

En conjunto, este punto de vista establece los límites, interacciones y contexto operativo del SCD, sirviendo como base para los siguientes puntos de vista, en los que se abordará su estructura lógica interna y la organización de sus componentes.

3.2 Punto de Vista Lógico.

El Punto de Vista Lógico describe la estructura interna del Sistema de Comanda Digital (SCD) desde la perspectiva del diseño de software, mostrando los componentes principales, sus relaciones y las reglas de interacción que soportan las funcionalidades del sistema. Este punto de vista representa cómo se organiza la información y el comportamiento dentro del sistema, de manera independiente a la infraestructura física.

El SCD se modela siguiendo un enfoque modular basado en el patrón de arquitectura Modelo–Vista–Controlador (MVC), complementado con el uso del patrón Observer para el manejo de notificaciones internas. Esta combinación permite una separación clara entre la lógica de negocio, la presentación de la información y la gestión de los eventos, asegurando la integridad de los datos y la sincronización en tiempo real entre los distintos módulos (clientes, cocina, caja y administración).

Justificación: el uso del patrón MVC se seleccionó porque el sistema requiere mantener sincronización constante entre múltiples vistas (clientes, cocineros, meseros y administradores), sin comprometer la integridad de los datos ni la experiencia de usuario. Al separar la lógica de presentación de la lógica de negocio, se facilita el mantenimiento del código, la incorporación de nuevas funcionalidades y la migración futura hacia entornos más escalables.

Además, la integración del patrón Observer (implementado mediante las clases NotificationManager y StockObserver) permite gestionar de forma eficiente las alertas de inventario y los eventos del sistema, mejorando la respuesta ante cambios en tiempo real.

Este enfoque está alineado con las buenas prácticas de diseño recomendadas por el estándar IEEE 1016, garantizando trazabilidad y coherencia entre las vistas lógicas y las operaciones del sistema.

Objetivos del punto de vista

1. Representar las principales entidades lógicas del sistema y sus relaciones internas.
2. Identificar los módulos y sus responsabilidades dentro del patrón MVC y el subsistema de notificaciones.
3. Asegurar dependencias claras y mínimas entre componentes para facilitar la mantenibilidad.
4. Servir como base para el diseño de la base de datos y para la implementación de los módulos funcionales.

Stakeholders a los que sirve este punto de vista.

Equipo de desarrollo, administrador del sistema, encargado de infraestructura y propietario del restaurante. Este punto de vista permite entender cómo se estructura internamente el sistema, facilitando su mantenimiento, validación de integridad de datos y planificación de futuras ampliaciones.

Criterios de consistencia.

El diagrama lógico debe ser coherente con los casos de uso identificados y con la vista de contexto (3.1), reflejando cómo las funcionalidades externas se sustentan en estructuras internas. Cada clase o componente descrito debe mantener trazabilidad hacia uno o más requerimientos funcionales.

Diagrama lógico del SCD

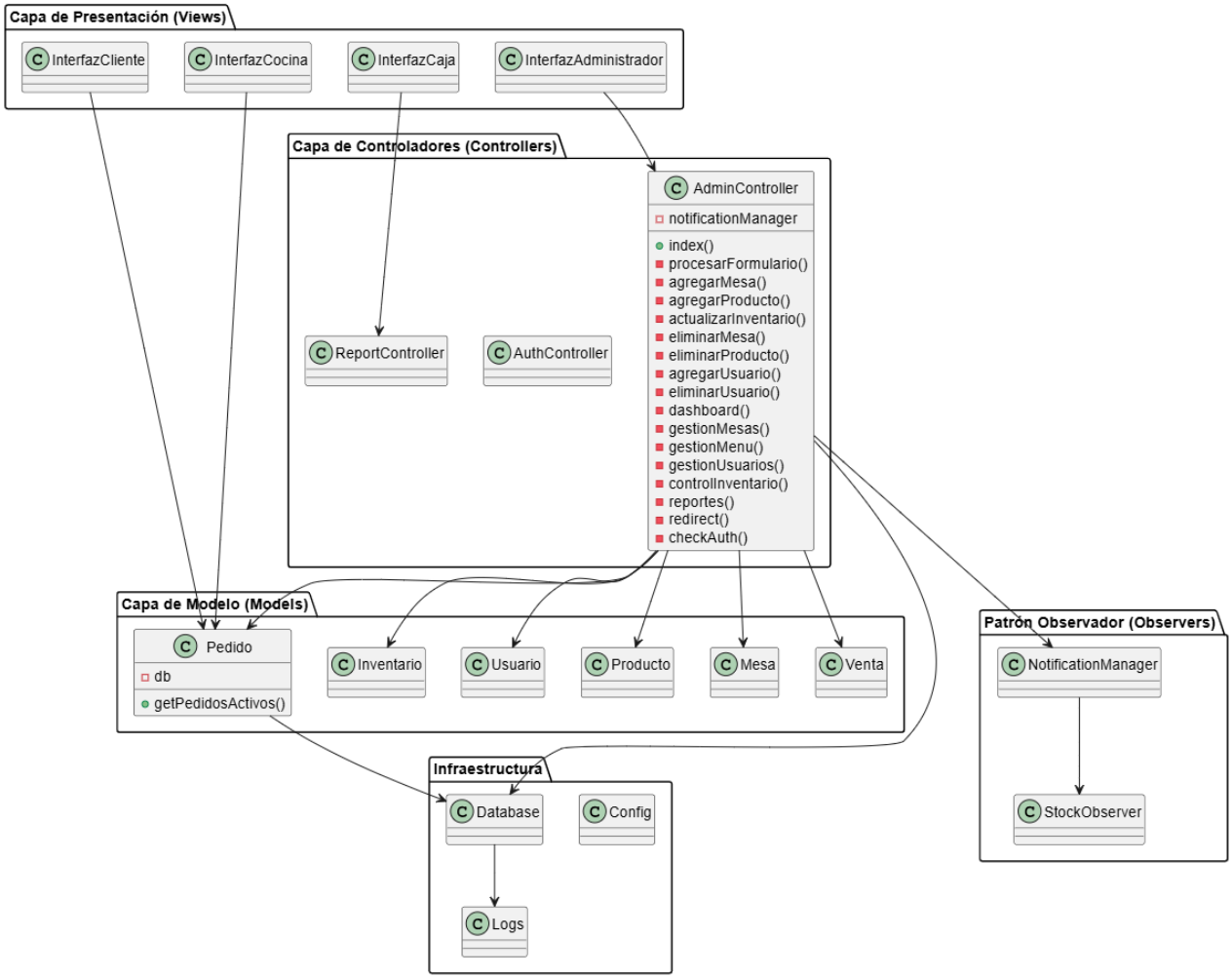


Figura 3.2: Diagrama lógico del SCD.

Descripción breve del contenido del diagrama.

El modelo lógico del Sistema de Comanda Digital (SCD) se organiza en tres capas principales que reflejan la separación de responsabilidades del patrón MVC, junto con un subsistema de observadores que complementa la arquitectura con notificaciones automáticas.

La Capa de Presentación (Views) contiene las interfaces con las que interactúan los usuarios, como *InterfazCliente*, *InterfazCocina*, *InterfazCaja* e *InterfazAdministrador*.

Estas vistas presentan la información proveniente de los controladores y permiten ejecutar las acciones principales del sistema.

La Capa de Controladores (Controllers) incluye los módulos encargados de coordinar la comunicación entre las vistas y el modelo. Los controladores principales (*AdminController*, *AuthController* y *ReportController*) gestionan las operaciones críticas del sistema, validan datos, aplican reglas de negocio y coordinan la persistencia de la información.

La Capa de Modelo (Models) representa las entidades centrales del dominio, tales como *Pedido*, *Mesa*, *Producto*, *Usuario*, *Inventario* y *Venta*. Estas clases encapsulan la lógica de negocio y las operaciones sobre la base de datos.

El Patrón Observer, implementado mediante las clases *NotificationManager* y *StockObserver*, permite notificar automáticamente a las vistas o controladores cuando ocurren eventos relevantes, como un bajo inventario o la actualización de un pedido.

Finalmente, la Infraestructura incluye los elementos técnicos que soportan la operación del sistema, como *Config*, *Database* y *Logs*, los cuales se encargan de la configuración general, la conexión con la base de datos y el registro de eventos.

Justificación de los elementos y decisiones lógicas.

Cada componente del modelo lógico cumple una función específica y responde a decisiones arquitectónicas orientadas a modularidad, la reutilización del código y la claridad de responsabilidades. La separación entre las clases de dominio (por ejemplo, *Pedido*, *Mesa*, *Producto*) y las clases de presentación garantiza un bajo acoplamiento, lo que facilita la incorporación de mejoras sin alterar la estructura interna del sistema.

El controlador *AdminController* centraliza la gestión administrativa del SCD, permitiendo administrar mesas, productos, usuarios, inventario y reportes desde un único punto de control; esta centralización mejora la coherencia operativa y simplifica las rutas de auditoría. La clase *Usuario* actúa como clase base para los distintos tipos de usuarios

(cliente, mesero, administrador), lo que permite un manejo uniforme de credenciales, roles y permisos.

El subsistema de notificaciones, implementado mediante *NotificationManager* y *StockObserver*, aplica el patrón Observer para fortalecer la comunicación interna del sistema: notifica automáticamente eventos relevantes —por ejemplo, actualizaciones de inventario o alertas críticas en cocina— sin acoplar la lógica de negocio al mecanismo de notificación. Esta decisión mejora la mantenibilidad y facilita la extensión de reglas de alerta sin tocar la lógica central.

En conjunto, este punto de vista ofrece una visión clara y estructurada de la arquitectura lógica del SCD, mostrando cómo las capas y componentes colaboran para sostener operaciones en tiempo real y garantizar la trazabilidad de las órdenes. La justificación detallada de estas decisiones se desarrollará en la Sección 6: Decisiones Arquitectónicas, donde se documentarán las alternativas consideradas, los trade-offs y los criterios (funcionales y no funcionales) que motivaron cada elección.

3.3 Punto de Vista de Procesos.

(Flujo del pedido desde su creación hasta la entrega)

El Punto de Vista de Procesos describe el comportamiento dinámico del Sistema de Comanda Digital (SCD), es decir, cómo los distintos componentes arquitectónicos colaboran para cumplir con una función esencial del sistema: la gestión completa de un pedido. Este punto de vista representa el flujo de información, las interacciones entre actores y módulos, y la secuencia de actividades necesarias para transformar una acción del cliente en un resultado tangible dentro del restaurante.

Justificación: El proceso de creación y seguimiento de un pedido es el núcleo funcional del SCD. Modelar este flujo permite visualizar cómo el sistema responde a eventos en tiempo real (como la creación, modificación y confirmación de pedidos), y cómo las distintas partes —desde la interfaz de cliente hasta la cocina— trabajan coordinadamente.

Además, esta vista ayuda a los stakeholders a comprender la dinámica del sistema,

facilitando la validación de los requerimientos de rendimiento, sincronización y disponibilidad. Su representación apoya la toma de decisiones en torno a optimización de tiempos, balance de carga y reducción de errores operativos.

Objetivos del punto de vista.

1. Representar la secuencia de interacciones que componen el ciclo de vida de un pedido.
2. Identificar los componentes y actores que intervienen en cada etapa del proceso.
3. Mostrar cómo el sistema garantiza trazabilidad, consistencia de datos y respuesta en tiempo real.
4. Proveer una base para evaluar requerimientos no funcionales como latencia, disponibilidad y confiabilidad.

Stakeholders a los que sirve este punto de vista.

Clientes (comensales), Meseros, Cocineros, Administrador, Propietario, Equipo de desarrollo e Infraestructura. Este punto de vista les permite comprender cómo el sistema coordina sus módulos internos para garantizar una operación fluida, rápida y confiable.

Criterios de consistencia.

El flujo de procesos debe mantener coherencia con el diagrama lógico (3.2) y el contexto operativo (3.1.). Cada componente que participe en el proceso debe existir en el modelo lógico y en la arquitectura física definida. Cualquier nuevo evento identificado deberá registrarse como posible caso de uso complementario.

Descripción breve del proceso principal: "Gestión de Pedido".

1. Inicio del pedido:

El cliente interactúa con la InterfazCliente desde la tablet de su mesa, seleccionando los platillos del menú disponibles.

2. Validación y confirmación:

Al enviar el pedido, la solicitud se transmite al AdminController, que valida la disponibilidad de productos en el Inventario y genera un nuevo registro en la base de datos.

3. Notificación a cocina:

Una vez confirmado el pedido, el sistema notifica automáticamente al módulo de InterfazCocina mediante el patrón observer implementado por NotificationManager, indicando los platillos a preparar y sus especificaciones.

4. Preparación y actualización:

El cocinero actualiza el estado del pedido conforme avanza la preparación ("En preparación", "Listo para entregar"). Estas actualizaciones se reflejan en tiempo real tanto en la tablet del cliente como en la InterfazAdministrador.

5. Entrega y registro:

Una vez completado el pedido, el sistema envía una notificación al módulo de caja (InterfazCaja), donde se genera el ticket de pago y se almacena el registro final en Venta.

6. Cierre y estadísticas:

El AdminController actualiza las métricas de venta y desempeño, que pueden ser consultadas por el administrador o propietario desde el panel correspondiente.

Justificación de decisiones.

El diseño de este flujo se basa en una arquitectura reactiva y modular, donde los componentes se comunican mediante eventos gestionados por observadores. Esto permite reducir la latencia, evitar bloqueos entre módulos y asegurar una actualización constante de la información visible para todos los actores. Además, la comunicación basada en red local (LAN) garantiza independencia de conexión a internet, manteniendo la operatividad del restaurante en todo momento. Esta organización facilita la trazabilidad completa del pedido desde su origen hasta su cierre, asegurando integridad de datos y control administrativo.

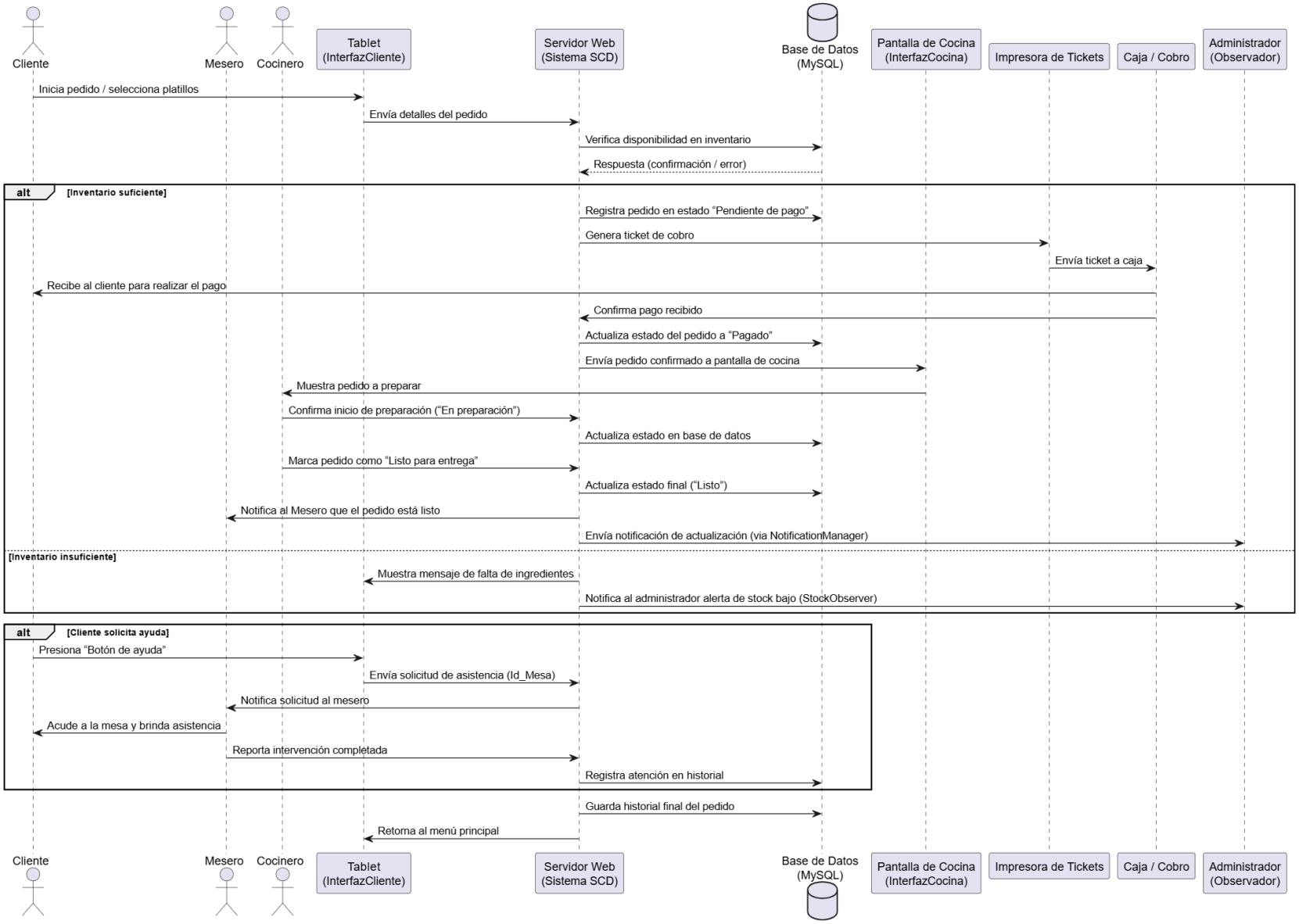

Cliente
Mesero
Cocinero
Tablet (InterfazCliente)
Servidor Web (Sistema SCD)
Base de Datos (MySQL)
Pantalla de Cocina (InterfazCocina)
Impresora de Tickets
Caja / Cobro
Administrador (Observador)
Inicia pedido / selecciona platillos
Envía detalles del pedido
Verifica disponibilidad en inventario
Respuesta (confirmación / error)
alt
[Inventario suficiente]
Registra pedido en estado "Pendiente de pago"
Genera ticket de cobro
Envía ticket a caja
Recibe al cliente para realizar el pago
Confirma pago recibido
Actualiza estado del pedido a "Pagado"
Envía pedido confirmado a pantalla de cocina
Muestra pedido a preparar
Confirma inicio de preparación ("En preparación")
Actualiza estado en base de datos
Marca pedido como "Listo para entrega"
Actualiza estado final ("Listo")
Notifica al Mesero que el pedido está listo
Envía notificación de actualización (via NotificationManager)
[Inventario insuficiente]
Muestra mensaje de falta de ingredientes
Notifica al administrador alerta de stock bajo (StockObserver)
alt
[Cliente solicita ayuda]
Presiona "Botón de ayuda"
Envía solicitud de asistencia (Id_Mesa)
Notifica solicitud al mesero
Acude a la mesa y brinda asistencia
Reporta intervención completada
Registra atención en historial
Guarda historial final del pedido
Retorna al menú principal

Figura 3.3: Diagrama de secuencias del SCD.

Descripción del contenido del diagrama.

El diagrama de secuencia presentado en la figura 3.3 modela de manera detallada el ciclo completo de un pedido dentro del Sistema de Comanda Digital (SCD), mostrando las interacciones entre los actores principales —cliente, mesero, cocinero y administrador— y los módulos internos del sistema.

El flujo comienza cuando el cliente, desde la Tablet (InterfazCliente), selecciona los platillos deseados y envía la orden al Servidor Web (SCD). Este componente valida la disponibilidad de los ingredientes consultando la Base de Datos (MySQL). Si el inventario es suficiente, el sistema registra el pedido en estado *"Pendiente de pago"*, genera un ticket a través de la Impresora de Tickets y envía la información al módulo de Caja/Cobro, donde el cliente realiza el pago.

Una vez confirmado el pago, el pedido pasa al estado *"Pagado"* y es transmitido automáticamente a la Pantalla de Cocina (InterfazCocina), donde el Cocinero visualiza la orden, inicia su preparación y actualiza los estados sucesivos (*"En preparación"*, *"Listo para entrega"*).

Cuando el pedido está finalizado, el sistema notifica al Mesero, quien procede con la entrega al cliente, y simultáneamente envía una notificación al Administrador mediante el NotificationManager, manteniendo un registro de todas las actualizaciones.

En caso de inventario insuficiente, el sistema alerta inmediatamente al cliente y notifica al Administrador mediante el StockObserver, asegurando la trazabilidad de la incidencia y la gestión preventiva de abastecimiento.

El diagrama también incluye un flujo alternativo donde el cliente puede solicitar asistencia a través del botón de ayuda, lo que activa un evento hacia el Mesero, quien atiende la mesa y registra su intervención en el sistema.

Finalmente, el servidor guarda el historial completo del pedido y devuelve a la interfaz principal, cerrando así el ciclo operativo.

Justificación de las decisiones de modelado.

El diagrama se elaboró bajo un enfoque de arquitectura dirigida por eventos (EDA), complementado con el patrón Observer, lo que permite manejar de manera eficiente las notificaciones automáticas entre los componentes del sistema.
Este enfoque asegura que los módulos funcionen de manera asincrónica, reduciendo la latencia y evitando dependencias directas entre el flujo de la cocina, la caja y el cliente. Asimismo, la validación del inventario previo al registro del pedido y la confirmación del pago antes de enviarlo a cocina garantizan la consistencia de datos y la trazabilidad completa de cada transacción.

El flujo alternativo de asistencia se incluye como parte del compromiso de mantener una atención personalizada, sin comprometer la automatización general del sistema.
Esta representación cumple con los principios del estándar ISO/IEC/IEEE 42010, al mostrar de forma clara las interacciones dinámicas entre los elementos arquitectónicos y los actores del entorno, fortaleciendo la comprensión del comportamiento operacional del SCD.

3.4 Punto de Vista de Desarrollo.

El Punto de Vista de Desarrollo describe la organización del Sistema de Comanda Digital (SCD) desde la perspectiva del equipo de desarrollo, mostrando cómo se agrupan los componentes lógicos en módulos, directorios y subsistemas dentro del entorno de implementación. Su propósito es facilitar la comprensión de la estructura del código, la asignación de responsabilidades y la gestión del mantenimiento evolutivo del sistema.

Justificación: El SCD se implementa bajo una arquitectura Modelo–Vista–Controlador (MVC) en un entorno PHP con MySQL, adoptando una estructura modular y jerárquica. Esta organización promueve la separación de responsabilidades, la reutilización de componentes, y la facilidad de depuración.

Asimismo, permite una integración coherente con los procesos de desarrollo y despliegue, asegurando que los cambios en una capa no afecten negativamente a las demás.

La adopción de una estructura en capas y módulos responde a los lineamientos del estándar IEEE 42010, que enfatiza la necesidad de mantener consistencia, trazabilidad y claridad entre los artefactos arquitectónicos y su implementación real.

Objetivos del punto de vista.

1. Documentar la estructura de directorios y módulos del sistema.
2. Mostrar las dependencias entre componentes y cómo se comunican entre sí.
3. Garantizar que la organización del código respalde las decisiones arquitectónicas descritas en las vistas previas.
4. Facilitar la gestión de mantenimiento y evolución del sistema.

Stakeholders a los que sirve este punto de vista.

Equipo de desarrollo, encargados de mantenimiento, administrador del sistema y personal de infraestructura.

Este punto de vista les permite identificar las responsabilidades de cada módulo y comprender cómo se implementan las funciones clave del sistema.

Criterios de consistencia.

Cada módulo documentado debe corresponder con una entidad lógica descrita en la vista 3.2, asegurando así coherencia entre el diseño lógico y la implementación física. Las dependencias reflejadas deben mantenerse alineadas con el flujo de procesos descrito en la vista 3.3, garantizando que las interacciones dinámicas estén sustentadas por una estructura de código consistente.

Además, la organización de carpetas y clases dentro del sistema debe permitir trazabilidad hacia los requerimientos funcionales, de modo que cada componente tenga un propósito verificable dentro de la arquitectura general del SCD.

Descripción general del contenido.

El Sistema de Comanda Digital (SCD) está estructurado de manera modular y jerárquica, siguiendo la organización propuesta por el patrón Modelo–Vista–Controlador (MVC). En la raíz del proyecto se encuentra el directorio /config, que contiene los archivos de configuración general (config.php y database.php), donde se definen los parámetros globales, la conexión a la base de datos y las rutas base del sistema.

El directorio /controllers agrupa los controladores principales, entre ellos AdminController.php, AuthController.php y ReportController.php, responsables de coordinar la lógica del sistema y gestionar la comunicación entre las vistas y los modelos.

En la carpeta /models se encuentran las clases que representan las entidades del dominio, como Pedido.php, Mesa.php, Producto.php, Usuario.php, Inventario.php y Venta.php. Dentro de este mismo nivel existe una subcarpeta denominada /models/observers, que alberga las clases NotificationManager.php y StockObserver.php. Estas implementan el patrón Observer, encargado de manejar las notificaciones automáticas y los eventos internos del sistema.

El directorio /views contiene las interfaces gráficas destinadas a los distintos actores del sistema —cliente, cocina, caja y administrador—, organizadas en subcarpetas que corresponden a cada módulo funcional.

El directorio /public almacena los elementos accesibles al cliente final, como hojas de estilo, archivos JavaScript, imágenes y el archivo principal index.php, encargado de inicializar el sistema. Finalmente, la carpeta /logs registra los eventos, alertas y errores generados durante la ejecución, permitiendo un control y seguimiento eficiente del funcionamiento del sistema.

Esta disposición modular refuerza la independencia entre capas y promueve la mantenibilidad del sistema, permitiendo actualizar o sustituir un módulo sin afectar el funcionamiento de los demás.

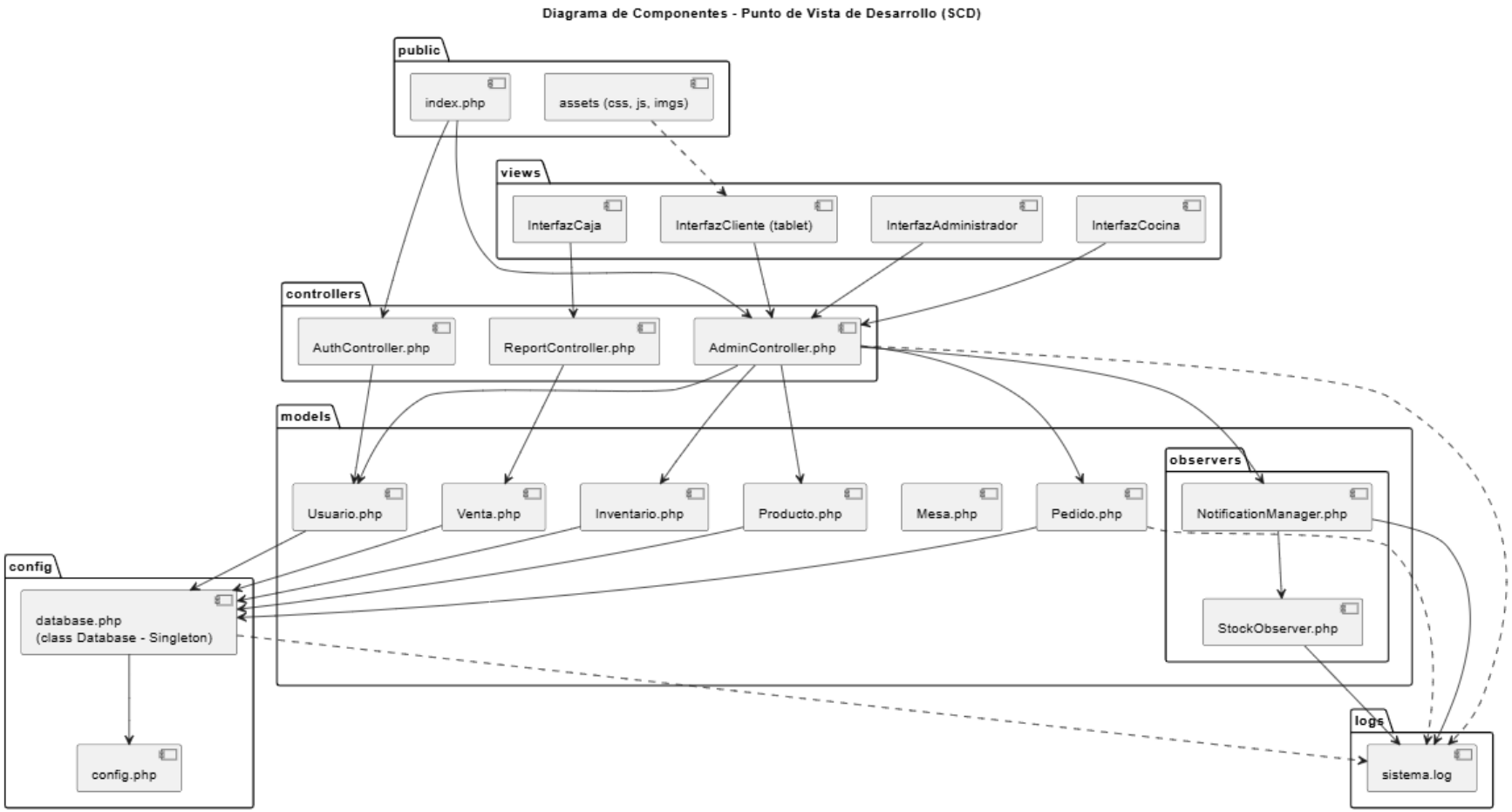


Figura 3.4: Diagrama de componentes del SCD.

Descripción del contenido del diagrama.

El diagrama de componentes anterior representa la organización física y lógica del código fuente del Sistema de Comanda Digital (SCD), ilustrando las dependencias de implementación entre sus módulos y las rutas principales del proyecto. Cada paquete corresponde directamente con una carpeta del repositorio, y cada componente dentro de esos paquetes representa un archivo o una responsabilidad técnica concreta dentro del sistema.

El paquete config agrupa los archivos config.php y database.php, que constituyen el núcleo de configuración del sistema.

config.php define las constantes globales utilizadas a lo largo del proyecto —como BASE_URL, credenciales de conexión y parámetros del entorno— y proporciona funciones auxiliares para la generación de rutas dinámicas.

Por su parte, database.php implementa la clase Database bajo el patrón Singleton, lo que garantiza una única conexión activa a MySQL en todo el sistema. Esto evita redundancias, reduce el riesgo de errores de concurrencia y asegura uniformidad en la codificación de caracteres (utf8mb4), manteniendo compatibilidad internacional con nombres de platillos y usuarios.

El paquete controllers contiene los controladores que orquestan la lógica de aplicación: AdminController, AuthController y ReportController gestionan la comunicación entre las vistas y los modelos, coordinando la creación, modificación y consulta de información. Estos controladores son responsables de aplicar las reglas de negocio, procesar formularios, generar respuestas y enviar datos a las vistas. Además, se integran con el subsistema de observadores (NotificationManager y StockObserver) para generar alertas automáticas —por ejemplo, avisos de inventario bajo o actualización de pedidos— sin requerir intervención manual.

El paquete models agrupa las entidades del dominio: Pedido, Mesa, Producto, Usuario, Inventario y Venta.

Cada modelo representa una tabla o concepto funcional del sistema y encapsula la lógica asociada a la gestión de datos, utilizando la clase Database para sus operaciones CRUD. Dentro de este mismo paquete, la subcarpeta models/observers implementa el patrón Observer, donde NotificationManager administra la lista de observadores y StockObserver responde ante eventos relevantes, como modificaciones en el inventario. Esta separación mantiene el bajo acoplamiento entre módulos y favorece la extensibilidad del sistema.

El paquete views contiene las distintas interfaces gráficas de usuario: InterfazCliente (para las tablets en las mesas), InterfazCocina, InterfazCaja y InterfazAdministrador. Cada una presenta información adaptada al rol del usuario y se comunica con su respectivo controlador.

Estas vistas son servidas desde el paquete public, el cual también alberga el archivo principal index.php y el directorio assets/, que contiene los recursos estáticos (CSS, JavaScript e imágenes).

Aunque en el diagrama se muestra su relación directa con InterfazCliente para evitar saturación visual, los recursos de assets son compartidos por todas las interfaces del sistema.

El paquete logs cumple una función transversal al registrar eventos del sistema en el archivo sistema.log. Tanto controladores como modelos pueden escribir en este registro, documentando errores, advertencias o alertas generadas por el subsistema Observer.

Esto permite al administrador o al equipo de desarrollo rastrear la ejecución, depurar incidentes y mantener trazabilidad de las operaciones críticas.

Justificación de las decisiones de desarrollo.

La estructura descrita responde a decisiones arquitectónicas que priorizan la claridad, mantenibilidad y modularidad del código. La adopción del patrón Modelo–Vista–Controlador (MVC) asegura la separación de responsabilidades, facilitando la localización de errores y la evolución del sistema sin afectar otras capas.

Centralizar la configuración y la conexión a la base de datos en config/ evita duplicación de lógica y favorece la portabilidad: un cambio en las credenciales o rutas afecta a todo el sistema desde un solo punto.

La inclusión del subsistema de observadores fortalece la capacidad de reacción del SCD ante eventos internos, manteniendo la coherencia entre el inventario, la cocina y la administración.
Al registrar los eventos relevantes en logs/sistema.log, se obtiene una bitácora de auditoría que facilita tanto la depuración como el análisis posterior de comportamiento, alineándose con las buenas prácticas de trazabilidad establecidas por el estándar ISO/IEC/IEEE 42010:2011.

Finalmente, la modularidad de los componentes y la jerarquía de carpetas promueven la trazabilidad entre requisitos, diseño y código, asegurando que cada funcionalidad implementada pueda vincularse con una decisión arquitectónica documentada. Esto convierte al SCD en una arquitectura coherente, extensible y fácilmente mantenible, preparada para evolucionar hacia futuras versiones o integraciones.

3.5 Punto de Vista de Despliegue.

El Punto de Vista de Despliegue describe la configuración física y tecnológica que soporta la operación del Sistema de Comanda Digital (SCD) en el entorno real del restaurante. Su propósito es mostrar cómo los componentes de software previamente definidos (en las vistas lógica y de desarrollo) se distribuyen sobre los distintos nodos de hardware y redes de comunicación, asegurando rendimiento, disponibilidad y confiabilidad durante la operación diaria.

Justificación: El despliegue del SCD requiere garantizar comunicación en tiempo real entre los dispositivos ubicados en el restaurante —tablets de los clientes, pantalla de cocina, caja y servidor local—. Por esta razón, se selecciona una arquitectura cliente-servidor en red LAN local, donde todos los dispositivos se conectan a un servidor central que aloja la aplicación web, la base de datos y los servicios de notificación. Esta configuración evita la dependencia de internet, asegurando continuidad operativa incluso ante fallos externos.

La impresora térmica y la pantalla de cocina se integran directamente al servidor a través de la red local, permitiendo sincronización inmediata entre los pedidos, su preparación y el registro de ventas.

Objetivos del punto de vista.

1. Representar la disposición física de los componentes de software sobre la infraestructura tecnológica.
2. Identificar los nodos de hardware, las conexiones de red y las interacciones principales.
3. Asegurar que el diseño de despliegue garantice disponibilidad, baja latencia y trazabilidad completa.
4. Proveer una base para el plan de instalación y mantenimiento del sistema.

Stakeholders.
Administrador del sistema, equipo de infraestructura, propietario del restaurante, personal técnico de soporte y equipo de desarrollo.

Criterios de consistencia.

El diagrama de despliegue debe ser coherente con las vistas de contexto (3.1) y de desarrollo (3.4). Cada componente de software mostrado debe corresponder a uno existente en el repositorio. Las relaciones de red deben mantener trazabilidad con las interacciones descritas en el Punto de Vista de Procesos (3.3).

Descripción del entorno físico

El Sistema de Comanda Digital (SCD) se despliega en un entorno local cuidadosamente diseñado para garantizar estabilidad, bajo tiempo de respuesta y operación continua incluso ante fallos de conexión externa. La infraestructura está conformada por diversos nodos interconectados que colaboran mediante una red local (LAN), permitiendo la comunicación en tiempo real entre las distintas áreas del restaurante.

El servidor local se encuentra alojado en un cuarto técnico y ejecuta los servicios principales del sistema: Apache, PHP y MySQL. En él residen tanto la aplicación web como la base de datos *sistema_comanda_digital_v1*, junto con los archivos de configuración (*config.php*, *database.php*) y los controladores centrales. Además, el servidor gestiona la cola de notificaciones a través de *NotificationManager* y mantiene un registro de eventos mediante *sistema.log*, lo que permite la trazabilidad y el monitoreo del comportamiento del sistema.

Las tablets de clientes son dispositivos Android o iPad con navegador web, configuradas para acceder directamente al sistema mediante la dirección IP del servidor local (por ejemplo, *http://192.168.0.10/comanda1*). Desde estas interfaces, los clientes pueden visualizar el menú, realizar pedidos y solicitar asistencia sin necesidad de intervención del mesero.

La pantalla de cocina es un dispositivo conectado por LAN o Wi-Fi que muestra en tiempo real los pedidos activos y sus estados. Mediante la interfaz de cocina, los cocineros pueden actualizar el progreso de cada orden, cambiando su estado a *"En preparación"* o *"Listo para entrega"*, con lo cual se mantiene sincronizada la información en todo el sistema.

La caja y la impresora de tickets conforman el módulo de cobro. La caja utiliza una computadora que ejecuta la interfaz de venta y genera los comprobantes físicos mediante una impresora térmica conectada directamente al servidor. Este flujo asegura que cada pedido validado y pagado quede registrado tanto en la base de datos como en el sistema de ventas.

El módulo administrativo se accede desde el equipo del administrador o propietario. A través de este módulo se consultan reportes de ventas, métricas de rendimiento, alertas de inventario y estadísticas de operación general, lo que facilita la supervisión integral del restaurante.

Finalmente, la red local (LAN) actúa como medio de comunicación entre todos los dispositivos. Esta red, con direccionamiento privado y configurada mediante un router o

switch central, garantiza una conexión estable, de baja latencia y controlada, asegurando la continuidad de la operación incluso ante la ausencia de conexión a internet.

En conjunto, esta disposición física asegura que el flujo de información y las operaciones entre los diferentes módulos del SCD se ejecuten de forma continua, confiable y sincronizada.

Justificación de decisiones de despliegue

El uso de un servidor local centralizado se justifica por su capacidad para evitar la dependencia de internet y mantener un tiempo de respuesta óptimo. Al concentrar la aplicación, la base de datos y los servicios de notificación en un mismo punto, se garantiza coherencia en el manejo de la información y facilidad de mantenimiento.

La adopción de clientes ligeros, como las tablets, reduce costos operativos y simplifica las actualizaciones, ya que el acceso se realiza únicamente a través del navegador. La conexión directa con la impresora térmica y la pantalla de cocina elimina pasos intermedios y mejora la eficiencia de los procesos internos.

Por su parte, el registro en *logs* y la separación de responsabilidades entre controladores, modelos y vistas permiten aislar fallas específicas sin comprometer el resto del sistema. Esta estructura modular favorece la recuperación ante errores, la trazabilidad de eventos y el mantenimiento técnico del sistema.

El Diagrama de Despliegue Técnico (Figura X) complementa el Mapa de Distribución Física (Figura Y) presentado en el Punto 3.1, mostrando ahora la disposición lógica y tecnológica del Sistema de Comanda Digital (SCD) sobre los distintos nodos de hardware.
Mientras el mapa anterior representaba la ubicación espacial de los dispositivos en el restaurante, este modelo UML enfatiza las conexiones de red, la asignación de componentes de software y las relaciones de comunicación entre ellos.

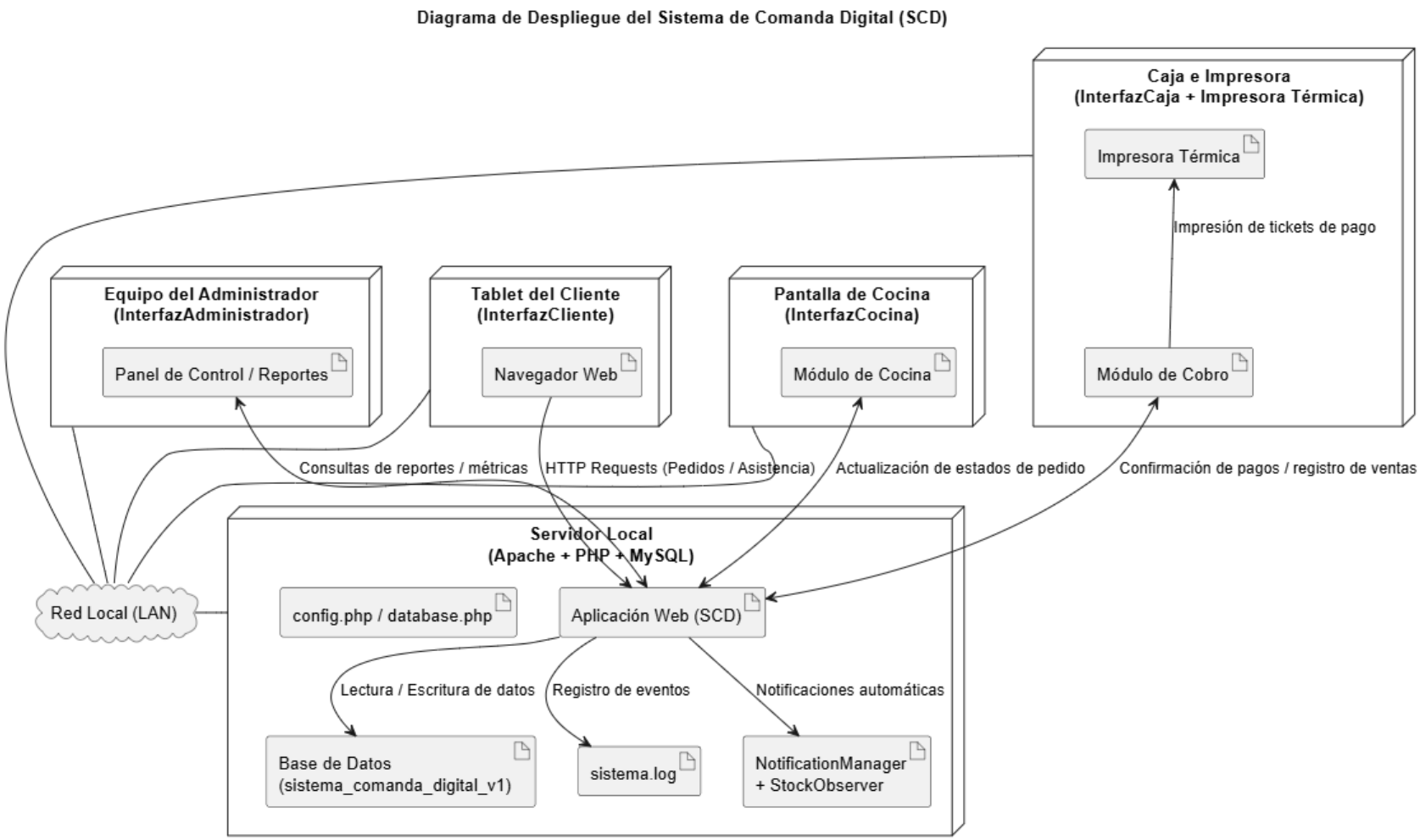


Figura 3.4: Diagrama de despliegue del SCD.

Descripción del contenido del diagrama.

El diagrama anterior muestra la distribución técnica del Sistema de Comanda Digital (SCD) sobre la infraestructura física del restaurante. Mientras el mapa del Punto 3.1 representa la ubicación espacial de los dispositivos (mesas, cocina, caja y cuarto técnico), este modelo UML se centra en la disposición lógica de los componentes de software dentro de los distintos nodos de hardware y las relaciones de comunicación entre ellos.

El sistema se implementa bajo una arquitectura cliente-servidor en red local (LAN), donde el servidor central funciona como núcleo del procesamiento, almacenamiento y gestión de datos.

Sobre él se ejecutan los servicios Apache, PHP y MySQL, junto con la aplicación web del SCD, el archivo de configuración (config.php), el módulo de conexión (database.php), los observadores (NotificationManager, StockObserver) y el registro de eventos (sistema.log). Este nodo es el punto de coordinación entre todos los módulos funcionales del sistema.

Los clientes ligeros, como las tablets del comedor, se conectan al servidor a través del navegador web, enviando solicitudes HTTP (puerto 80) para crear o consultar pedidos. La pantalla de cocina, también conectada por LAN o Wi-Fi, recibe notificaciones automáticas de nuevos pedidos y actualiza su estado conforme avanza la preparación. El módulo de caja e impresora térmica está sincronizado con el servidor para registrar pagos y generar comprobantes físicos, mientras que el equipo del administrador accede al panel de control para revisar métricas, inventarios y reportes.

La red local (LAN) actúa como medio común de comunicación, interconectando todos los dispositivos mediante un direccionamiento privado (por ejemplo, 192.168.0.x). Gracias a esta red, el flujo de información ocurre con baja latencia, permitiendo actualizaciones en tiempo real sin depender de internet.

Justificación técnica del despliegue.

El diseño del despliegue responde a criterios de eficiencia, confiabilidad y mantenibilidad, alineados con las buenas prácticas del estándar *ISO/IEC/IEEE 42010*. Centralizar la aplicación y la base de datos en un único servidor local permite un control total sobre los datos y evita la dependencia de servicios externos, asegurando que el restaurante pueda seguir operando incluso sin conexión a internet. Además, facilita el respaldo y la restauración del sistema ante fallas o actualizaciones.

El uso de clientes ligeros reduce el mantenimiento, ya que las tablets y pantallas solo requieren un navegador para acceder al sistema.

La comunicación directa con la impresora térmica y el registro de eventos en sistema.log garantiza trazabilidad y evidencia de cada operación realizada.

Asimismo, la presencia del módulo de observer (NotificationManager y StockObserver) dentro del servidor permite la emisión automática de alertas de inventario o notificaciones administrativas, sin sobrecargar el resto del sistema.

En conjunto, este despliegue ofrece un equilibrio entre simplicidad, control y escalabilidad. Cada nodo cumple una función bien delimitada dentro del ecosistema del SCD, contribuyendo a la operación continua del restaurante y asegurando la coherencia entre los procesos físicos y los componentes de software.

4. Vistas de Arquitectura.

La presente sección reúne las vistas arquitectónicas que representan, desde diferentes perspectivas, el diseño del Sistema de Comanda Digital (SCD). Cada vista complementa a los puntos de vista establecidos en la Sección 3, proporcionando ahora modelos concretos, diagramas y descripciones formales de cómo se estructura, opera y despliega el sistema.

Mientras la sección anterior definió los objetivos, justificaciones y criterios de cada punto de vista, en esta sección se materializan dichas decisiones mediante representaciones visuales y explicaciones detalladas. El propósito es mostrar la arquitectura del SCD de manera completa, coherente y trazable, permitiendo comprender la solución tanto a nivel conceptual como técnico.

Cada vista se construyó siguiendo las convenciones del estándar IEEE 42010, manteniendo consistencia con los requerimientos funcionales, las preocupaciones de los interesados y la organización interna del sistema. Asimismo, se garantiza que los modelos que aquí se presentan correspondan de manera directa a los componentes descritos en el diseño lógico, desarrollo y despliegue.

A continuación, se presentan las vistas que conforman la arquitectura del Sistema de Comanda Digital (SCD).

4.1 Vista de Contexto.

La Vista de Contexto presenta al Sistema de Comanda Digital (SCD) como una unidad funcional completa, mostrando los límites del sistema, los actores externos que interactúan con él y el tipo de comunicación que se establece con cada uno. Su propósito es ofrecer una representación clara del entorno en el cual opera el SCD y de las dependencias externas que condicionan su funcionamiento.

Esta vista complementa el Punto de Vista de Contexto descrito en la Sección 3.1, proporcionando ahora una representación formal del sistema como “caja negra”. Aquí se muestran únicamente las interacciones esenciales, sin detallar la estructura interna del sistema, con el fin de resaltar los flujos principales de información y el alcance funcional del SCD.

El SCD funciona como un sistema web centralizado que administra pedidos, inventario, ventas y notificaciones dentro del restaurante. Los actores externos que interactúan con él forman parte directa del proceso operativo del negocio. Los clientes realizan pedidos desde las tablets instaladas en las mesas mediante la InterfazCliente; los meseros reciben notificaciones de asistencia o pedidos listos; los cocineros visualizan y actualizan el estado de preparación a través de la InterfazCocina; el personal de caja utiliza la InterfazCaja para registrar pagos y generar tickets; finalmente, el administrador accede al panel de control para gestionar inventarios, usuarios, reportes y métricas generales.

El sistema mantiene comunicación continua con la base de datos ubicada en el servidor local para consultar y actualizar información relativa a menú, inventario, pedidos y ventas.

A su vez, el módulo NotificationManager y sus observadores permiten generar alertas automáticas sobre situaciones relevantes, como inventario bajo o cambios en el estado de un pedido.

Esta vista asegura que se comprendan los límites del sistema antes de analizar sus componentes internos, facilitando la validación de responsabilidades y puntos de integración con el entorno del restaurante.

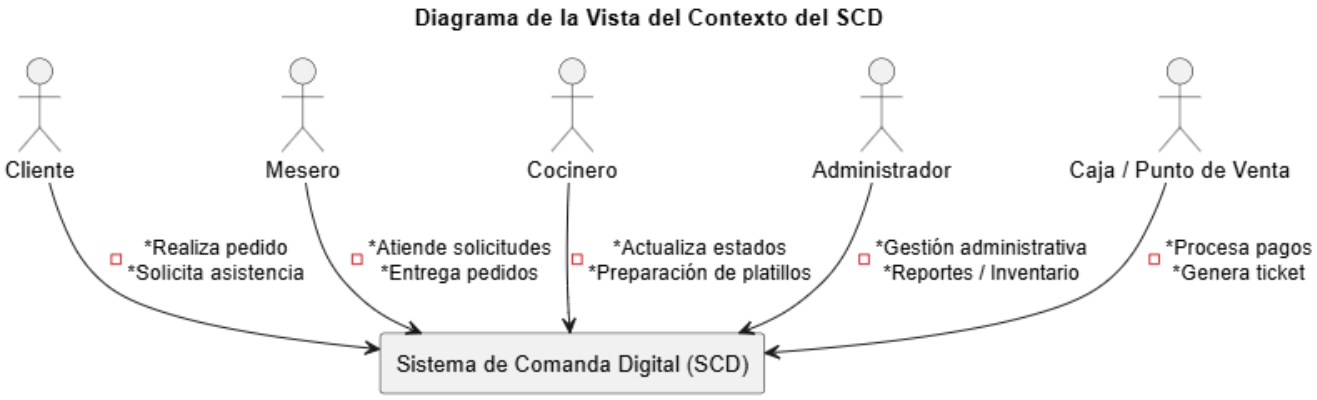


Figura 4.1: Diagrama de contexto de la vista de contexto del SCD.

Descripción del diagrama.

El diagrama muestra al Sistema de Comanda Digital como una entidad cerrada que centraliza todas las operaciones relacionadas con los pedidos. Los clientes envían solicitudes desde sus tablets para generar pedidos o pedir asistencia. Los meseros reciben notificaciones, apoyan al cliente y gestionan la entrega de los platillos. Los cocineros interactúan actualizando el estado de los pedidos conforme avanzan en su preparación. El administrador accede al sistema para revisar métricas, inventarios y reportes generales del restaurante. Finalmente, la caja procesa los pagos y genera tickets relacionados con cada pedido. Esta figura resume de manera visual cómo el sistema coordina la operación entre todas las áreas.

Justificación.

Esta vista se elabora siguiendo las recomendaciones del estándar IEEE 42010, el cual establece que una Vista del Contexto debe ser simple, comprensible y enfocada únicamente en los elementos externos que interactúan con el sistema. Utilizar un modelo limpio y sin detalles internos permite una comunicación clara con los diferentes interesados del proyecto, especialmente aquellos sin perfil técnico. De esta manera, se logra visualizar de forma directa qué actores dependen del sistema y qué tipo de interacción mantienen con él.

La simplicidad de esta vista evita redundancias con diagramas más detallados que aparecen en las vistas subsecuentes, especialmente en las vistas de procesos, lógica y despliegue, donde se desglosan los componentes internos del SCD.

Criterios de consistencia.

La Vista del Contexto mantiene alineación con los casos de uso del sistema, con los roles descritos en la Sección 3.1 y con los flujos operativos explicados en la Sección 3.3. Cada actor representado aquí tiene equivalencia directa con los elementos del modelo lógico presentado en la Sección 3.2 y con los componentes definidos en la Vista de Desarrollo. Además, las interacciones incluidas corresponden a los procesos reales del restaurante, manteniendo cohesión con el diseño general del sistema.

4.2 Vista lógica.

La Vista Lógica describe la estructura interna del Sistema de Comanda Digital (SCD) desde la perspectiva del diseño de software, mostrando los componentes principales, sus relaciones y la forma en que colaboran para soportar las funciones del sistema. Esta vista representa al SCD desde un enfoque orientado a objetos y organiza sus elementos en capas claramente definidas para asegurar modularidad, mantenibilidad y coherencia con los procesos operativos descritos en las secciones anteriores.

Esta vista se construye a partir del Punto de Vista Lógico documentado en la Sección 3.2, pero ahora ofrece un modelo formal mediante diagramas UML que concretan la organización interna del sistema. El modelo se estructura siguiendo el patrón

arquitectónico MVC (Modelo-Vista-Controlador), complementado con el patrón Observer para el manejo de notificaciones internas. Esta separación permite distribuir las responsabilidades de manera clara y mantener la independencia entre presentación, lógica de negocio y controlador, evitando acoplamientos innecesarios y facilitando futuras extensiones.

El Modelo agrupa las clases que representan las entidades fundamentales del dominio, como Pedido, Mesa, Producto, Usuario, Inventario y Venta. Estas clases encapsulan la lógica de negocio y las reglas operativas del restaurante, además de gestionar la persistencia y el acceso a la base de datos mediante la clase Database. Los Controladores coordinan las operaciones principales del sistema, gestionando solicitudes provenientes de las distintas interfaces, validando datos, invocando métodos del modelo y actualizando las vistas. Entre los controladores más importantes se encuentra AdminController, encargado de administrar mesas, productos, usuarios, inventario y reportes. Finalmente, las Vistas representan las interfaces con las que interactúan los diferentes actores, como clientes, cocineros, meseros, personal de caja y administradores.

Esta vista también integra componentes transversales como NotificationManager y StockObserver, que permiten reaccionar automáticamente ante eventos relevantes, como inventario bajo o cambios en pedidos. Su incorporación refuerza la arquitectura al proporcionar un mecanismo centralizado para la emisión de alertas sin afectar el comportamiento del modelo ni duplicar lógica en los controladores.

En conjunto, esta vista ofrece una representación clara de la organización interna del SCD y de la colaboración entre sus componentes para sostener las operaciones en tiempo real del restaurante. Además, funciona como puente directo entre los requerimientos funcionales, los diagramas de procesos y las vistas de desarrollo y despliegue, asegurando rastreabilidad total dentro del diseño del sistema

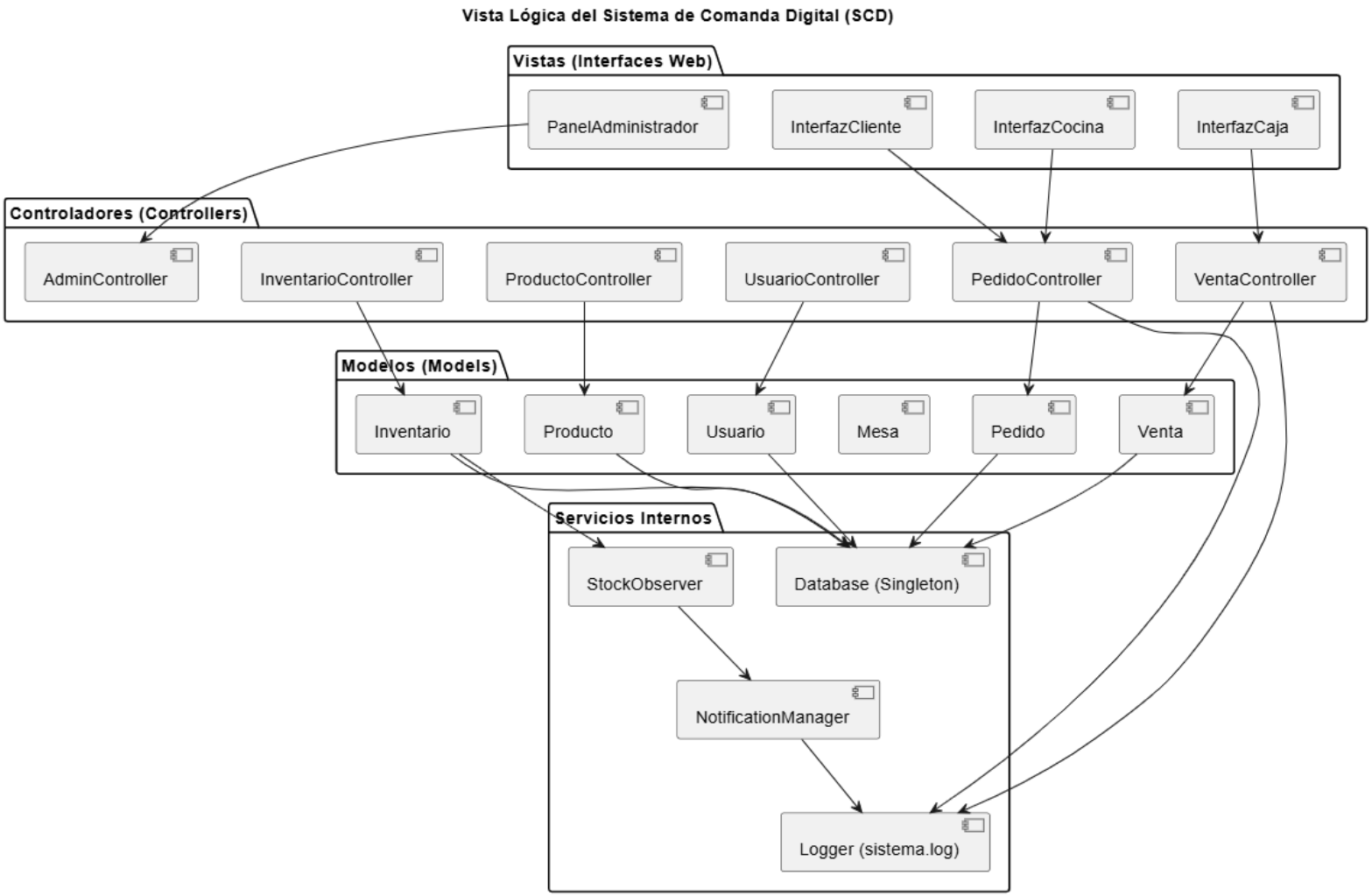


Figura 4.2: Diagrama de componentes de la vista lógica del SCD.

Descripción del diagrama.

El diagrama presenta la estructura interna del Sistema de Comanda Digital (SCD) desde la perspectiva lógica, mostrando cómo se organizan los módulos principales que conforman la aplicación y las relaciones de dependencia entre ellos. La arquitectura se basa en el modelo MVC, donde cada capa concentra responsabilidades específicas y evita mezclas entre la lógica del dominio, la interfaz y los servicios transversales.

La capa de vistas agrupa las interfaces utilizadas por los distintos actores operativos del restaurante, como clientes, cocineros, personal de caja y administrador. Estas vistas se comunican directamente con los controladores, que actúan como intermediarios entre las interfaces y la lógica del negocio. Los controladores gestionan las operaciones del sistema, coordinan la interacción con los modelos y utilizan servicios especializados, como el administrador de notificaciones o el gestor de logs, para manejar procesos complementarios.

La capa de modelos contiene las entidades principales del dominio, como pedidos, productos, inventario, usuarios, ventas y mesas. Estas clases encapsulan reglas de negocio, validaciones y transformaciones de datos. Los modelos acceden a la información persistida mediante la clase Database, ubicada en la capa Core, que proporciona una conexión unificada al sistema de gestión de base de datos y asegura su reutilización.

La capa Core aloja módulos de soporte como NotificationManager, StockObserver y LogSystem, que encapsulan procesos transversales relacionados con la notificación de eventos, seguimiento de inventario y registro de actividad del sistema. Estos servicios pueden ser utilizados tanto por controladores como por modelos cuando se requiere mantener coherencia con los flujos del negocio o registrar información relevante.

El resultado es una estructura clara, coherente y modular, que permite mantener una separación de responsabilidades estricta y favorece la legibilidad y extensibilidad del sistema. Cada dependencia mostrada en el diagrama responde a un propósito funcional y se ajusta al diseño general del SCD.

Justificación.

Esta vista se construye siguiendo las recomendaciones del estándar IEEE 42010, el cual establece que la Vista Lógica debe describir la organización estructural del sistema desde una perspectiva conceptual y de diseño. En este modelo se presenta la arquitectura del SCD como un conjunto de módulos interconectados, cada uno con una responsabilidad bien definida y con relaciones mínimas necesarias para garantizar una operación eficiente.

El uso del patrón MVC como base de esta vista permite lograr un diseño cohesivo, escalable y fácil de mantener. Al separar la interfaz de usuario, la lógica de negocio y los servicios de infraestructura, se posibilita que las modificaciones en una capa no afecten a las demás. Esto es indispensable para sistemas en constante actualización como el SCD, donde se agregan platillos, cambian inventarios o se ajustan funcionalidades de administrador.

La incorporación explícita de módulos como NotificationManager, StockObserver y LogSystem dentro de la capa Core responde a la necesidad de mantener un control estructurado sobre procesos transversales, evitando su dispersión en múltiples partes del sistema. Presentarlos en la vista lógica permite visualizar su rol dentro del funcionamiento interno y su relación con los demás componentes, especialmente con controladores y modelos.

Este diseño facilita además la trazabilidad entre requisitos, casos de uso y componentes, permitiendo que el lector pueda identificar fácilmente dónde se ubica cada responsabilidad, cómo fluye la información y qué parte del sistema interviene en cada operación.

Criterios de consistencia.

La Vista Lógica mantiene coherencia directa con las otras vistas documentadas. Cada componente representado corresponde a un elemento del repositorio descrito en la Vista de Desarrollo y se relaciona con los dispositivos y flujos descritos en la Vista de Despliegue. Todas las dependencias siguen el flujo descrito en el Punto 3.3, lo que

garantiza trazabilidad entre la arquitectura conceptual, el proceso operativo y la implementación real.

Los servicios internos se ubican correctamente dentro de la capa Core, preservando la separación del modelo MVC y evitando inconsistencias con la lógica del sistema. Las relaciones entre vistas, controladores y modelos se mantienen simples, unidireccionales y alineadas con el funcionamiento real del SCD. Gracias a esta consistencia, el diseño se mantiene claro, escalable y adecuadamente documentado conforme a lo requerido por el estándar IEEE 42010.

4.3 Vista de Desarrollo.

La Vista de Desarrollo describe la organización interna del código fuente del Sistema de Comanda Digital (SCD), mostrando cómo se estructura la solución dentro del repositorio y cómo se materializan las decisiones arquitectónicas definidas en las vistas lógica y de procesos. Su propósito es representar la arquitectura de implementación, evidenciando la distribución de responsabilidades en carpetas, clases y módulos, así como las dependencias que se establecen entre ellos durante la ejecución del sistema. Esta vista permite comprender el sistema desde la perspectiva del desarrollador y facilita tareas de mantenimiento, depuración y evolución futura.

El repositorio del SCD se encuentra organizado de manera modular siguiendo el patrón MVC. La carpeta public contiene todos los elementos accesibles externamente, como hojas de estilo, scripts JavaScript, recursos gráficos y el archivo principal index.php que sirve como punto de entrada para los usuarios. La carpeta config agrupa archivos esenciales para la configuración general del sistema, como config.php, donde se declaran rutas base y parámetros globales, y database.php, que implementa una conexión MySQL reutilizable mediante un patrón singleton para garantizar eficiencia y centralización en el acceso a datos.

La carpeta models reúne las clases que representan las entidades del dominio, entre ellas Pedido, Mesa, Producto, Usuario, Venta e Inventario. Estas clases encapsulan la lógica de negocio y se comunican directamente con la clase Database para realizar operaciones sobre la base de datos. Dentro de esta misma carpeta se integra el

submódulo observers, conformado por NotificationManager y StockObserver, encargado de gestionar eventos y notificaciones internas relacionadas con el inventario o el estado de los pedidos, manteniendo una arquitectura reactiva y desacoplada.

La carpeta controllers incluye los módulos que coordinan las operaciones del sistema y sirven como puente entre las vistas y los modelos. AdminController, AuthController y ReportController procesan solicitudes del usuario, validan datos, invocan operaciones del modelo y gestionan acciones transversales como registros en el sistema de logs o activación de observadores. Las vistas, ubicadas en la carpeta views, se organizan por tipo de usuario e implementan la interfaz gráfica para clientes, cocineros, administradores y personal de caja. Estas vistas interactúan únicamente con los controladores, manteniendo una separación clara entre lógica de presentación y lógica de negocio.

Finalmente, la carpeta logs almacena registros de eventos del sistema mediante archivos como sistema.log. Esta sección permite auditoría, detección de errores y análisis de comportamiento, funcionando como un mecanismo complementario de soporte técnico y mejora continua del sistema.

La Vista de Desarrollo demuestra cómo las decisiones arquitectónicas documentadas previamente, como la modularidad del patrón MVC, el manejo centralizado de la base de datos y la existencia de un sistema interno de notificaciones, se proyectan directamente en una estructura de archivos coherente y mantenible. Esta correspondencia garantiza que el diseño conceptual no solo sea adecuado en teoría, sino que se encuentre implementado de manera consistente.

La estructura descrita mantiene trazabilidad con las entidades del modelo lógico y con los actores del sistema definidos en las fases anteriores. Las dependencias entre carpetas reflejan de manera fiel los flujos descritos en la Vista de Procesos, mientras que la separación en módulos facilita la escalabilidad del sistema y la incorporación futura de nuevas funcionalidades sin comprometer la arquitectura existente.

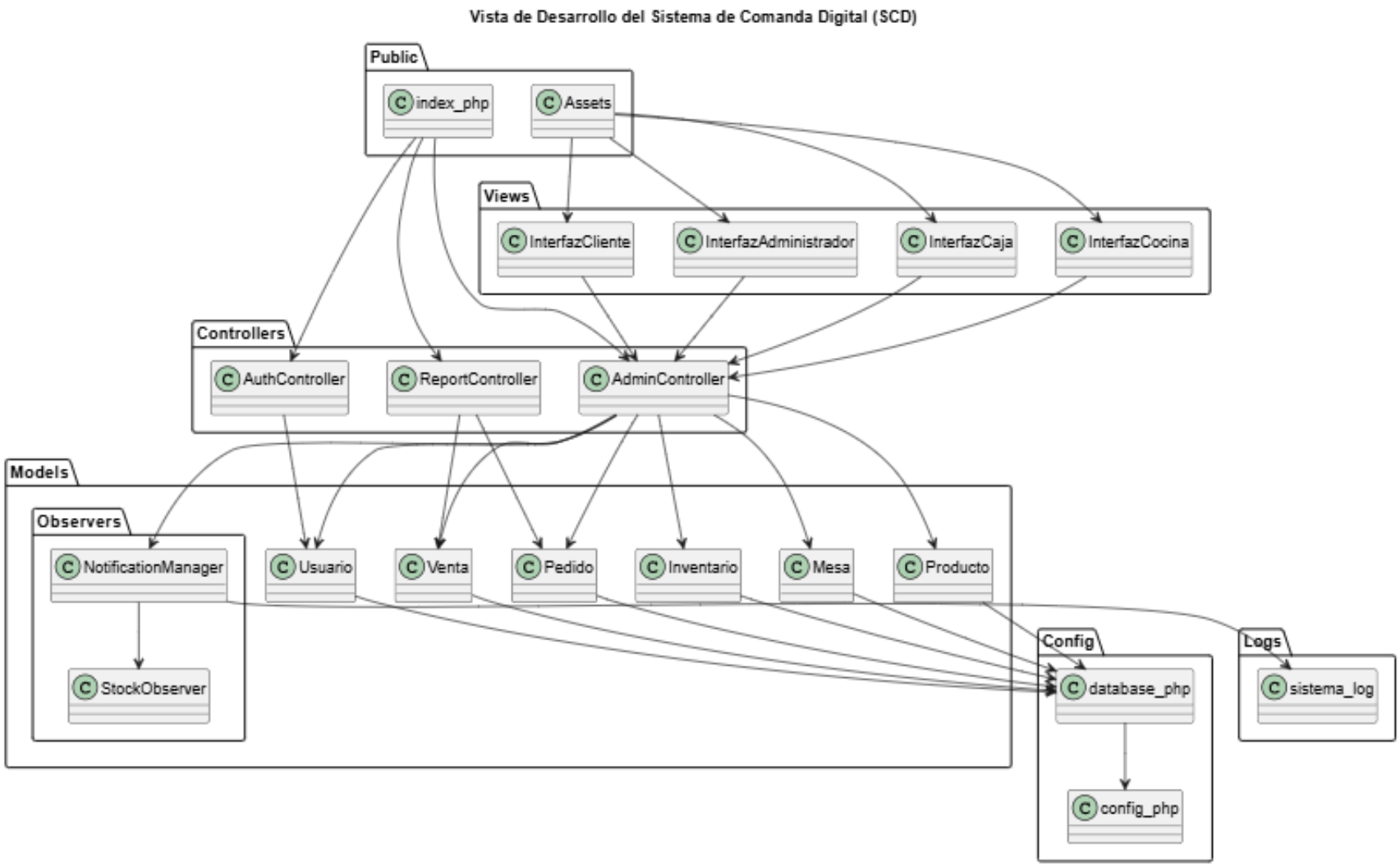


Figura 4.3: Diagrama de paquetes de la vista de desarrollo del SCD.

Descripción del diagrama.

El diagrama de la Vista de Desarrollo representa la estructura interna del proyecto del Sistema de Comanda Digital (SCD) tal como se encuentra organizada en el repositorio. Cada paquete refleja una carpeta física del sistema, y las clases dentro de cada paquete corresponden a archivos o módulos con responsabilidades específicas.

El paquete Public contiene los elementos expuestos al usuario final: el archivo principal *index.php*, que actúa como punto de entrada al sistema, y la carpeta de *Assets*, donde residen los recursos estáticos utilizados por las interfaces.

El paquete Config incluye los elementos responsables de la configuración global del proyecto. *config.php* define constantes esenciales como rutas del sistema y credenciales de acceso, mientras que *database.php* implementa la clase *Database*, encargada de centralizar la conexión a MySQL utilizando un patrón singleton.

El paquete Models representa la capa del dominio del sistema. Contiene las clases que modelan las entidades principales: *Pedido*, *Producto*, *Mesa*, *Usuario*, *Inventario* y *Venta*. Dentro de este paquete se encuentra también el subpaquete Observers, el cual implementa los mecanismos de notificación mediante *NotificationManager* y *StockObserver*, encargados de emitir alertas y registrar eventos relevantes del sistema.

El paquete Controllers agrupa la lógica de control mediante *AdminController*, *AuthController* y *ReportController*. Estos controladores reciben solicitudes desde las vistas, gestionan reglas de negocio y coordinan la interacción con los modelos y servicios auxiliares.

El paquete Views reúne las interfaces para los diferentes usuarios del sistema: cliente, cocina, caja y administrador. Cada interfaz está organizada como un módulo independiente que se comunica con los controladores para solicitar datos y ejecutar operaciones.

Finalmente, el paquete Logs contiene el archivo *sistema.log*, utilizado para registrar eventos operativos, fallas, alertas y acciones importantes, lo cual mejora la trazabilidad del sistema.

Las relaciones del diagrama muestran cómo las vistas se comunican con los controladores, cómo estos invocan a los modelos, cómo los modelos consultan la base de datos y cómo los servicios transversales (Observers, Logs y Config) se integran sin romper la estructura del MVC.

Justificación del diagrama.

La estructura presentada en el diagrama respeta y fortalece la arquitectura basada en el patrón MVC, manteniendo una separación clara de responsabilidades entre presentación, lógica de negocio y acceso a datos.

El subpaquete Observers dentro de Models no rompe el MVC, ya que los observadores forman parte del dominio y operan sobre el estado de los modelos. Su propósito es reaccionar a cambios en las entidades (por ejemplo, verificar stock al registrar una venta) sin involucrar la capa de presentación ni la capa de control. Esto es coherente con los principios del diseño orientado a objetos, donde los modelos pueden poseer comportamientos complementarios a su lógica de dominio.

Los paquetes Public, Config y Logs se encuentran fuera del MVC de manera intencional. Public representa el punto de entrada y los recursos accesibles; Config contiene parámetros globales que no pertenecen a ninguna capa funcional; y Logs actúa como un servicio transversal que registra eventos del sistema. Su ubicación externa evita acoplamientos innecesarios y mantiene la arquitectura limpia y modular.

Las relaciones del diagrama evidencian el flujo correcto dentro del MVC: las vistas se comunican exclusivamente con los controladores, los controladores operan sobre los modelos, y los modelos acceden a la base de datos a través de la clase centralizada *Database*. Esto garantiza coherencia, facilita el mantenimiento y permite modificar o ampliar partes del sistema sin afectar otras capas.

Esta organización también refleja fielmente las decisiones documentadas en vistas anteriores, como la centralización de la conexión en *database.php*, la presencia del sistema de notificaciones y la modularidad del repositorio.

Criterios de consistencia.

La Vista de Desarrollo debe mantener trazabilidad y coherencia con todas las demás vistas del documento. Cada paquete y componente aquí representado corresponde a un artefacto real en el repositorio: las clases listadas en el paquete Models deben existir como archivos en la carpeta /models, los controladores en /controllers, las interfaces en /views, index.php y recursos en /public, y los archivos de configuración en /config. Cualquier discrepancia entre el diagrama y la estructura física del repositorio se considera una inconsistencia que debe corregirse ya sea actualizando el código o ajustando la documentación.

Las dependencias mostradas en el diagrama son contractuales: las vistas solo deben invocar a controladores; los controladores deben delegar la lógica al modelo y no realizar acceso directo a recursos de presentación; los modelos deben acceder a la persistencia a través de database.php; y los servicios transversales (NotificationManager, StockObserver, logs) actúan como componentes de soporte que pueden ser usados por controladores y modelos sin romper la separación de responsabilidades. Si durante la implementación se detecta una dependencia inversa o acoplamiento indebido (por ejemplo, un modelo que renderiza HTML o una vista que accede directamente a la base de datos), esa situación debe registrarse como un hallazgo y resolverse mediante refactorización o corrección de la vista de desarrollo.

Todas las entidades y relaciones del diagrama mantienen correspondencia con los requerimientos y casos de uso documentados en la Sección 3; cada controlador representado tiene trazabilidad hacia uno o más casos de uso y cada modelo debe poder relacionarse con requisitos funcionales y entradas del inventario de requisitos. Para efectos de verificación, se recomienda incluir en el repositorio una tabla de trazabilidad que relacione archivos (por ejemplo, controllers/AdminController.php, models/Pedido.php, config/database.php) con los elementos del AD (casos de uso,

escenas de proceso y vistas correspondientes), y actualizarla cada vez que se modifique alguno de los artefactos.

4.4 Vista de Despliegue.

La Vista de Despliegue presenta la estructura física y tecnológica sobre la cual opera el Sistema de Comanda Digital (SCD). Su propósito es mostrar cómo los componentes de software previamente definidos se distribuyen en los distintos nodos de hardware que integran la infraestructura del restaurante. Esta vista materializa el Punto de Vista de Despliegue documentado en la Sección 3.5, proporcionando ahora una representación formal mediante un modelo UML que detalla la relación entre dispositivos, el servidor local y la red interna.

El sistema se ejecuta en un entorno basado en una arquitectura cliente-servidor dentro de una red LAN privada. El servidor local aloja los componentes centrales del SCD, incluyendo la aplicación web, la base de datos MySQL, los archivos de configuración y el módulo de notificaciones implementado mediante NotificationManager y StockObserver. Desde este servidor se gestiona toda la lógica, el procesamiento de pedidos, la auditoría mediante sistema.log y la sincronización con los distintos dispositivos del restaurante.

Los clientes interactúan con el sistema desde tablets instaladas en cada mesa, conectadas al servidor mediante navegadores web que consumen la InterfazCliente. La cocina utiliza un dispositivo conectado por LAN o Wi-Fi que ejecuta la InterfazCocina para visualizar y actualizar los pedidos. El módulo de caja incluye una computadora de cobro y una impresora térmica, ambas integradas al servidor para el procesamiento de pagos y generación de comprobantes. Finalmente, el administrador accede a la InterfazAdministrador desde un equipo conectado a la misma red, donde consulta métricas, inventario y reportes generales del restaurante.

La red local funciona como el medio de comunicación entre todos los nodos. Al utilizar direccionamiento privado y un router o switch central, se garantiza baja latencia, estabilidad y operación continua sin depender de conexión a internet. Este entorno asegura que los pedidos se sincronicen en tiempo real, que las notificaciones se

propaguen de inmediato y que los registros del sistema queden almacenados correctamente para auditoría y soporte técnico.

Esta vista permite comprender cómo el diseño lógico del sistema se concreta en un despliegue real, asegurando que los componentes trabajen de forma coordinada y consistente dentro de la infraestructura física del restaurante.

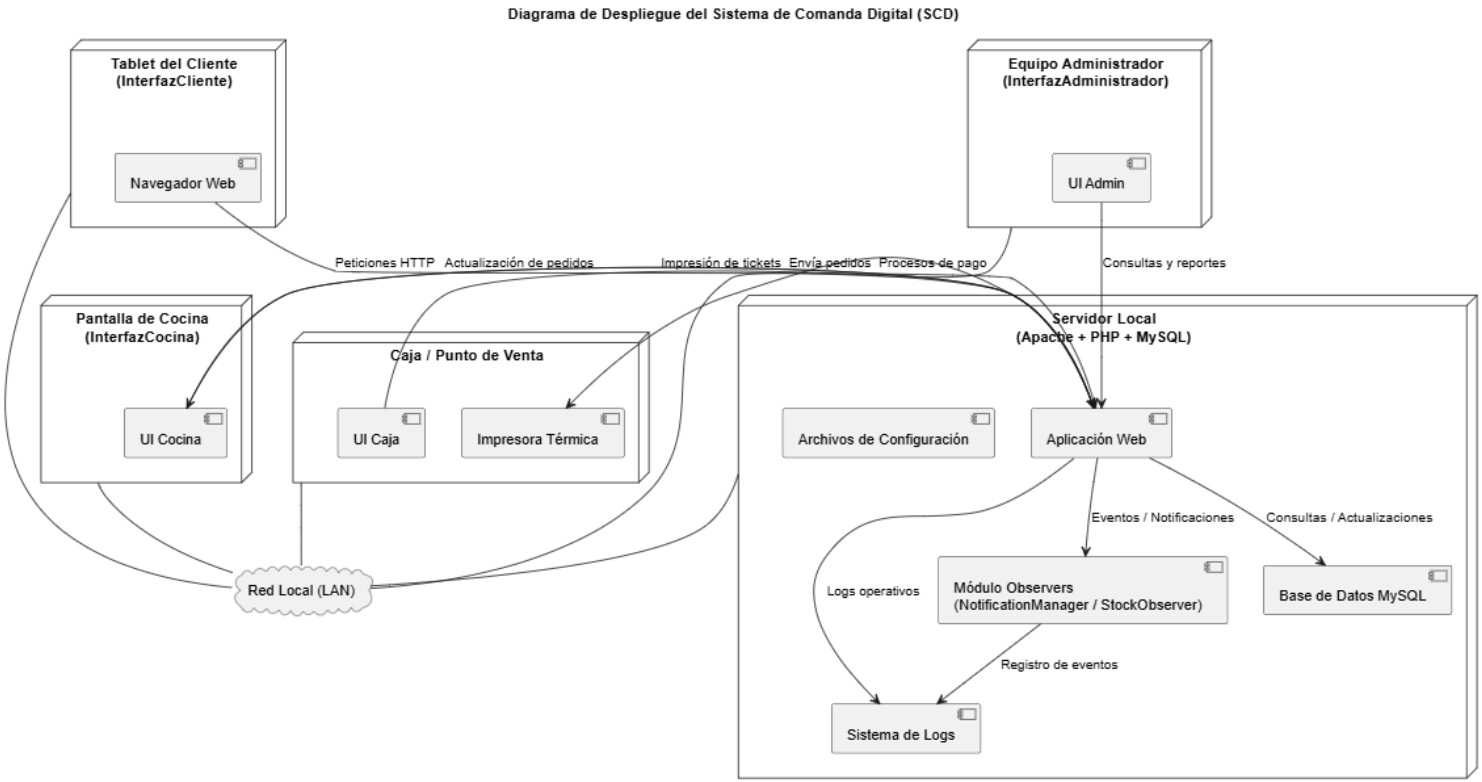


Figura 4.4: Diagrama de despliegue UML de la vista de despliegue.

Descripción del diagrama.

El diagrama de la Vista de Despliegue representa la infraestructura física en la que opera el Sistema de Comanda Digital (SCD) dentro del restaurante. Cada nodo del modelo corresponde a un dispositivo real conectado a la red local, y en su interior se muestran los componentes de software o servicios principales que se ejecutan en dicho dispositivo. El Servidor Local constituye el núcleo del sistema, ya que aloja la aplicación web del SCD, la base de datos MySQL, los archivos de configuración, el sistema de notificaciones implementado mediante NotificationManager y StockObserver, así como los registros almacenados en sistema.log. Desde este nodo se gestionan todos los procesos críticos, como el procesamiento de pedidos, la actualización del inventario y la sincronización de los estados de preparación.

Los clientes interactúan con el SCD a través de las tablets instaladas en las mesas, donde únicamente se ejecuta un navegador web encargado de cargar la InterfazCliente desde el servidor. De manera similar, en la cocina se utiliza un dispositivo conectado a la red para ejecutar la InterfazCocina, permitiendo visualizar y actualizar los pedidos en tiempo real según su avance. El módulo de caja incluye un equipo de cómputo que ejecuta la InterfazCaja y controla la impresora térmica, la cual se sincroniza con el servidor para generar tickets de pago directamente desde la aplicación. Finalmente, el administrador accede a la InterfazAdministrador mediante un navegador desde cualquier computadora conectada a la misma LAN, desde donde puede consultar reportes, métricas e inventarios.

Todos los dispositivos se comunican a través de la red privada del restaurante, representada en el diagrama como LAN. Esta red constituye el medio de comunicación que permite el intercambio de solicitudes y respuestas entre los clientes, empleados y el servidor central. La infraestructura propuesta asegura que los procesos se ejecuten con baja latencia, estabilidad y autonomía, incluso sin acceso a internet, lo cual es fundamental para garantizar el funcionamiento continuo del restaurante.

Justificación del diagrama.

Este diagrama se elabora siguiendo las recomendaciones del estándar IEEE 42010, el cual indica que una Vista de Despliegue debe representar de forma clara la arquitectura física del sistema, diferenciando los nodos de hardware y los artefactos de software que residen en ellos. La arquitectura seleccionada se basa en un modelo cliente-servidor tradicional, adecuado para sistemas internos de restaurante por su fiabilidad, baja dependencia de servicios externos y facilidad de mantenimiento. Al centralizar la aplicación y la base de datos en un único servidor local, se asegura una administración coherente del sistema y una sincronización inmediata entre todos los módulos.

La separación de dispositivos según el rol que desempeñan en la operación del restaurante permite reflejar de manera precisa cómo se distribuyen las interfaces del SCD. Las tablets de los clientes, el dispositivo de cocina, el equipo de caja y la computadora del administrador representan puntos de interacción independientes que consultan la misma fuente de información alojada en el servidor. Esto facilita la escalabilidad del sistema, ya que nuevos dispositivos pueden añadirse a la red sin alterar la arquitectura central.

Además, la presencia de módulos como Logs y Observers dentro del servidor permite reforzar la trazabilidad y el comportamiento reactivo del sistema, elementos ya definidos en la Vista Lógica y en la Vista de Desarrollo. Su ubicación en el nodo principal asegura que estos componentes funcionen de manera consistente con el flujo de datos general y que puedan supervisar adecuadamente los eventos generados por las distintas áreas del restaurante.

Criterios de consistencia.

La Vista de Despliegue mantiene coherencia con todas las vistas elaboradas previamente. Los nodos representados en el diagrama corresponden directamente a los actores y módulos descritos en la Vista de Contexto, mientras que la distribución de interfaces en los distintos dispositivos coincide con los elementos definidos en la Vista Lógica y la organización establecida en la Vista de Desarrollo. Cada componente ubicado en el Servidor Local tiene un equivalente en la estructura

del repositorio: la aplicación web, la base de datos, los archivos de configuración, el sistema de notificaciones y el módulo de registros.

Las interacciones representadas respetan los flujos de comunicación establecidos en la Vista de Procesos. Las tablets solo consumen la InterfazCliente mediante navegador, los dispositivos de cocina y caja actúan como estaciones de operación y el administrador ingresa al sistema mediante su propia interfaz dedicada. De igual manera, las flechas del diagrama muestran únicamente las comunicaciones permitidas por la arquitectura MVC descrita previamente: ninguna vista accede directamente a la base de datos, ningún controlador reside en los nodos cliente y toda la lógica de negocio permanece en el servidor.

Finalmente, la infraestructura propuesta garantiza que los requisitos funcionales y no funcionales definidos en las secciones anteriores puedan cumplirse. La disposición en red privada asegura tiempos de respuesta adecuados, comunicación estable y disponibilidad interna; la centralización del procesamiento evita inconsistencias en los pedidos y el almacenamiento local de registros facilita auditoría y depuración. Esta alineación confirma que la Vista de Despliegue refleja fielmente la arquitectura esperada y permite implementar el SCD en un entorno real sin riesgo de incoherencias o contradicciones.

5. Correspondencias y Consistencia.

La sección de Correspondencias y Consistencia describe cómo las diferentes vistas arquitectónicas del sistema se relacionan entre sí y cómo se garantiza que todas las decisiones de diseño mantengan coherencia a lo largo del modelo. Esta sección no solo documenta qué se relaciona, sino también por qué se tomó cada decisión y cómo contribuye a la estabilidad, trazabilidad y mantenibilidad del Sistema de Comanda Digital.

La importancia de esta sección radica en que, si bien cada vista (lógica, procesos, despliegue, etc.) presenta un nivel diferente de detalle, todas deben narrar la misma historia del sistema, evitando contradicciones, duplicidades o elementos huérfanos.

5.1 Relaciones entre vistas.

Las relaciones entre vistas arquitectónicas constituyen un eje fundamental del diseño, ya que permiten que el sistema sea analizado desde múltiples perspectivas sin perder coherencia interna. Cada vista resuelve un conjunto específico de preocupaciones y, cuando se relacionan adecuadamente, permiten comprender el sistema de forma integral.

5.1.1 Vista de Contexto.

La Vista de Contexto constituye el punto de partida fundamental para comprender el sistema en su totalidad, ya que establece los límites de la solución y define cómo interactúa con su entorno. Esta vista permite mostrar, de manera clara y sintetizada, qué hace el sistema, quién lo utiliza, y qué expectativas existen sobre él antes de adentrarse en detalles técnicos o estructurales.

En esta etapa, se representan los actores externos, tales como el Cliente, el Mesero, el Cocinero y el Administrador, junto con las funciones principales que cada uno realiza dentro del sistema. La finalidad de esta vista es ofrecer una perspectiva global que responda a la pregunta: ¿Cuál es el propósito del sistema y para quién fue construido?

La razón por la cual se desarrolla esta vista en primera instancia es que funciona como un puente entre los requisitos del usuario y el diseño técnico posterior. Es decir, antes de modelar clases, secuencias o componentes físicos, se necesita tener una comprensión sólida de los servicios que el sistema debe proporcionar. Esta claridad inicial permite evitar ambigüedades y reduce significativamente el riesgo de inconsistencias en las fases posteriores del diseño.

Además, la Vista de Contexto cumple un papel crucial en la trazabilidad, asegurando que cada actor identificado tenga representación y justificación en las demás vistas de arquitectura. Por ejemplo, el actor “Cliente” no solo aparece en el diagrama de casos de uso, sino que también debe reflejarse en roles, interfaces, controladores y entidades del sistema. Esto garantiza que ninguna funcionalidad quede sin soporte

técnico y que el diseño no introduzca elementos innecesarios o inexistentes en los requerimientos.

Esta vista también permite establecer los límites operativos del sistema: qué procesos pertenecen al software y cuáles quedan fuera. Definir estos límites evita sobrecargar el sistema con responsabilidades que no le corresponden y ayuda a distribuir adecuadamente las funciones entre actores reales (como el Cocinero) y elementos digitales (como la Pantalla de Cocina).

Desde una perspectiva metodológica, la Vista de Contexto también sirve como referencia continua durante la validación del diseño. Al momento de pasar a las vistas lógica, de procesos y de despliegue, esta vista actúa como un mapa general que permite verificar si cada componente del sistema contribuye a cumplir correctamente las funciones descritas. Cualquier actor o proceso definido aquí debe poder rastrearse a lo largo del modelo; si no, se considera una brecha que debe corregirse para garantizar la consistencia arquitectónica.

En síntesis, la Vista de Contexto no solo muestra “quién usa el sistema y para qué”, sino que representa una decisión estratégica en el diseño, ya que establece una visión compartida entre el equipo de desarrollo, el cliente y los stakeholders. Su correcta elaboración es esencial para que las etapas posteriores cuenten con una base sólida, clara y alineada con los objetivos reales del software.

5.1.2 Vista Lógica (Componentes a Clases/Estructura).

La Vista Lógica representa la estructura interna del sistema, modelando entidades, atributos, asociaciones y relaciones de dependencia necesarias para cumplir con los requisitos funcionales.

En ella se detallan componentes como Pedido, Platillo, Inventario, Mesa, Carrito, Usuario y Controladores, todos relacionados mediante clases con responsabilidades claras y específicas.

La adopción del patrón Modelo–Vista–Controlador (MVC) responde a una necesidad de separación estricta entre lógica, presentación y gestión de datos.

Gracias a esta separación: la lógica de negocio no depende de la forma en que se muestran los datos, las vistas no tienen conocimiento de cómo están almacenados los datos, y los controladores actúan como intermediarios regulando el flujo de información.

Esto no solo facilita el mantenimiento y la escalabilidad, sino que permite que el modelo pueda evolucionar sin romper la presentación o viceversa.

La Vista Lógica también funciona como punto de referencia para las vistas de procesos. Es decir, todo lo que aparezca en los diagramas de secuencia debe existir como clase o componente aquí, lo que refuerza la coherencia y evita inconsistencias entre vistas.

5.1.3 Vista de Procesos (Clases a Secuencias/Comportamiento Dinámico).

La Vista de Procesos se presenta mediante diagramas de secuencia que explican la interacción temporal entre los elementos del sistema. Estas secuencias ilustran cómo los actores externos disparan eventos que involucran a controladores, modelos y base de datos.

En particular, el caso de uso “Realización de Pedidos” sirve como un flujo representativo donde intervienen múltiples componentes:

1. El Cliente selecciona platillos mediante la Vista.
2. El Controlador valida la disponibilidad en el Inventario.
3. El Modelo Pedido registra la operación.
4. La Base de Datos almacena los cambios.
5. La Pantalla de Cocina recibe actualizaciones.
6. El Cocinero cambia el estado del pedido conforme avanza la preparación.

Este flujo demuestra cómo los elementos estáticos de la Vista Lógica se materializan en un comportamiento dinámico y real dentro del sistema.

Además, esta vista valida que las clases cumplen con la responsabilidad que se les asignó. Si en un diagrama de secuencia aparece un método que no existe en la clase correspondiente, entonces hay una inconsistencia que debe resolverse. Por ello, el diagrama de secuencia no solo documenta un proceso: es una herramienta de verificación interna de la arquitectura.

5.1.4 Vista de Desarrollo y Despliegue (Implementación Física).

La Vista de Desarrollo organiza los elementos lógicos en archivos, paquetes y módulos físicos. Este mapeo es esencial, ya que permite mantener una estructura clara en el código y garantiza que el repositorio refleje fielmente el diseño arquitectónico.

Ejemplos comunes incluyen:

Pedido.php para la lógica del modelo Pedido, AdminController.php para la gestión de solicitudes administrativas, Database.php como capa central de acceso a datos.

Este nivel de estructura facilita el mantenimiento del sistema y la localización de componentes específicos, evitando confusiones entre desarrolladores.

Por otro lado, la Vista de Despliegue detalla cómo estos componentes se instalan y ejecutan en un entorno físico. El sistema involucra:

1.- un servidor local donde reside la lógica completa,

2.- tablets o dispositivos del cliente/mesero,

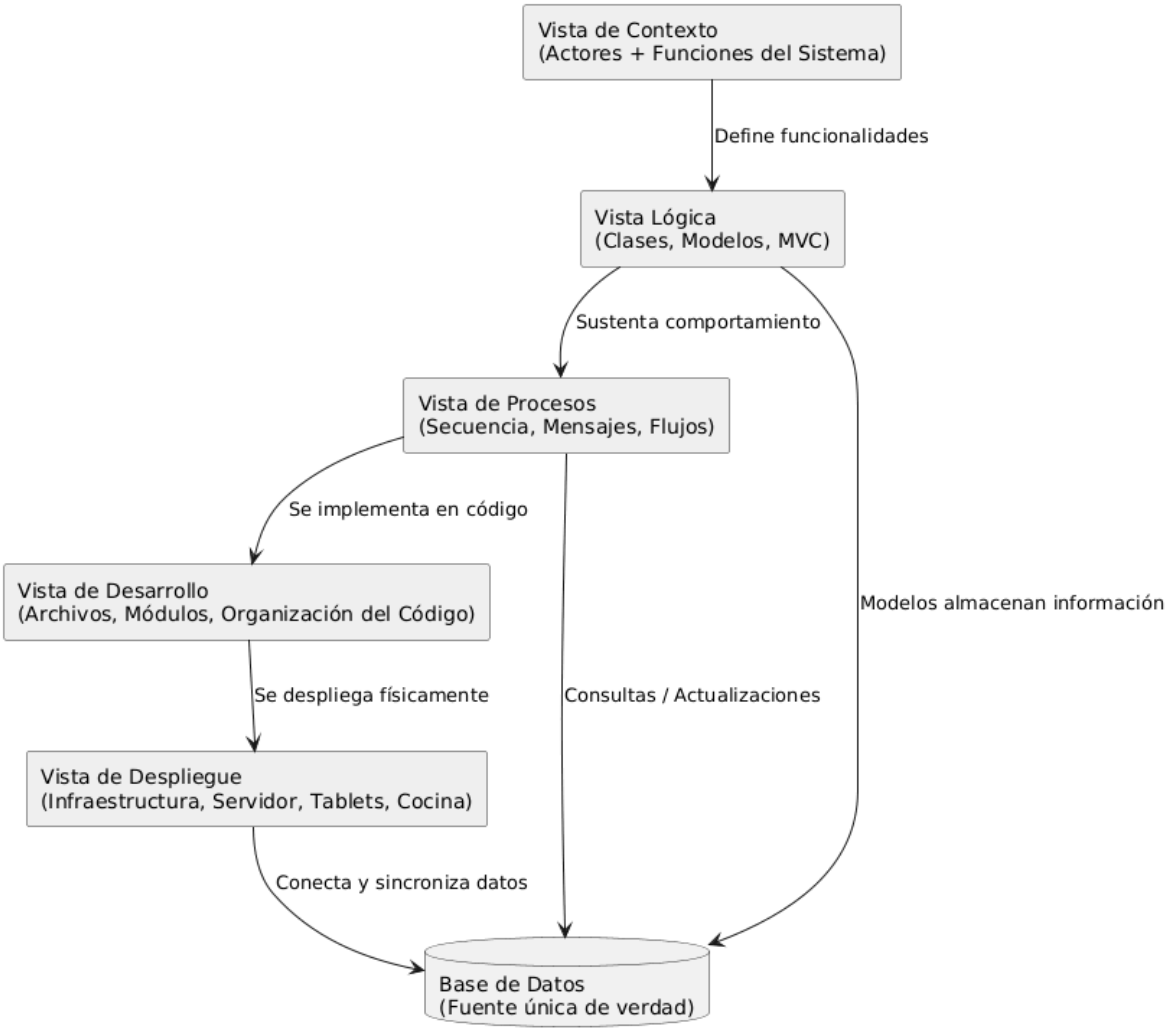
3.- pantallas de cocina conectadas a la red,

una infraestructura LAN que soporta el intercambio constante de información.

La principal razón para incluir esta vista es asegurar que la infraestructura física realmente soporta el comportamiento descrito en la Vista de Procesos. Por ejemplo, si la pantalla de cocina necesita recibir pedidos en tiempo real, la red debe tener baja latencia y disponibilidad adecuada.

Si el servidor almacena la base de datos centralizada, los dispositivos deben mantener una conexión estable para evitar pérdidas de información.

En otras palabras, el despliegue valida la viabilidad real del sistema más allá del diseño conceptual.



5.2 Reglas de Consistencia aplicadas en los modelos UML.

Para garantizar un diseño sólido, se aplican un conjunto de reglas de consistencia que alinean todas las vistas arquitectónicas. Estas reglas aseguran que cada diagrama represente la misma realidad del sistema desde diferentes perspectivas, sin contradicciones o huecos.

Las reglas también contribuyen a la trazabilidad, facilitando el proceso de verificación, pruebas y mantenimiento del software.

A continuación, se describen las reglas con mayor detalle:

Trazabilidad de Actores y Elementos.

Cada actor identificado debe aparecer representado a lo largo del diseño. Esta trazabilidad garantiza que todas las necesidades del sistema estén contempladas y respaldas por componentes internos.

Si el actor Mesero aparece en la Vista de Contexto, su interacción debe observarse también en los casos de uso, en los controladores que atienden sus operaciones y en las interfaces con las que interactúa.

Justificación: Evita dejar actores o funcionalidades sin soporte técnico y previene agregar elementos que no provienen de requisitos reales.

Coherencia Lógica–Funcional.

Cada clase o componente dentro del diagrama lógico debe tener una relación directa con uno o más requisitos funcionales. No deben existir clases “decorativas” sin propósito.

Justificación: Permite validar que el sistema no se complique innecesariamente y que cada elemento cumpla un objetivo funcional específico. Además, asegura que la Vista Lógica da soporte completo a los Casos de Uso.

Consistencia de Información.

Todas las vistas del sistema deben basarse en la misma fuente de datos. En este caso, la base de datos centralizada actúa como el único origen de verdad para clientes, cocina y administración.

Justificación: Evita discrepancias en los datos, como que un pedido aparezca actualizado en un módulo, pero no en otro. Esto es crucial en sistemas donde la sincronización es esencial.

Separación MVC.

El sistema debe respetar estrictamente el patrón Modelo–Vista–Controlador. Esto significa que:

Las Vistas nunca deben acceder directamente a la base de datos, los Modelos nunca deben generar contenido visual, los Controladores deben actuar como intermediarios regulando el flujo.

Justificación: Este principio protege la arquitectura de acoplamientos innecesarios, facilita la reutilización y mejora la capacidad de mantenimiento.

Consistencia Procesos–Estructura.

Todo lo que se modela en los diagramas de secuencia debe estar soportado por componentes existentes. Si una instancia en un diagrama de secuencia se comunica con un objeto inexistente en la Vista Lógica, hay una inconsistencia.

Justificación: Sin esta regla, podrían documentarse procesos imposibles de implementar o clases sin responsabilidad clara.

Consistencia del Despliegue.

El diseño físico del sistema debe ser capaz de soportar las interacciones descritas en la Vista de Procesos. Si un módulo requiere comunicación en tiempo real, la infraestructura debe garantizarla.

Justificación: Asegura la viabilidad práctica del sistema y previene fallas relacionadas con el entorno físico.

Reglas de Correspondencia (ISO/IEC/IEEE 42010).

Las Correspondencias definen relaciones entre elementos arquitectónicos, mientras que las Reglas de Correspondencia describen restricciones que deben cumplirse para garantizar coherencia entre vistas.

Por ejemplo, la regla:

“Un pedido solo puede generarse si el carrito contiene platillos y el inventario confirma disponibilidad.”

Refleja una relación lógica entre el Pedido, Inventario y Carrito.

Justificación: Estas reglas permiten detectar, documentar y prevenir violaciones arquitectónicas, actuando como un mecanismo de calidad que asegura integridad y compatibilidad entre los elementos del sistema.

6. Decisiones y Racional de arquitectura.

6.1 Introducción general al proceso de decisiones.

La arquitectura del Sistema de Comanda Digital (SCD) se construyó a partir de un proceso iterativo de análisis, evaluación y selección de alternativas, guiado por los requerimientos funcionales, las necesidades operativas del restaurante y las buenas prácticas establecidas por el estándar IEEE 42010. Este proceso de toma de decisiones buscó asegurar que cada componente del sistema, cada tecnología adoptada y cada estructura implementada respondieran directamente a las preocupaciones de los interesados y permitieran una operación confiable, escalable y comprensible.

Las decisiones arquitectónicas no se tomaron de manera aislada; surgieron de la interacción entre el análisis del dominio, las restricciones tecnológicas del entorno, las prioridades del negocio y la necesidad de mantener una arquitectura clara y mantenible. A lo largo del desarrollo del proyecto se evaluaron diversas alternativas para la organización del código, el diseño lógico, la distribución de componentes, el almacenamiento de datos y los mecanismos de comunicación interna. Algunas decisiones se mantuvieron desde el inicio por su alineación natural con el problema (como el uso del patrón MVC), mientras que otras se ajustaron conforme surgieron nuevos requerimientos, como la inclusión del módulo de caja para validar pagos antes de enviar pedidos a cocina.

El racional documentado en esta sección explica el “por qué” detrás de la arquitectura actual: por qué se eligió un enfoque web, por qué se optó por un servidor centralizado en LAN, por qué se adoptó una base de datos relacional y por qué se integró un sistema de notificaciones basado en observadores, entre muchas otras decisiones. Cada una de estas elecciones impacta directamente en las vistas documentadas en los capítulos 3 y 4, por lo que esta sección complementa y justifica los modelos previamente elaborados.

El objetivo de este racional no es únicamente describir las decisiones, sino demostrar que cada una de ellas responde a necesidades técnicas reales, mejora el desempeño operativo y mantiene coherencia con las exigencias del entorno del restaurante. Esta trazabilidad asegura que la arquitectura del SCD sea sólida, defendible y preparada para futuras extensiones o cambios tecnológicos.

6.2 Decisiones Arquitectónicas Principales.

Las decisiones arquitectónicas principales del Sistema de Comanda Digital (SCD) se tomaron para asegurar que el sistema fuera coherente, mantenible, seguro y adecuado al entorno operativo del restaurante. Cada elección responde a una justificación técnica o funcional y se encuentra directamente relacionada con las vistas y modelos presentados en las secciones anteriores. A continuación, se describen las decisiones fundamentales que dieron forma a la arquitectura final.

La primera decisión clave fue adoptar una arquitectura basada en el patrón Modelo–Vista–Controlador (MVC). Esta estructura permitió separar claramente la lógica de negocio, la gestión de datos y la presentación, favoreciendo la mantenibilidad y la posibilidad de extender el sistema sin afectar a módulos no relacionados. La elección de MVC resultó apropiada debido a la naturaleza del proyecto, el número de actores involucrados y la necesidad de mantener interfaces independientes para clientes, cocina, caja y administración.

Se decidió también implementar el sistema como una aplicación web centralizada desplegada en un servidor local. Esta elección surgió de la necesidad de garantizar

tiempos de respuesta mínimos y operación continua incluso ante fallos de conexión a internet. Además, un servidor central facilita la administración, evita instalaciones individuales en cada dispositivo y permite mantener todos los módulos sincronizados de manera uniforme a través de una red LAN interna.

El uso de una base de datos relacional fue otra decisión relevante. La operación del restaurante implica manejar información estructurada, con relaciones claras entre pedidos, platillos, inventario, mesas, usuarios y ventas. Un modelo relacional permite garantizar integridad referencial, realizar consultas consistentes y asegurar la trazabilidad de los datos. Alternativas como bases de datos NoSQL fueron descartadas debido a que no ofrecían ventajas claras para el tipo de información que maneja el sistema y podrían complicar la gestión de consistencia que el entorno requiere.

La integración de un sistema de notificaciones basado en el patrón Observer representó una decisión orientada a mejorar la reactividad del sistema. Con NotificationManager y StockObserver se logra que ciertos eventos internos —como cambios en inventario o actualizaciones de estado de pedidos— generen alertas automáticas sin necesidad de acoplar lógicamente los módulos entre sí. Esta solución evita dependencias innecesarias y proporciona flexibilidad para escalar o modificar la lógica de notificaciones en el futuro.

Otra decisión importante fue incluir el módulo de caja como parte central del flujo de pedidos. Inicialmente se consideró que los pedidos podrían enviarse directamente a cocina, pero esta aproximación generaba inconsistencias en la operación real del restaurante, especialmente en el control de pagos y en la trazabilidad del inventario. Por ello, se redefinió la arquitectura para que la cocina recibiera únicamente pedidos ya pagados, lo que redujo errores, evitó mermas y alineó mejor el sistema con el funcionamiento cotidiano del negocio.

Finalmente, la incorporación del botón de asistencia en la InterfazCliente respondió tanto a necesidades operativas como de experiencia de usuario. El restaurante requiere un mecanismo eficiente para que el cliente solicite apoyo del mesero sin

interrumpir su actividad, y la arquitectura del sistema permitió integrar esta funcionalidad sin alterar los componentes principales. La decisión se sustentó en la necesidad de mantener una comunicación fluida entre las mesas y el personal, mejorando la atención sin incrementar la complejidad técnica.

Estas decisiones forman la base de la arquitectura actual del SCD y aseguran coherencia entre el diseño conceptual, la implementación técnica y el funcionamiento real del restaurante. En conjunto, garantizan una solución robusta, escalable y alineada con los requerimientos del proyecto.

6.3 Decisiones sobre Tecnologías y Plataformas.

Las decisiones tecnológicas que sustentan el Sistema de Comanda Digital (SCD) se establecieron considerando la viabilidad del proyecto, las restricciones del entorno real del restaurante, la experiencia técnica disponible y la necesidad de garantizar estabilidad operativa dentro de una red local. Esta sección documenta y justifica la elección de las tecnologías principales que conforman la plataforma del sistema, explicando por qué fueron seleccionadas frente a posibles alternativas y cómo estas decisiones impactaron en la arquitectura general del proyecto.

La primera decisión clave consistió en seleccionar una base de datos relacional, optando específicamente por MySQL. Esta elección se fundamentó en las características propias del dominio del sistema, donde la información maneja relaciones directas y estructuradas entre entidades como pedidos, productos, usuarios, inventario y ventas. Un modelo relacional permite mantener integridad referencial, aplicar restricciones, ejecutar consultas complejas de manera eficiente y garantizar trazabilidad entre registros, procesos que resultan imprescindibles para evitar inconsistencias en operaciones tan sensibles como el registro de un pedido, la actualización del inventario o la generación de ventas. Aunque se consideraron bases de datos no relacionales, estas no ofrecían ventajas significativas para el tipo de información del SCD y habrían implicado una mayor complejidad innecesaria en la implementación.

La plataforma principal para la ejecución del sistema se basó en PHP como lenguaje de desarrollo y Apache como servidor web. Esta decisión obedeció a su estabilidad, facilidad de despliegue en entornos locales, bajo costo operativo y compatibilidad directa con MySQL. PHP permite una curva de aprendizaje accesible y favorece el uso del patrón MVC, además de ser ampliamente utilizado en aplicaciones administrativas y de comercio. Apache, por su parte, proporciona una configuración sencilla, soporte robusto en LAN y excelente rendimiento para servicios web internos. La combinación PHP + Apache + MySQL resulta especialmente adecuada para aplicaciones autocontenidas en un solo servidor, como las que se ejecutan dentro de un restaurante.

En relación con los dispositivos cliente, se decidió que las tablets utilizadas en mesas, cocina, caja y administración accedieran al sistema mediante un navegador web, en lugar de desarrollar aplicaciones nativas o híbridas. Esta decisión simplificó el despliegue, eliminó la necesidad de instalar software en cada dispositivo y permitió actualizaciones centralizadas desde el servidor. El uso del navegador garantiza compatibilidad universal, reduce costos de mantenimiento y aprovecha al máximo la arquitectura basada en páginas web, donde cada interfaz se adapta a las necesidades del usuario correspondiente.

Para el módulo de caja se seleccionó una impresora térmica conectada a la estación de cobro. Esta decisión se tomó debido a que los tickets de venta son un requerimiento operativo tradicional en establecimientos gastronómicos y porque las impresoras térmicas tienen un costo reducido, alta fiabilidad y mantenimiento mínimo. Además, al integrarse directamente con el servidor local mediante el controlador de caja, permiten un flujo rápido de generación de comprobantes sin depender de servicios externos.

Finalmente, se decidió implementar el sistema completo sobre una red LAN local en lugar de una solución basada en la nube. Esta elección respondió a la necesidad de garantizar operación continua incluso si no hubiera acceso a internet, reducir la latencia entre pedidos, evitar dependencias externas y proteger información

sensible del restaurante. La arquitectura cliente-servidor en LAN asegura sincronización inmediata entre mesas, cocina y caja, además de proporcionar control total del entorno técnico.

Todas estas decisiones tecnológicas no solo respondieron a requerimientos funcionales y restricciones del contexto, sino que también consolidaron una arquitectura robusta, accesible y coherente con los objetivos generales del Sistema de Comanda Digital. Su impacto se refleja en la simplicidad del despliegue, la facilidad de mantenimiento y la coherencia entre las vistas documentadas en las secciones anteriores.

6.4 Decisiones sobre Modelado y Datos.

El modelado del Sistema de Comanda Digital (SCD) se construyó con base en los requerimientos funcionales, las preocupaciones de los interesados y la necesidad de representar de manera fiel los procesos reales del restaurante. Las decisiones tomadas en torno a las entidades, relaciones y estructuras de datos no surgieron únicamente de la intuición del diseño, sino de un análisis directo de cómo fluye la información dentro de la operación diaria. Cada entidad incluida en el modelo responde a una función concreta y necesaria, asegurando que el sistema mantenga coherencia, trazabilidad y capacidad de auditoría.

La decisión de utilizar entidades como Pedido, Producto, Usuario, Mesa, Inventario y Venta se fundamenta en la identificación de los objetos reales que participan en el ciclo completo de atención dentro del restaurante. Un Pedido, por ejemplo, no es simplemente un registro aislado, sino un elemento central que conecta al cliente, la cocina, la caja y la administración. Por ello, se decidió modelarlo con estados definidos, reflejando su ciclo de vida desde que el cliente realiza la solicitud hasta que el pedido es entregado y finalmente pagado. Estos estados permitieron establecer reglas claras para las notificaciones, la actualización en cocina y la visualización en caja, evitando inconsistencias como pedidos duplicados, pendientes o no entregados.

La entidad Inventario se incluyó como un módulo independiente debido a su relevancia operativa. En un restaurante, la disponibilidad de insumos afecta directamente la capacidad de preparar platillos, por lo que se decidió vincular cada actualización de inventario con la creación de ventas y la preparación de pedidos. Separar Inventario como entidad propia posibilita una gestión detallada, auditorías internas y el funcionamiento del sistema de alertas mediante StockObserver, que se activa cuando se detectan niveles críticos. Esta separación evita acoplamientos innecesarios y permite que cada módulo del sistema consulte el inventario sin intervenir directamente en él.

La entidad Usuario se definió como un modelo flexible, capaz de representar distintos tipos de roles: administrador, mesero, cocinero y personal de caja. Esta decisión permitió simplificar la lógica de acceso mediante una sola estructura de autenticación, evitando sistemas paralelos y reduciendo complejidad en la validación de permisos. Mantener un único modelo para múltiples roles facilita la administración y evita duplicidad de información, manteniendo la trazabilidad de acciones por usuario.

La entidad Venta se incorporó para garantizar un registro formal de transacciones, asociado tanto al cierre del pedido como a los reportes administrativos. Este modelo permite generar métricas de ingresos, consultas históricas y auditoría de operaciones, funciones imprescindibles para la toma de decisiones del administrador. Además, su inclusión consolidó la integración entre caja e inventario, permitiendo que cada venta genere simultáneamente un movimiento que registre la salida de insumos.

El diseño de datos privilegió la normalización para evitar redundancia y garantizar integridad. Las relaciones entre entidades se definieron manteniendo claves foráneas y reglas de integridad referencial, permitiendo operaciones confiables durante el registro de pedidos, ventas e inventarios. La elección de un modelo relacional contribuyó a definir estructuras claras y garantizar que cualquier transacción refleje correctamente los cambios en todos los módulos del sistema.

Todas estas decisiones de modelado consolidan un sistema coherente con las necesidades operativas del restaurante. Se privilegió la claridad sobre la complejidad, se mantuvo una estructura que facilita mantenimiento y crecimiento, y se garantizó que los datos representen con fidelidad la realidad de la operación. El diseño final del modelo no solo soporta las funciones actuales del sistema, sino que permite la incorporación de futuros módulos sin alterar el núcleo conceptual del SCD.

6.5 Decisiones sobre la Estructura del Código y Modularidad.

Las decisiones relacionadas con la estructura del código del Sistema de Comanda Digital (SCD) surgieron a partir de la necesidad de mantener una arquitectura clara, mantenible y coherente con el modelo conceptual definido en las vistas anteriores. La modularidad y la separación de responsabilidades fueron principios rectores durante el desarrollo, especialmente porque la solución debía ser comprensible para futuros colaboradores y escalable sin generar acoplamientos innecesarios. El patrón MVC se adoptó no solo por su consistencia con los requisitos de presentación y lógica del sistema, sino también porque su organización natural facilita la trazabilidad entre el diseño conceptual, el diseño lógico y la implementación final.

Una de las decisiones clave fue mantener la carpeta Models como el punto central de la lógica del dominio. Dentro de ella se agruparon las entidades principales del sistema, tales como Pedido, Producto, Inventario y Usuario. Estas entidades no solo representan los datos, sino que encapsulan comportamientos propios del dominio del restaurante. Bajo esta lógica, se decidió incorporar el submódulo Observers dentro de la carpeta Models, lo cual inicialmente generaba dudas respecto a si alteraba o no la estructura del MVC. Tras analizarlo, se concluyó que los observadores forman parte directa de la lógica del dominio, ya que reaccionan a eventos del sistema como cambios en inventario o confirmaciones de venta. Su función es monitorear y responder a modificaciones en los modelos, no interactuar con vistas ni manejar la coordinación general del sistema. Por tanto, su ubicación

dentro de Models no rompe el MVC, sino que enriquece el comportamiento interno del dominio y favorece el desacoplamiento entre componentes.

En la capa de controladores se tomaron decisiones orientadas a mantener la modularidad del sistema y evitar la sobrecarga de responsabilidades. Por ello, se separaron los controladores en tres módulos: AdminController, AuthController y ReportController. Esta división obedeció a criterios de cohesión, ya que cada uno opera sobre un conjunto claro de funcionalidades. AdminController maneja operaciones del restaurante como inventario, menú, pedidos y usuarios. AuthController gestiona autenticación y control de acceso, y ReportController centraliza la generación y consulta de reportes. Esta separación evita que un solo controlador se convierta en un punto crítico de complejidad y facilita el mantenimiento a mediano plazo.

Asimismo, se decidió que las vistas se mantendrían completamente dependientes de los controladores y no interactuarían directamente con los modelos. Esto asegura que la presentación permanezca aislada de la lógica de negocio y que cualquier cambio estructural en el código interno no afecte la interfaz de usuario. Las vistas se organizaron en subcarpetas según el módulo que representan (cliente, cocina, administrador y caja), lo cual facilita su localización y la escalabilidad futura del proyecto.

Por otro lado, se determinó que las carpetas Public, Config y Logs debían ubicarse fuera del patrón MVC. Public contiene los elementos accesibles desde el exterior, incluyendo index.php y los recursos estáticos, por lo que su presencia dentro de MVC hubiese generado confusión respecto a su rol. Config reúne archivos que definen parámetros globales del sistema, como rutas y credenciales; no forman parte ni de modelos ni de controladores. Logs almacena el registro completo de eventos del sistema, un servicio transversal importante para auditoría y depuración, pero sin relación directa con las responsabilidades del MVC. Mantener estos paquetes fuera de la estructura de presentación, control y dominio garantiza una arquitectura más limpia y coherente.

Las decisiones tomadas en esta vista responden directamente a la necesidad de mantener claridad estructural, facilitar el mantenimiento del sistema y asegurar que cada módulo tenga un rol claramente delimitado. Su alineación con el patrón MVC y su organización modular reflejan de manera fiel las decisiones documentadas en las vistas lógicas y de desarrollo, garantizando consistencia global en la arquitectura del SCD.

6.6 Decisiones rechazadas o modificadas durante el proyecto.

Durante el desarrollo del Sistema de Comanda Digital (SCD) surgieron diversas decisiones que, tras análisis técnico y operativo, debieron ser modificadas, ajustadas o descartadas. Estas modificaciones representan la evolución natural de la arquitectura conforme se esclarecieron los requerimientos reales del restaurante, se validaron procesos operativos y se identificaron riesgos no previstos en las etapas iniciales del diseño. Registrar estas decisiones es esencial para garantizar transparencia arquitectónica, justificar los cambios implementados y mantener coherencia con las vistas documentadas anteriormente.

Incorporación tardía de la Interfaz de Caja.

Una de las decisiones más relevantes fue la inclusión de la Interfaz de Caja, un módulo que inicialmente no formaba parte del diseño del sistema. El proyecto comenzó centrado en tres áreas principales: clientes, cocina y administración. Sin embargo, conforme se analizaban los flujos operativos reales de un restaurante, se detectó un problema crítico: no existía una instancia responsable de confirmar pagos, registrar ventas y asegurar que únicamente llegaran a cocina los pedidos que habían sido validados por el personal de caja.

En una primera versión, se asumió que los clientes pagarían directamente al mesero o en un proceso externo al sistema, lo cual podía provocar inconsistencias importantes. Por ejemplo, un pedido podía enviarse a cocina sin haberse cobrado, lo que generaba pérdidas, errores contables y riesgos de merma operativa. Tras identificar este punto, se decidió agregar un módulo de caja que cumpliera con las

funciones de registrar pagos, generar tickets a través de una impresora térmica y actualizar el estado del pedido con una confirmación formal de pago.

Esta incorporación afectó diversas vistas del proyecto: en la Vista de Desarrollo se añadió un nuevo módulo y una nueva interfaz; en la Vista de Despliegue se integró un dispositivo adicional y su relación con el servidor; en la Vista Lógica se actualizó el modelo de procesos para incluir la transición del estado “Pendiente” a “Pagado” antes de permitir su envío a cocina. Esta modificación fortaleció la arquitectura al alinearla con la operación real del restaurante y al reducir riesgos operativos significativos.

Refinación del flujo de asistencia

Otra decisión ajustada fue el flujo de asistencia solicitada por el cliente desde la tablet. Al inicio, el sistema enviaba la alerta directamente a la cocina, lo cual era incoherente con la dinámica real del restaurante. Se corrigió este error asignando estas alertas exclusivamente al mesero, lo que permitió conservar la lógica correcta de roles y responsabilidades. Este cambio impactó únicamente el modelo lógico y el flujo operativo, pero no alteró la estructura general del código.

Ajustes en el modelo lógico del inventario.

Durante la implementación del sistema de notificaciones se identificó que el modelo de Inventario no contemplaba estados críticos ni niveles mínimos para activar observadores. Se decidió modificar la entidad Inventario para incluir su umbral de existencia mínima. Esto permitió que StockObserver actuara correctamente y que el sistema tuviera coherencia con los procesos de reposición de insumos. Fue un cambio interno que no modificó la estructura del MVC ni los controladores, pero sí el comportamiento interno del dominio.

Revisión del diagrama de procesos para evitar redundancias.

En etapas iniciales se utilizaba un diagrama de procesos excesivamente detallado, con pasos que reproducían operaciones ya explicadas en casos de uso y vistas

lógicas. Se decidió rehacerlo para enfocarlo en los flujos esenciales y eliminar redundancia documental. Esto permitió que las vistas técnicas tuvieran una carga de información clara y no contradictoria, alineándose mejor con las recomendaciones del IEEE 42010.

6.7 Evaluación de Alternativas Consideradas

La definición de la arquitectura del Sistema de Comanda Digital (SCD) requirió analizar diversas alternativas tecnológicas, estructurales y operativas. Aunque algunas opciones pudieron implementarse desde el punto de vista técnico, se descartaron tras evaluar su viabilidad, impacto en el restaurante, costos, mantenibilidad y coherencia con los requerimientos funcionales del proyecto. A continuación, se presenta una comparación general de las alternativas consideradas frente a las decisiones adoptadas en la arquitectura final.

Tabla comparativa de alternativas vs decisión

Área evaluada	Alternativa considerada	Descripción	Motivo de descarte	Decisión adoptada
Base de datos	Sistema NoSQL (MongoDB)	Modelo flexible sin esquema fijo	No ofrece relaciones fuertes entre pedidos, inventario y ventas; complicaría reportes y métricas del restaurante	MySQL relacional, por consistencia, ACID y facilidad de consultas
Arquitectura	Sistema distribuido basado en microservicios	Alta escalabilidad	Exceso para el tamaño del restaurante; complejidad innecesaria; requiere infraestructura adicional	Servidor único centralizado en LAN
Patrón de diseño	Arquitectura sin MVC (estructurada)	Menor complejidad inicial	Dificulta el mantenimiento, no separa vistas, controladores y modelos; riesgo de “código espagueti”	Patrón MVC, claro, escalable y didáctico
Notificaciones	Llamadas directas modelo → vista	Acoplamiento inmediato	Rompe MVC; vistas dependerían de modelos; difícil de escalar	Patrón Observer colocado en models/observers

Dispositivos cliente	Aplicación nativa Android/iOS	Flujo fluido	Mayor tiempo de desarrollo; mantenimiento múltiple; dependencias de tiendas	Interfaz web en navegador, accesible en cualquier tablet
Red / conectividad	Sistema en la nube (hosting externo)	Acceso remoto	Requiere internet estable; riesgo ante caídas; mayor latencia	LAN local privada, autónoma y estable
Interfaz de caja	Sin módulo de caja (solo cliente–cocina)	Simplicidad inicial	No valida pagos; riesgo de enviar pedidos no pagados a cocina	Módulo de Caja incorporado, que confirma pago antes de liberar pedido
Estados del pedido	Modelo más simple (solo “pendiente” y “listo”)	Flujo corto	Sin control de preparación; no permite métricas ni trazabilidad	Modelo completo: Pendiente de pago → Pagado → En preparación → Listo

Tabla 6.7: Tabla comparativa de alternativas vs desiciones.

Síntesis del análisis y justificación

El análisis comparativo permitió determinar que las alternativas descartadas no ofrecían un beneficio real respecto al contexto del restaurante ni a los objetivos del sistema, y en varios casos añadían complejidad innecesaria. Optar por una base de datos relacional, una arquitectura MVC, una LAN local, un servidor único centralizado y un módulo Observer para notificaciones resultó fundamental para garantizar simplicidad, coherencia técnica y un entorno estable que funcione incluso sin conexión a internet.

Asimismo, la incorporación tardía de la Interfaz de Caja corrigió un riesgo funcional importante: la posibilidad de enviar pedidos no pagados a cocina. Este ajuste impactó en los diagramas de procesos, en la Vista de Desarrollo, en la Vista de Despliegue y en el diseño de estados del pedido, fortaleciendo la arquitectura y alineando el sistema con el funcionamiento real de un restaurante.

Por otro lado, mantener la aplicación como interfaz web en lugar de una aplicación móvil nativa redujo drásticamente tiempos de desarrollo, costos de mantenimiento y problemas de compatibilidad, aprovechando que todos los dispositivos del restaurante pueden ejecutar navegadores modernos.

La decisión de incluir un modelo de estados detallado para los pedidos permitió lograr trazabilidad, controlar tiempos de preparación y sincronizar adecuadamente a los actores del sistema. Esto facilita métricas internas y evita incongruencias operativas.

En conjunto, la evaluación de alternativas asegura que la arquitectura del SCD no solo es técnicamente correcta, sino también práctica, coherente, mantenible y adecuada al contexto real del restaurante.

6.8 Impacto de las Decisiones en las Vistas Arquitectónicas.

Las decisiones arquitectónicas establecidas en las secciones anteriores tuvieron efectos directos sobre cada una de las vistas documentadas en los capítulos 3 y 4. De acuerdo con lo establecido por el estándar IEEE 42010, esta sección presenta

la relación entre dichas decisiones y la manera en que moldearon, restringieron o modificaron la estructura final de la arquitectura del Sistema de Comanda Digital (SCD). Esto asegura que todas las vistas mantengan coherencia entre sí y que la arquitectura global pueda justificarse de forma trazable y verificable.

La adopción de la arquitectura MVC se reflejó en la Vista Lógica y se concretó en la Vista de Desarrollo, donde la separación entre controllers, models y views se materializó en paquetes independientes y con dependencias bien definidas. Esta decisión obligó a reorganizar la estructura del repositorio y determinó qué elementos podían interactuar entre sí, asegurando que las interfaces siempre se comuniquen a través de controladores y que las operaciones del dominio permanezcan encapsuladas en los modelos. Gracias a este enfoque, la Vista de Desarrollo resultó modular, escalable y fácil de mantener.

La elección de una red LAN local influyó directamente en la Vista de Despliegue. Al descartar el uso de servicios externos o infraestructura en la nube, el diagrama de despliegue se simplificó hacia una topología cliente-servidor en la que un único nodo central aloja la aplicación web, la base de datos y los servicios de auditoría. Esta decisión redujo la complejidad y eliminó dependencias externas, lo cual también fortaleció la Vista de Contexto, ya que los actores externos se conectan al sistema únicamente mediante la red interna del restaurante.

La decisión de utilizar un servidor local con Apache, PHP y MySQL impactó en la Vista de Desarrollo al definir el tipo de componentes necesarios en la carpeta config y la naturaleza del archivo database.php. Asimismo, esta decisión condicionó la Vista Lógica, dado que las clases del dominio debían diseñarse para operar con un modelo relacional, lo que derivó en la existencia explícita de entidades como Pedido, Inventario, Mesa, Usuario y Venta. Estas entidades se alinean con la representación física definida en el modelo de datos del servidor.

La incorporación del sistema de notificaciones y el patrón Observer modificó varias vistas del documento. En la Vista Lógica, obligó a definir comportamientos adicionales y flujos de notificación entre modelos y servicios internos. En la Vista de

Desarrollo generó la necesidad del submódulo observers dentro de models, impactando en la modularidad del código. Finalmente, en la Vista de Procesos se representaron los puntos en los que las alertas de stock, de pedidos listos o de cambios de estado intervienen dentro del flujo operativo del restaurante.

La introducción tardía de la Interfaz de Caja tuvo un efecto significativo en todas las vistas. En la Vista de Contexto añadió un nuevo actor externo; en la Vista de Procesos generó un cambio en el flujo del pedido, pues ahora la cocina solo recibe órdenes una vez procesado el pago; en la Vista de Desarrollo obligó a crear nuevas interfaces y dependencias hacia ReportController y AdminController; y en la Vista de Despliegue incorporó un nuevo nodo físico compuesto por la computadora de cobro y la impresora térmica. Esta decisión fortaleció la consistencia global del sistema al representar un flujo de trabajo más realista y alineado con la operación de un restaurante.

En conjunto, estas decisiones generaron una arquitectura interna coherente, donde cada vista complementa a las demás y todas responden al mismo modelo conceptual. La trazabilidad entre decisiones y vistas asegura que el diseño final no sea arbitrario, sino el resultado de elecciones fundamentadas y alineadas tanto con los requerimientos del sistema como con las limitaciones y condiciones del entorno real en el que será utilizado.

6.9 Conclusiones del Racional Arquitectónico.

El análisis y la documentación de todas las decisiones arquitectónicas realizadas durante el desarrollo del Sistema de Comanda Digital (SCD) permiten concluir que la arquitectura obtenida es coherente, estable, justificable y alineada con los objetivos funcionales y operativos del proyecto. Cada decisión se tomó considerando criterios técnicos, restricciones del entorno del restaurante, necesidades de los usuarios y buenas prácticas de ingeniería de software.

La elección de una arquitectura basada en MVC proporcionó una separación clara de responsabilidades, permitiendo que la lógica de negocio, la presentación y el

control evolucionaran de manera independiente. Esto facilitó la escalabilidad del sistema, su mantenimiento y la trazabilidad entre modelos conceptuales y su implementación real. La decisión de utilizar un servidor local con red LAN privada garantizó baja latencia, disponibilidad constante y operación independiente de la conexión a internet, lo cual es crucial para un entorno donde los pedidos deben procesarse en tiempo real.

El uso de MySQL como base de datos relacional brindó integridad, consistencia y trazabilidad de la información, especialmente para pedidos, ventas e inventario. PHP y Apache resultaron adecuados para un despliegue rápido, eficiente y compatible con la infraestructura disponible. La integración del patrón Observer proporcionó un mecanismo reactivo y desacoplado para notificaciones internas, mejorando la eficiencia en el control del inventario y la comunicación entre módulos.

Las vistas arquitectónicas documentadas muestran que todas las decisiones tuvieron un impacto real en la estructura conceptual, lógica, de procesos, desarrollo y despliegue. Incorporaciones como la Interfaz de Caja —añadida tras identificar un riesgo operativo crítico— demuestran que la arquitectura fue capaz de adaptarse a escenarios reales, aumentando la confiabilidad del sistema sin comprometer su diseño base.

El análisis de alternativas descartadas confirmó que las elecciones tomadas fueron las más adecuadas para el alcance y contexto del proyecto. En conjunto, el racional arquitectónico evidencia que la arquitectura del SCD es el resultado de un proceso reflexivo, fundamentado y consistente, alineado con estándares internacionales y prácticas profesionales.

7. Escenarios de Evaluación Arquitectónica.

Los escenarios de evaluación permiten analizar cómo responde la arquitectura del Sistema de Comanda Digital (SCD) ante situaciones críticas o relevantes para su operación. Aunque el sistema aún no ha sido implementado, estos escenarios se plantean de manera teórica con base en el diseño arquitectónico descrito en las

secciones anteriores. Su finalidad es verificar anticipadamente que la estructura del sistema, las decisiones arquitectónicas y los mecanismos internos asegurarán estabilidad, coherencia y confiabilidad en condiciones reales dentro del restaurante.

Cada escenario explora un evento relevante, el estímulo que lo provoca, la porción de la arquitectura que participa y la respuesta esperada del sistema. Con ello se evalúa la solidez del diseño, lo logrado con la modularidad mediante el patrón MVC, el funcionamiento de la red local y la capacidad del servidor central para coordinar solicitudes simultáneas, notificaciones internas y flujos críticos del proceso de comanda.

7.1 Escenario 1: Alta concurrencia de pedidos en hora pico.

Situación: En un periodo de alta demanda, como la hora de comida, diez mesas realizan pedidos de manera simultánea desde sus tablets.

Estímulo: Se generan diez solicitudes HTTP al mismo tiempo desde la InterfazCliente hacia el AdminController.

Respuesta arquitectónica esperada:

La arquitectura basada en servidor centralizado debe procesar las solicitudes mediante el controlador correspondiente, delegando rápidamente la operación a los modelos Pedido y Producto. La conexión MySQL, administrada por database.php bajo un patrón singleton, evita la creación de conexiones redundantes, permitiendo que la base de datos procese múltiples consultas concurrentes de forma eficiente.

El sistema de notificaciones (NotificationManager + StockObserver) debe emitir alertas a la InterfazCocina en tiempo real, sin retrasos perceptibles. La Vista de Cocina visualizará los pedidos en el orden correcto, demostrando que la lógica de sincronización y el uso de una red LAN privada garantizan baja latencia y coherencia en todos los estados de pedido.

7.2 Escenario 2: Caída temporal de la conexión Wi-Fi en la cocina.

Situación: El dispositivo de cocina pierde conexión Wi-Fi durante 30 segundos.

Estímulo: La InterfazCocina queda temporalmente sin recibir pedidos o actualizaciones.

Respuesta arquitectónica esperada:

Debido al uso de arquitectura cliente-servidor, la lógica de negocio permanece íntegra en el servidor local. Los pedidos continúan registrándose correctamente en la base de datos, aun cuando la cocina no esté conectada. Una vez restablecida la conexión, la InterfazCocina consulta automáticamente los pedidos pendientes mediante el controlador, recuperando información sin inconsistencias. El diseño evita pérdidas de información porque ninguna capa de presentación almacena datos críticos: todo está centralizado en los modelos y la base de datos.

7.3 Escenario 3: Inventario insuficiente en el momento de registrar una venta.

Situación: En caja se intenta cobrar una orden que incluye un producto cuyo stock se agotó segundos antes por otro pedido.

Estímulo: El AdminController valida la venta y solicita a Inventario verificar disponibilidad.

Respuesta arquitectónica esperada:

El modelo Inventario detecta el stock insuficiente y notifica a NotificationManager, el cual registra el evento en sistema.log y alerta al administrador. La venta no se confirma; el sistema devuelve un mensaje claro a la InterfazCaja indicando el problema. Esto es posible gracias al patrón Observer y a la separación entre la lógica de venta y la lógica de inventario. La arquitectura evita ventas inválidas y mantiene la integridad de datos.

7.4 Escenario 4: Falla del servidor central.

Situación: El servidor local sufre una caída o reinicio inesperado.

Estímulo: Las vistas pierden conexión temporal al backend.

Respuesta arquitectónica esperada:

La arquitectura centralizada garantiza que ninguna información crítica se encuentra en los clientes; no existe riesgo de corrupción de pedidos o inventarios. Tras reiniciarse el servidor, todas las vistas se reconectan automáticamente y consultan el estado actualizado mediante los controladores. El archivo sistema.log permite identificar el momento de la falla y facilita la recuperación del sistema. La simplicidad de un solo nodo central hace que la restauración sea sencilla y predecible.

7.5 Escenario 5: Pedido marcado como pagado sin pasar por caja.

Situación: Un mesero intenta marcar manualmente un pedido como pagado desde su propia interfaz.

Estímulo: Se envía al servidor una acción no autorizada.

Respuesta arquitectónica esperada:

El AuthController, mediante validación de roles, bloquea la acción porque el estado "Pagado" solo puede ser asignado por la InterfazCaja. Este comportamiento se deriva de la arquitectura modular y del control estricto de permisos aplicado en los controladores, lo que evita manipulación indebida del flujo operativo del restaurante.

7.6 Escenario 6: Flujo incorporado tardíamente – Interfaz de Caja.

Situación: Durante el diseño inicial del proyecto, la interfaz de caja no estaba contemplada; los pedidos pasaban del cliente a cocina sin una etapa formal de validación de pago.

Estímulo: Durante la revisión arquitectónica se detecta un riesgo funcional: pedidos no pagados podían cocinarse, generando merma y pérdida económica.

Respuesta arquitectónica esperada:

Se decide integrar la InterfazCaja como nuevo módulo, adicionando un nuevo actor, un nodo en la Vista de Despliegue, y un estado adicional en el modelo de Pedido. La arquitectura demostró capacidad de adaptación porque la modularidad del MVC

permitió agregar el controlador y las vistas correspondientes sin alterar el núcleo del sistema. Esto evidencia la flexibilidad del diseño y la solidez del racional arquitectónico.

8. Conclusiones Finales de la Arquitectura.

La documentación completa de la Descripción de Arquitectura (AD) del Sistema de Comanda Digital (SCD) demuestra que el diseño propuesto es coherente con las necesidades operativas del restaurante, factible de implementar con los recursos y restricciones identificadas, y suficientemente modular para permitir evolución sin comprometer la estabilidad. Las decisiones principales —adopción del patrón MVC, centralización en un servidor local sobre LAN, uso de MySQL, implementación del patrón Observer para notificaciones y la incorporación del módulo de caja— se tomaron con criterios técnicos claros (consistencia, mantenibilidad, coste, disponibilidad y seguridad) y se justificaron frente a alternativas razonables (NoSQL, nube, arquitectura distribuida, etc.).

La arquitectura satisface los requisitos funcionales básicos —toma y personalización de pedidos, gestión de inventario, notificaciones a cocina, control administrativo y generación de comprobantes físicos— y atiende las preocupaciones no funcionales críticas: baja latencia (LAN), trazabilidad y coherencia de datos (ACID en MySQL), mantenibilidad (separación MVC), y tolerancia operativa (servidor local y logs). El registro de decisiones y su racional (Sección 6) aportan la trazabilidad necesaria para comprender por qué cada elección técnica fue tomada y qué impacto tiene en las vistas lógicas, de desarrollo, de procesos y de despliegue.

La evaluación de escenarios (Sección 7) confirma que, desde el punto de vista del diseño, la arquitectura está preparada para manejar situaciones reales relevantes: picos de concurrencia, fallos temporales de conectividad, conflictos de inventario y la recuperación ante caída del servidor. En todos estos casos el diseño asegura que la integridad de los datos no dependa de clientes y que las responsabilidades estén

correctamente aisladas, facilitando la recuperación y la auditoría. Además, la incorporación tardía del módulo de caja ilustra la capacidad de la arquitectura para adaptarse a necesidades operativas detectadas durante el análisis, lo cual es una virtud importante en proyectos con contexto real.

En términos prácticos, esta Descripción de Arquitectura alcanza tres objetivos esenciales: primero, ofrece a los desarrolladores una guía clara y trazable para implementar el SCD de forma coherente con el diseño; segundo, proporciona a los responsables del restaurante y a los evaluadores académicos una visión comprensible y justificable de cómo y por qué opera el sistema; tercero, deja una base documentada para futuras ampliaciones (integración de pagos, analítica avanzada, despliegue en la nube si fuera necesario) sin poner en riesgo la versión inicial del producto.

Recomendaciones finales y próximos pasos: dejar registradas en el repositorio las trazas que vinculen archivos concretos con los elementos del AD (por ejemplo controllers/AdminController.php ⇄ Caso de uso “Gestión de mesas”), completar la sección de referencias y anexos con normas citadas y diagramas fuente (PlantUML) y preparar una breve guía de despliegue e instalación para el servidor local (checklist de hardware mínimo, configuración de red, copia de seguridad y procedimientos de arranque). Estos productos complementarios facilitarán la entrega, la evaluación y la transición a la implementación práctica.